

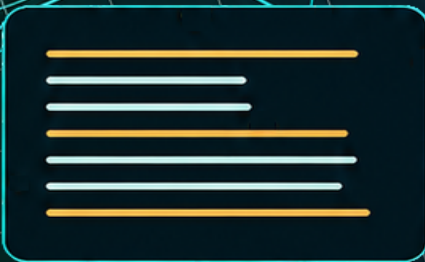
Go Essencial

Volume 2 — Desenvolvimento de APIs

APIs HTTP robustas com Go

Rudson Alves

Julho de 2026



Licenca

Copyright (c) 2026 Rudson Alves.

Este livro utiliza licenca dupla, separando o conteudo editorial do codigo-fonte dos exemplos.

Conteudo do livro

O conteudo textual do livro, imagens, diagramas, arquivos Markdown, PDFs e demais materiais editoriais sao licenciados sob a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0).

Voce pode copiar, compartilhar e adaptar este material, desde que:

- atribua a autoria a Rudson Alves;
- nao utilize o material para fins comerciais sem autorizacao previa;
- distribua obras derivadas sob a mesma licenca.

Texto oficial da licenca:

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

Codigo-fonte dos exemplos

Os exemplos de codigo em Go localizados em `vol_2-desenvolvimento-de-apis/codigo` sao licenciados sob a licenca MIT.

O texto completo da licenca MIT aplicada aos exemplos esta disponivel em:

`vol_2-desenvolvimento-de-apis/codigo/LICENSE`

Agradecimentos

Agradeço a todas as pessoas que apoiaram este projeto, em especial família, amigos e colegas de comunidade que incentivaram a construção desta coleção.

Meu reconhecimento também vai para os profissionais e comunidades que compartilham conhecimento aberto sobre Go, APIs e engenharia de software. Muito do amadurecimento deste volume nasce desse ecossistema colaborativo.

Apresentacao

Este segundo volume da colecao **Go Essencial** foi planejado para conectar os fundamentos da linguagem ao desenvolvimento de APIs usadas em sistemas reais.

Aqui, o foco deixa de ser apenas a sintaxe e passa a incluir decisoes de projeto, organizacao de codigo e praticas operacionais que tornam uma API mais segura, testavel e sustentavel.

O percurso parte de `net/http` e avanca por roteamento, middleware, serializacao, validacao, persistencia, autenticacao, testes, logging, observabilidade e deploy.

A proposta e manter uma trilha progressiva, com exemplos diretos e explicacoes objetivas, para que o material funcione tanto como leitura sequencial quanto como referencia durante o desenvolvimento.

Índice

Licença	iii
Conteúdo do livro	iii
Código-fonte dos exemplos	iii
Agradecimentos	v
Apresentação	vii
1 Prefácio	1
1.1 Objetivos do Livro	1
1.2 Pré-requisitos	1
1.3 Como Utilizar este Livro	2
1.4 O Projeto GoList	3
2 HTTP e net/http	5
2.1 Handler	5
2.1.1 Implementando um Handler	6
2.1.2 Associando o Handler ao Servidor	6
2.1.3 Um Handler para Cada Responsabilidade	7
2.1.4 Considerações	8
2.2 HandlerFunc	8
2.2.1 Utilizando HandlerFunc	8
2.2.2 Utilizando HandleFunc	9
2.2.3 Handler ou HandlerFunc?	10
2.2.4 Considerações	11
2.3 ListenAndServe	11
2.3.1 Iniciando um servidor	11
2.3.2 O parâmetro addr	12
2.3.3 O parâmetro handler	12
2.3.4 Tratando erros	13
2.3.5 Funcionamento	13
2.3.6 Considerações	13
2.4 Request	14
2.4.1 Método HTTP	15
2.4.2 URL	15
2.4.3 Cabeçalhos	15
2.4.4 Corpo da Requisição	16

2.4.5	Outras Informações	16
2.4.6	O Ciclo de Vida da Requisição	17
2.4.7	Considerações	18
2.5	ResponseWriter	18
2.5.1	Escrevendo o Corpo da Resposta	18
2.5.2	Definindo o Código de Status	19
2.5.3	Definindo Cabeçalhos	20
2.5.4	Construindo uma Resposta Completa	20
2.5.5	O Ciclo da Resposta	20
2.5.6	Considerações	21
2.6	Headers	21
2.6.1	Acessando Headers da Requisição	22
2.6.2	Definindo Headers da Resposta	22
2.6.3	Headers Mais Utilizados	23
2.6.4	Quando Definir os Headers	23
2.6.5	Considerações	24
2.7	Status Codes	24
2.7.1	Categorias de Status Codes	24
2.7.2	Principais Códigos de Sucesso	25
2.7.3	Principais Erros do Cliente	25
2.7.4	Erros do Servidor	26
2.7.5	Utilizando as Constantes da Biblioteca	26
2.7.6	Escolhendo o Código Correto	27
2.7.7	Considerações	27
2.8	Métodos HTTP	27
2.8.1	Os Principais Métodos	28
2.8.2	GET	28
2.8.3	POST	28
2.8.4	PUT	29
2.8.5	PATCH	29
2.8.6	DELETE	29
2.8.7	Identificando o Método	30
2.8.8	Métodos e REST	30
2.8.9	Considerações	31
3	Requisições e Respostas	33
3.1	Path Parameters	33
3.1.1	Path Parameters e Recursos	34
3.1.2	Obtendo o Caminho da Requisição	34
3.1.3	Extraindo Parâmetros Manualmente	35
3.1.4	Extraindo Valores Numéricos	36
3.1.5	Boas Práticas	36
3.1.6	Considerações	36
3.2	Query Parameters	37
3.2.1	Quando Utilizar Query Parameters	37
3.2.2	Obtendo os Query Parameters	37
3.2.3	Múltiplos Valores	39
3.2.4	Considerações	39

3.3	Body	39
3.3.1	Acessando o Corpo da Requisição	40
3.3.2	Convertendo o Corpo para uma Estrutura	41
3.3.3	Lendo o Corpo Apenas Uma Vez	42
3.3.4	Tratando Erros	42
3.3.5	Quando Utilizar o Body	43
3.3.6	Considerações	43
3.4	Headers	43
3.4.1	Lendo Headers da Requisição	44
3.4.2	Definindo Headers da Resposta	45
3.4.3	Headers Mais Utilizados	45
3.4.4	Múltiplos Valores	46
3.4.5	Considerações	46
3.5	Cookies	46
3.5.1	Criando um Cookie	46
3.5.2	Lendo um Cookie	47
3.5.3	Configurando um Cookie	48
3.5.4	Cookies e Autenticação	48
3.5.5	Removendo um Cookie	49
3.5.6	Considerações	49
3.6	Content-Type	49
3.6.1	Definindo o Content-Type	50
3.6.2	Obtendo o Content-Type da Requisição	51
3.6.3	Tipos Mais Utilizados	51
3.6.4	Charset	52
3.6.5	Content-Type e APIs HTTP	52
3.6.6	Considerações	52
3.7	Respostas Padronizadas	52
3.7.1	Estrutura Base	53
3.7.2	Respostas de Sucesso	53
3.7.3	Respostas de Erro	54
3.7.4	Criando Estruturas em Go	55
3.7.5	Enviando uma Resposta JSON	55
3.7.6	Criando Funções Auxiliares	56
3.7.7	Considerações	57
3.8	Desafios Preparatórios	57
3.8.1	Desafio 1: Consulta com Query Parameters	57
3.8.2	Desafio 2: Leitura de Body JSON	58
3.8.3	Desafio 3: Validação de Content-Type	59
3.8.4	Desafio 4: Resposta Padronizada	59
3.8.5	Desafio 5: Erros de Validação	60
3.8.6	Testes Sugeridos	60
3.8.7	Considerações	60
4	Roteamento	61
4.1	ServeMux	62
4.2	Métodos HTTP	65
4.3	Agrupamento de rotas	66

4.4	Organização das rotas	69
4.5	Rotas da GoList	71
4.6	Atividade: rotas da GoList	73
5	Json	77
5.1	encoding/json	78
5.1.1	Tipos suportados	79
5.1.2	Exportação dos campos	79
5.1.3	Leitura e escrita	80
5.1.4	Simplicidade da biblioteca padrão	80
5.2	Marshal	81
5.2.1	Convertendo uma struct	81
5.2.2	Convertendo outros tipos	82
5.2.3	JSON formatado	83
5.2.4	Tratando erros	84
5.2.5	Utilização em APIs	84
5.3	Unmarshal	85
5.3.1	Convertendo JSON para uma struct	85
5.3.2	Convertendo listas	86
5.3.3	Convertendo mapas	87
5.3.4	Tratando erros	87
5.3.5	Utilização em APIs	88
5.4	Struct tags	88
5.4.1	Leitura e escrita	89
5.4.2	Ignorando campos	90
5.4.3	Omitindo campos vazios	90
5.4.4	Múltiplas opções	91
5.4.5	Boas práticas	91
5.5	DTOs	92
5.6	Respostas JSON	94
5.6.1	Definindo o tipo do conteúdo	95
5.6.2	Códigos de status	95
5.6.3	Respostas de erro	96
5.6.4	Padronização das respostas	96
6	Validação	99
6.1	Campos obrigatórios	101
6.1.1	Espaços em branco	103
6.1.2	Campos obrigatórios em outros tipos	104
6.1.3	Validação próxima ao DTO	106
6.2	Tipos inválidos	107
6.2.1	Campos desconhecidos	109
6.2.2	Erros de sintaxe	110
6.2.3	Corpo vazio	110
6.2.4	Múltiplos documentos JSON	110
6.2.5	Validação estrutural versus validação de domínio	111
6.3	Regras de domínio	111
6.3.1	Regras dependem da aplicação	112

6.3.2	Regras simples	113
6.3.3	Validações que dependem de outros dados	113
6.3.4	Onde validar?	114
6.3.5	Validação em camadas	115
6.4	Erros de validação	116
6.4.1	Acumulando erros	116
6.4.2	Padronizando a resposta	118
6.4.3	Associando erros aos campos	118
6.4.4	Código de status	120
6.4.5	Boas práticas	120
6.5	Validação de itens e listas	121
6.5.1	Validando parâmetros da rota	123
6.5.2	Fluxo no handler	124
6.5.3	Fluxo completo	125
7	Middleware	127
7.1	Conceito	128
7.1.1	Handlers em Go	129
7.1.2	A forma de um middleware	129
7.1.3	Um exemplo simples	130
7.1.4	Executando código depois do handler	131
7.1.5	Interrompendo a requisição	131
7.1.6	Responsabilidades transversais	132
7.2	Cadeia de middlewares	132
7.2.1	Composição manual	133
7.2.2	Ordem de execução	135
7.2.3	A ordem importa	136
7.2.4	Criando um tipo para middleware	137
7.2.5	Uma função para encadear middlewares	137
7.2.6	Aplicando ao ServeMux	138
7.2.7	Middlewares globais e por rota	139
7.3	Logging	140
7.3.1	Um primeiro middleware de log	140
7.3.2	Medindo duração	141
7.3.3	Registrando o status da resposta	141
7.3.4	Incluindo o endereço remoto	144
7.3.5	Aplicando o middleware globalmente	144
7.3.6	Logging e erros	144
7.3.7	Código completo	145
7.4	Recover	147
7.4.1	O papel do recover	147
7.4.2	Um handler com panic	147
7.4.3	Implementando o middleware	148
7.4.4	Registrando mais contexto	149
7.4.5	Stack trace	150
7.4.6	Resposta JSON	151
7.4.7	Ordem na cadeia	152
7.4.8	Limites do recover	153

7.4.9	Código completo	153
7.5	CORS	154
7.5.1	O problema	155
7.5.2	Access-Control-Allow-Origin	155
7.5.3	Um middleware simples de CORS	156
7.5.4	Métodos permitidos	156
7.5.5	Headers permitidos	157
7.5.6	Requisições preflight	157
7.5.7	Permitindo origens específicas	158
7.5.8	Credenciais	159
7.5.9	CORS não é autenticação	160
7.5.10	Ordem na cadeia	160
7.5.11	Código completo	161
7.6	Request ID	162
7.6.1	O que é um request ID?	163
7.6.2	Gerando um identificador	163
7.6.3	Middleware de request ID	164
7.6.4	Contexto da requisição	164
7.6.5	Chave de contexto	165
7.6.6	Função auxiliar	166
7.6.7	Reaproveitando um ID recebido	166
7.6.8	Integrando com logging	167
7.6.9	Propagando para outros serviços	168
7.6.10	Código completo	169
7.7	Middleware de autenticação	171
7.7.1	Responsabilidade do middleware	171
7.7.2	O header Authorization	172
7.7.3	Extraindo o token	172
7.7.4	Representando o usuário autenticado	173
7.7.5	Middleware básico	174
7.7.6	Usuário no contexto	174
7.7.7	Função auxiliar	175
7.7.8	Aplicando apenas em rotas protegidas	176
7.7.9	401 e 403	177
7.7.10	Resposta JSON	177
7.7.11	Código completo	178
7.8	Atividades: middlewares na GoList	181
7.8.1	Atividade 1: registrar as requisições da API	181
7.8.2	Atividade 2: identificar cada requisição	182
7.8.3	Atividade 3: proteger a API contra falhas inesperadas	182
7.8.4	Atividade 4: permitir o acesso do frontend	183
7.8.5	Atividade 5: separar rotas públicas e protegidas	183
7.8.6	Atividade 6: montar a cadeia final	184
8	Tratamento de Erros em APIs	185
8.1	Erros da aplicação	187
8.1.1	Erros esperados	187
8.1.2	Erros inesperados	188

8.1.3	Erros sentinela	188
8.1.4	Adicionando contexto ao erro	189
8.1.5	Erros com tipo próprio	190
8.1.6	Identificando erros tipados	191
8.1.7	Mensagem interna e mensagem externa	192
8.1.8	No contexto da GoList	192
8.2	Status HTTP	192
8.2.1	Famílias de status	193
8.2.2	400 Bad Request	193
8.2.3	401 Unauthorized	194
8.2.4	403 Forbidden	194
8.2.5	404 Not Found	195
8.2.6	409 Conflict	195
8.2.7	500 Internal Server Error	196
8.2.8	Uma tabela para a GoList	196
8.2.9	Mapeando erros para status	196
8.3	Respostas padronizadas	197
8.3.1	Um formato inicial	198
8.3.2	Código de erro	198
8.3.3	Detalhes adicionais	199
8.3.4	Request ID na resposta	200
8.3.5	Função auxiliar	200
8.3.6	Evitando vazamento de detalhes internos	202
8.3.7	Consistência para o cliente	202
8.4	Erros centralizados	203
8.4.1	Handlers que retornam erro	204
8.4.2	Adaptando para http.Handler	205
8.4.3	A função central de erro	206
8.4.4	Mapeando códigos para status	206
8.4.5	Funções construtoras	207
8.4.6	Validação com detalhes	208
8.4.7	Benefícios da centralização	209
8.4.8	Cuidados ao escrever a resposta	210
8.4.9	Integração com middleware	210
9	Configuração da Aplicação	213
9.0.1	Configuração não é regra de negócio	214
9.0.2	O problema de valores fixos	214
9.0.3	Configuração e segurança	215
9.0.4	Formas comuns de configuração	215
9.0.5	Configuração centralizada	215
9.0.6	Neste capítulo	216
9.1	Variáveis de ambiente	216
9.1.1	Lendo variáveis com os.Getenv	216
9.1.2	Valor padrão	217
9.1.3	Variáveis obrigatórias	218
9.1.4	os.LookupEnv	218
9.1.5	Convertendo tipos	219

9.1.6	Booleanos	220
9.1.7	Usando na GoList	220
9.1.8	Cuidados importantes	221
9.2	Flags	221
9.2.1	O pacote flag	222
9.2.2	flag.Parse	222
9.2.3	Tipos comuns	223
9.2.4	Ajuda automática	224
9.2.5	Flags e variáveis de ambiente	224
9.2.6	Usando FlagSet	225
9.2.7	Flags para a GoList	225
9.2.8	Quando usar flags	227
9.3	Config struct	227
9.3.1	Por que usar uma struct?	228
9.3.2	Definindo a configuração inicial	228
9.3.3	Carregando a Config	229
9.3.4	Validando configuração	230
9.3.5	Valores derivados	231
9.3.6	Passando configuração para componentes	232
9.3.7	Evitando configuração global	232
9.3.8	Testando configuração	233
9.3.9	No contexto da GoList	234
9.4	Ambientes	234
9.4.1	Desenvolvimento	235
9.4.2	Teste	235
9.4.3	Produção	235
9.4.4	Constantes para ambientes	236
9.4.5	Validando o ambiente	236
9.4.6	Métodos auxiliares	237
9.4.7	Ajustando comportamento por ambiente	238
9.4.8	Exigindo configuração em produção	238
9.4.9	Carregando ambiente primeiro	239
9.4.10	Ambientes não substituem configuração	240
9.4.11	No contexto da GoList	241
9.5	Inicialização	241
9.5.1	Carregando e validando	242
9.5.2	Criando o ServeMux	242
9.5.3	Aplicando middlewares	243
9.5.4	Criando o servidor	243
9.5.5	Função NewServer	244
9.5.6	Um main inicial	245
9.5.7	Falhando cedo	245
9.5.8	Logs de inicialização	246
9.5.9	Inicialização e testes	246
9.5.10	No contexto da GoList	246
A	Projeto GoList	249
A.1	Escopo do Projeto	249

A.2	Entidades Iniciais	250
A.3	Evolução do Domínio	250
A.4	Regras de Domínio Iniciais	251
A.5	Recursos HTTP Iniciais	251
A.6	Formato das Respostas	252
A.7	Organização Inicial	253
A.8	Arquitetura do Projeto	253
A.9	Responsabilidades das Camadas	254
A.10	Organização Final Esperada	255
A.11	Persistência	257
A.12	Decisões de Base	257
A.13	Autenticação e Permissões	257
A.14	Eventos, SSE e Observabilidade	258
A.15	Princípios do Projeto	258

Capítulo 1

Prefácio

1.1 Objetivos do Livro

O objetivo deste volume é apresentar, de forma gradual e prática, como desenvolver APIs HTTP utilizando a linguagem Go e sua biblioteca padrão. Ao longo dos capítulos, o leitor aprenderá não apenas a utilizar os recursos oferecidos pelo pacote `net/http`, mas também a compreender os princípios que fundamentam a construção de serviços web modernos.

Diferentemente de uma abordagem baseada em exemplos isolados, todo o conteúdo será desenvolvido em torno de um único projeto. A cada capítulo, novos conceitos serão incorporados à aplicação, permitindo observar como uma API evolui desde uma implementação simples até uma estrutura organizada, testável e preparada para implantação em ambiente de produção.

Além da programação em Go, este livro busca desenvolver uma compreensão sólida dos conceitos envolvidos na comunicação entre clientes e servidores. Serão abordados temas como requisições e respostas HTTP, roteamento, serialização de dados, validação, persistência, autenticação, tratamento de erros, testes, observabilidade e deploy, sempre aplicados em um contexto prático.

Ao concluir a leitura, espera-se que o leitor seja capaz de:

- compreender o funcionamento do protocolo HTTP e dos princípios REST;
- desenvolver APIs utilizando a biblioteca padrão da linguagem Go;
- estruturar aplicações de forma organizada e incremental;
- implementar autenticação, autorização e persistência de dados;
- produzir serviços testáveis, observáveis e preparados para produção;
- utilizar o projeto desenvolvido como base para aplicações reais.

Mais do que apresentar uma sequência de funcionalidades, este livro procura ensinar uma forma de pensar o desenvolvimento de APIs em Go. Sempre que possível, serão privilegiadas soluções simples, idiomáticas e alinhadas à filosofia da linguagem, demonstrando que é possível construir aplicações robustas sem recorrer, necessariamente, a frameworks ou abstrações excessivas.

1.2 Pré-requisitos

Este volume dá continuidade aos conceitos apresentados no **Go Essencial — Volume 1: Fundamentos**. Por esse motivo, espera-se que o leitor já possua familiaridade com a sintaxe da

linguagem Go e com os principais recursos da biblioteca padrão.

Em particular, recomenda-se que o leitor compreenda os seguintes tópicos:

- declaração de variáveis e tipos;
- estruturas de controle de fluxo;
- arrays, slices e maps;
- funções e métodos;
- ponteiros;
- structs e interfaces;
- tratamento de erros;
- packages e módulos;
- utilização básica da biblioteca padrão;
- testes com o pacote `testing`;
- noções de concorrência utilizando goroutines e canais.

Também é desejável possuir conhecimentos básicos sobre o funcionamento da Web, como navegação por meio de um navegador, utilização de URLs e noções gerais sobre comunicação entre aplicações. Entretanto, não é necessário ter experiência prévia no desenvolvimento de APIs ou serviços HTTP, pois esses conceitos serão apresentados desde seus fundamentos.

Para acompanhar os exemplos, o leitor deverá possuir um ambiente de desenvolvimento configurado com uma versão recente da linguagem Go e um editor de código de sua preferência. Os exemplos foram desenvolvidos utilizando apenas recursos disponíveis na distribuição oficial da linguagem, sem dependência de frameworks externos, permitindo sua execução em qualquer sistema operacional suportado pelo Go.

Embora o livro apresente conceitos utilizados em aplicações profissionais, o objetivo não é ensinar um framework específico nem uma arquitetura rígida. Ao contrário, busca-se desenvolver uma compreensão sólida dos mecanismos oferecidos pela linguagem e pela biblioteca padrão, fornecendo uma base que facilitará a adoção de outras ferramentas e tecnologias quando elas se fizerem necessárias.

1.3 Como Utilizar este Livro

Este livro foi concebido para ser estudado de forma sequencial. Cada capítulo introduz novos conceitos que servem de base para os assuntos apresentados nos capítulos seguintes. Embora seja possível consultar tópicos específicos de forma independente, recomenda-se que a leitura seja realizada na ordem proposta.

Os exemplos de código procuram ser pequenos, objetivos e focados em um único conceito por vez. Sempre que possível, utilize-os como ponto de partida para realizar modificações, experimentar novas soluções e observar o comportamento da aplicação. A prática é parte essencial do processo de aprendizagem.

Ao longo da primeira metade do livro, serão desenvolvidas pequenas aplicações independentes, destinadas a demonstrar o funcionamento dos recursos da linguagem e da biblioteca padrão relacionados ao desenvolvimento de serviços HTTP. Esses exemplos têm caráter didático e foram elaborados para facilitar a compreensão de cada tema isoladamente.

Na segunda metade do livro, esses conhecimentos serão reunidos na construção da **GoList**, uma

API para gerenciamento de listas de compras compartilhadas. A aplicação evoluirá gradualmente a cada capítulo, incorporando novas funcionalidades e refinando sua organização à medida que novos conceitos forem apresentados. Dessa forma, o leitor poderá acompanhar não apenas a implementação de recursos individuais, mas também a evolução de um projeto completo.

Sempre que possível, procure implementar os exemplos antes de consultar o código apresentado. Reproduzir os programas, executar testes, modificar comportamentos e investigar eventuais erros contribui significativamente para a consolidação do conhecimento.

Embora ferramentas de inteligência artificial possam ser utilizadas como apoio ao estudo, recomenda-se que elas sejam empregadas principalmente para esclarecer dúvidas conceituais, explicar mensagens de erro ou sugerir melhorias após uma tentativa de implementação. A resolução direta dos exercícios ou a simples reprodução de soluções prontas reduz parte importante do processo de aprendizagem.

Por fim, lembre-se de que o objetivo deste livro não é apenas ensinar a escrever APIs em Go, mas desenvolver uma compreensão sólida dos princípios envolvidos na comunicação entre sistemas. Com esses fundamentos bem estabelecidos, será muito mais fácil compreender outras bibliotecas, frameworks e arquiteturas que poderão ser explorados ao longo de sua carreira.

1.4 O Projeto GoList

Ao longo deste volume, utilizaremos uma aplicação chamada **GoList** como projeto-base para demonstrar a construção de APIs HTTP em Go. A GoList é uma API para gerenciamento de listas de compras compartilhadas, permitindo que usuários criem listas, adicionem itens e compartilhem essas listas com outras pessoas.

A escolha desse domínio é intencional. Uma lista de compras é simples o bastante para ser compreendida sem grande esforço, mas suficientemente rica para representar problemas comuns em aplicações reais. A partir dela, será possível estudar cadastro de usuários, autenticação, permissões, persistência de dados, upload de arquivos, eventos em tempo real, webhooks, testes, logs, métricas e deploy.

A GoList não será apresentada pronta desde o início. Sua estrutura evoluirá gradualmente, acompanhando os conceitos estudados em cada capítulo. Nos primeiros momentos, trabalharemos com exemplos menores e independentes. Depois, esses conhecimentos serão reunidos na implementação da aplicação principal.

Inicialmente, o domínio da GoList será composto por três entidades centrais:

```
User
└─ participa de uma ou mais listas

ShoppingList
└─ possui vários itens

ShoppingItem
└─ representa um item da lista
```

Com o avanço do livro, novas entidades serão incorporadas, como membros, anexos, eventos e

webhooks. Essa evolução permitirá introduzir novos conceitos apenas quando eles forem necessários, evitando uma arquitetura excessivamente complexa logo no início.

Alguns recursos previstos para a API incluem:

```
POST    /users
GET      /users/{id}

POST    /lists
GET      /lists
GET      /lists/{id}
PUT      /lists/{id}
DELETE   /lists/{id}

POST    /lists/{id}/items
PATCH   /items/{id}
DELETE   /items/{id}
```

Mais adiante, a aplicação também passará a incluir autenticação, autorização por lista, upload de imagens para itens, notificações em tempo real com Server-Sent Events, envio de webhooks, persistência em banco de dados relacional, testes automatizados, logging estruturado, métricas e implantação em ambiente de produção.

O objetivo da GoList não é servir como produto final completo, nem cobrir todos os detalhes possíveis de uma aplicação comercial. Seu papel é didático: oferecer um contexto contínuo no qual os conceitos apresentados no livro possam ser aplicados de forma progressiva e coerente.

Com isso, o leitor poderá acompanhar a transformação de uma API simples em uma aplicação mais próxima de um projeto profissional, compreendendo não apenas como escrever handlers HTTP, mas também como organizar responsabilidades, lidar com erros, validar dados, proteger recursos, persistir informações e preparar um serviço para execução em produção.

Capítulo 2

HTTP e net/http

Toda API HTTP é construída sobre um conjunto relativamente pequeno de conceitos fundamentais. Antes de lidar com autenticação, persistência de dados, testes ou integração com outros serviços, é necessário compreender como um servidor recebe requisições, encaminha o processamento e produz respostas para os clientes.

A biblioteca padrão da linguagem Go oferece os recursos necessários para esse trabalho por meio do pacote `net/http`. Esse pacote implementa as principais funcionalidades do protocolo HTTP, permitindo criar servidores, tratar requisições e construir respostas sem a necessidade de bibliotecas externas.

Uma das características mais marcantes de `net/http` é sua simplicidade. Com poucas linhas de código é possível iniciar um servidor funcional e responder a requisições HTTP. Ao mesmo tempo, essa simplicidade não limita a construção de aplicações robustas: a mesma biblioteca usada em exemplos introdutórios serve como base para aplicações executadas em ambientes de produção.

Neste capítulo serão apresentados os principais elementos que compõem uma aplicação HTTP em Go. Veremos o papel dos handlers, a diferença entre `Handler` e `HandlerFunc`, como iniciar um servidor com `ListenAndServe`, como acessar dados por meio de `http.Request` e como construir respostas usando `http.ResponseWriter`. Também estudaremos headers, status codes e métodos HTTP.

Embora os exemplos sejam intencionalmente simples, eles introduzem conceitos que servirão de base para todo o restante deste volume. Compreender esses componentes é essencial para acompanhar a evolução da **GoList**, que começará a ganhar forma nos próximos capítulos.

Ao final deste capítulo, o leitor será capaz de criar um servidor HTTP usando apenas a biblioteca padrão, compreender o ciclo básico de uma requisição e reconhecer os elementos fundamentais envolvidos na comunicação entre clientes e servidores.

2.1 Handler

Toda aplicação HTTP possui um objetivo fundamental: receber uma requisição, processá-la e produzir uma resposta. Em Go, a parte do programa responsável por esse processamento é chamada de **handler**.

Um *handler* é qualquer componente capaz de tratar uma requisição HTTP. Ele recebe as informações enviadas pelo cliente, executa a lógica necessária e escreve a resposta que será retornada. Em uma API, praticamente todas as operações passam por handlers.

A biblioteca padrão define esse conceito por meio da interface `http.Handler`:

```
type Handler interface {  
    ServeHTTP(ResponseWriter, *Request)  
}
```

Essa interface possui apenas um método, `ServeHTTP`, que é chamado pelo servidor HTTP sempre que uma requisição precisa ser processada por aquele handler.

Os dois parâmetros representam os elementos centrais da comunicação HTTP:

- `ResponseWriter`: utilizado para construir a resposta enviada ao cliente;
- `*Request`: contém todas as informações da requisição recebida.

A simplicidade dessa interface é uma das características marcantes da biblioteca `net/http`. Qualquer tipo que implemente o método `ServeHTTP` se torna automaticamente um handler, sem necessidade de herança, anotações ou registros especiais.

2.1.1 Implementando um Handler

O exemplo a seguir implementa um handler que responde com uma mensagem simples.

```
1 package main  
2  
3 import (  
4     "fmt"  
5     "net/http"  
6 )  
7  
8 type HelloHandler struct{}  
9  
10 func (HelloHandler) ServeHTTP(w http.ResponseWriter, r *http.Request) {  
11     fmt.Fprintln(w, "Olá, mundo!")  
12 }
```

A estrutura `HelloHandler` não possui campos, mas implementa o método `ServeHTTP`. Isso é suficiente para satisfazer a interface `http.Handler`.

O método será executado sempre que o servidor encaminhar uma requisição para esse handler. O handler, por si só, não abre uma porta nem escuta conexões; ele apenas define como uma requisição será tratada quando chegar até ele.

2.1.2 Associando o Handler ao Servidor

Depois de criar um handler, é necessário associá-lo a um servidor HTTP.


```
1 package main
2
3 import (
4     "fmt"
5     "net/http"
6 )
7
8 type HelloHandler struct{}
9
10 func (HelloHandler) ServeHTTP(w http.ResponseWriter, r *http.Request) {
11     fmt.Fprintln(w, "Olá, mundo!")
12 }
13
14 func main() {
15     handler := HelloHandler{}
16
17     http.ListenAndServe(":8080", handler)
18 }
```

Ao executar esse programa, `ListenAndServe` inicia um servidor na porta 8080 e usa `handler` para processar as requisições recebidas. Como `HelloHandler` implementa `http.Handler`, ele pode ser passado diretamente como segundo argumento da função.

Toda requisição recebida será encaminhada ao método `ServeHTTP`, que responderá com:

```
Olá, mundo!
```

Como o servidor recebeu apenas esse handler, qualquer caminho solicitado pelo cliente produzirá a mesma resposta.

2.1.3 Um Handler para Cada Responsabilidade

Embora seja possível construir uma aplicação inteira utilizando um único handler, essa abordagem rapidamente se torna difícil de manter. Em aplicações reais, é comum que cada recurso da API tenha seu próprio handler ou conjunto de handlers.

Por exemplo:

```
UserHandler
ListHandler
ItemHandler
AuthHandler
```

Cada um deles fica responsável pelas operações relacionadas ao seu domínio, favorecendo uma organização mais clara do código e facilitando a manutenção.

Nas próximas seções veremos como associar diferentes handlers a diferentes rotas, permitindo que cada URL da aplicação execute uma lógica específica.

2.1.4 Considerações

O conceito de handler é a base sobre a qual toda a biblioteca net/http foi construída. Antes mesmo de compreender roteamento, middlewares ou autenticação, é importante entender que toda requisição HTTP termina sendo processada por um objeto que implementa a interface `http.Handler`.

Essa decisão de projeto torna a biblioteca extremamente flexível. Como handlers são apenas implementações de uma interface simples, eles podem ser reutilizados, combinados e decorados com facilidade, formando a base para recursos mais avançados que serão apresentados ao longo deste livro.

2.2 HandlerFunc

Implementar a interface `http.Handler` é simples e oferece grande flexibilidade. Ainda assim, em muitas situações, criar uma estrutura apenas para implementar um único método adiciona mais código do que o necessário.

Para esses casos, a biblioteca net/http disponibiliza o tipo `http.HandlerFunc`, que permite usar funções comuns como handlers HTTP.

Sua definição é a seguinte:

```
type HandlerFunc func(ResponseWriter, *Request)
```

Embora `HandlerFunc` seja um tipo de função, ele também implementa a interface `http.Handler`. Isso significa que qualquer função com essa assinatura pode ser adaptada para se tornar um handler válido.

Sua implementação mostra bem essa adaptação:

```
func (f HandlerFunc) ServeHTTP(w ResponseWriter, r *Request) {  
    f(w, r)  
}
```

Quando o servidor chama o método `ServeHTTP`, a função representada por `HandlerFunc` é executada. Esse detalhe permite usar funções no lugar de estruturas quando o handler não precisa armazenar estado próprio.

2.2.1 Utilizando HandlerFunc

O exemplo a seguir cria um servidor HTTP usando uma função comum como handler.

```
1 package main
2
3 import (
4     "fmt"
5     "net/http"
6 )
7
8 func hello(w http.ResponseWriter, r *http.Request) {
9     fmt.Fprintln(w, "Olá, mundo!")
10 }
11
12 func main() {
13     http.ListenAndServe(":8080", http.HandlerFunc(hello))
14 }
```

Nesse exemplo, a função `hello` é convertida para `http.HandlerFunc`. Como `HandlerFunc` implementa `http.Handler`, o valor resultante pode ser passado diretamente para `ListenAndServe`.

Embora essa conversão explícita ajude a entender o mecanismo, ela raramente aparece em aplicações reais. Na maioria dos casos, registramos a função diretamente em uma rota.

2.2.2 Utilizando HandlerFunc

A biblioteca padrão oferece uma forma ainda mais conveniente de registrar funções como handlers em rotas específicas: `http.HandleFunc`.

```
1 package main
2
3 import (
4     "fmt"
5     "net/http"
6 )
7
8 func hello(w http.ResponseWriter, r *http.Request) {
9     fmt.Fprintln(w, "Olá, mundo!")
10 }
11
12 func message(w http.ResponseWriter, r *http.Request) {
13     fmt.Fprintln(w, "Página de documentação")
14 }
15
16 func main() {
17     http.HandleFunc("/", hello)
18     http.HandleFunc("/doc", message)
19
20     http.ListenAndServe(":8080", nil)
21 }
```

Nesse caso, `HandleFunc` converte cada função para `HandlerFunc` automaticamente e a registra no roteador padrão da biblioteca (`DefaultServeMux`). A função `hello` fica associada ao caminho `/`, enquanto `message` fica associada ao caminho `/doc`. No roteador padrão, o caminho `/` também funciona como fallback para requisições que não encontram um handler mais específico.

Ao chamar `ListenAndServe` com `nil` como segundo argumento, o servidor utiliza esse roteador padrão. Assim, uma requisição para `http://localhost:8080/` executa `hello`, enquanto uma requisição para `http://localhost:8080/doc` executa `message`.

Essa combinação entre `HandleFunc` e `ListenAndServe` é uma das formas mais comuns de iniciar uma aplicação HTTP pequena em Go.

2.2.3 Handler ou HandlerFunc?

As duas abordagens produzem handlers compatíveis com `http.Handler`, mas cada uma se encaixa melhor em um tipo de situação.

Implementar a interface `http.Handler` costuma ser mais adequado quando o handler possui estado próprio ou quando suas dependências devem ficar explícitas em uma estrutura.

```
type UserHandler struct {
    service UserService
}
```

Nesse caso, o handler pode armazenar dependências como serviços, repositórios ou configurações.

Já `HandlerFunc` é uma boa escolha para handlers simples, principalmente em exemplos, protótipos ou aplicações pequenas, onde uma função é suficiente para processar a requisição.

2.2.4 Considerações

É importante compreender que `HandlerFunc` **não substitui** a interface `http.Handler`. Na realidade, ele existe justamente para implementá-la de forma conveniente.

Essa decisão de projeto demonstra uma característica recorrente da biblioteca padrão de Go: oferecer atalhos úteis sem esconder os conceitos fundamentais.

Nas próximas seções, utilizaremos predominantemente funções como handlers por serem mais concisas. À medida que a aplicação **GoList** evoluir, também usaremos estruturas que implementam `http.Handler`, permitindo organizar melhor as responsabilidades e injetar dependências de forma explícita.

2.3 ListenAndServe

Depois de implementar um ou mais handlers, é necessário iniciar um servidor para receber conexões HTTP e encaminhar cada requisição ao código responsável por tratá-la. Na biblioteca padrão, essa tarefa é realizada pela função `http.ListenAndServe`.

Sua assinatura pode ser representada assim:

```
func ListenAndServe(addr string, handler http.Handler) error
```

Ela recebe dois argumentos:

- `addr`: endereço em que o servidor ficará disponível;
- `handler`: handler usado como ponto de entrada para as requisições recebidas.

Ao ser executada, a função cria um servidor HTTP, passa a escutar conexões no endereço informado e mantém o programa bloqueado enquanto o servidor estiver em execução. Ela só retorna quando ocorre algum erro ao iniciar ou manter o servidor.

2.3.1 Iniciando um servidor

O exemplo a seguir inicia um servidor na porta 8080 e usa a função `hello` para responder às requisições.

```
1 package main
2
3 import (
4     "fmt"
5     "net/http"
6 )
7
8 func hello(w http.ResponseWriter, r *http.Request) {
9     fmt.Fprintln(w, "Olá, mundo!")
10 }
11
12 func main() {
13     http.ListenAndServe(":8080", http.HandlerFunc(hello))
14 }
```

Após executar o programa, o servidor passa a aceitar requisições em:

`http://localhost:8080`

Ao acessar esse endereço em um navegador ou com uma ferramenta como `curl`, a requisição será encaminhada para `hello`.

2.3.2 O parâmetro `addr`

O primeiro parâmetro define o endereço em que o servidor ficará disponível.

Alguns exemplos comuns são:

```
":8080"
```

Escuta conexões na porta 8080 em todas as interfaces de rede da máquina.

```
"localhost:8080"
```

Aceita conexões apenas da própria máquina, usando o nome `localhost`.

```
"127.0.0.1:3000"
```

Também restringe o acesso à máquina local, usando o endereço de loopback `127.0.0.1` e a porta 3000.

Em ambientes de produção, é comum usar portas como 80 (HTTP) e 443 (HTTPS). Durante o desenvolvimento, normalmente usamos portas acima de 1024, como 8080 ou 3000, que não exigem privilégios administrativos na maioria dos sistemas operacionais.

2.3.3 O parâmetro `handler`

O segundo parâmetro informa qual handler será usado como entrada da aplicação HTTP.

Quando a aplicação possui apenas um handler principal, ele pode ser informado diretamente:

```
http.ListenAndServe(":8080", http.HandlerFunc(hello))
```

Também existe um comportamento especial bastante usado em exemplos e aplicações pequenas.

Se o valor informado for `nil`, `ListenAndServe` usa automaticamente o roteador padrão da biblioteca, chamado `http.DefaultServeMux`.

```
http.HandleFunc("/", hello)

http.ListenAndServe(":8080", nil)
```

Nesse caso, as rotas registradas previamente por funções como `http.Handle` e `http.HandleFunc` serão usadas para processar as requisições.

Esse padrão aparecerá com frequência nos exemplos iniciais deste livro.

2.3.4 Tratando erros

A função `ListenAndServe` retorna um valor do tipo `error`.

Na prática, ela permanece bloqueada enquanto o servidor está em execução. Se a porta não puder ser aberta, se o endereço for inválido ou se ocorrer alguma falha durante a execução, a função retorna um erro.

Por esse motivo, é recomendável tratar o erro retornado, especialmente em programas reais.

```
func main() {
    http.HandleFunc("/", hello)

    if err := http.ListenAndServe(":8080", nil); err != nil {
        panic(err)
    }
}
```

Um erro comum ocorre quando a porta escolhida já está sendo usada por outra aplicação.

2.3.5 Funcionamento

O ciclo de execução de `ListenAndServe` pode ser resumido assim:

Esse processo se repete continuamente enquanto o servidor permanece em execução.

2.3.6 Considerações

`ListenAndServe` é uma das principais portas de entrada para aplicações HTTP desenvolvidas com a biblioteca padrão. Apesar da interface simples, ela reúne o trabalho necessário para criar um servidor, aceitar conexões, interpretar requisições HTTP e encaminhá-las ao handler apropriado.

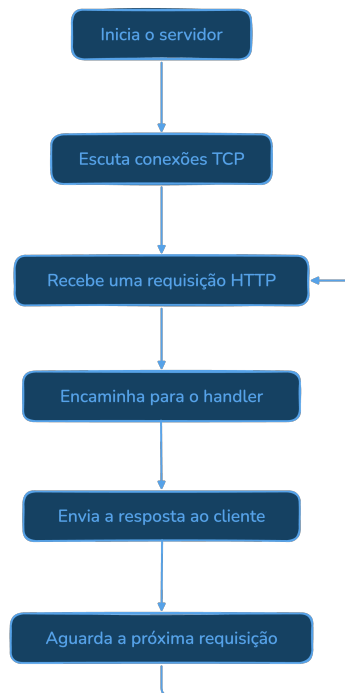


Figura 2.1: Ciclo de Execução do ListenAndServe

Nas próximas seções, utilizaremos essa função em praticamente todos os exemplos, adicionando gradualmente novos handlers, rotas e recursos até construir a API da **GoList**.

2.4 Request

Sempre que um cliente faz uma requisição a um servidor HTTP, diversas informações acompanham essa chamada. Além do caminho solicitado, a requisição pode conter o método HTTP utilizado, cabeçalhos, parâmetros de consulta, cookies e, em alguns casos, um corpo com dados enviados pelo cliente.

Na biblioteca `net/http`, essas informações são representadas pela estrutura `http.Request`.

De forma simplificada, alguns de seus campos principais são:

```
type Request struct {  
    Method string  
    URL    *url.URL  
    Header http.Header  
    Body   io.ReadCloser  
  
    ...  
}
```

A estrutura completa possui diversos outros campos e métodos, mas esses elementos já são suficientes para compreender o funcionamento das primeiras aplicações HTTP.

Sempre que um handler é executado, o servidor fornece um ponteiro para uma instância de `http.Request`.

```
func hello(w http.ResponseWriter, r *http.Request) {  
    // utiliza a requisição  
}
```

É por meio da variável `r` que o handler consulta as informações da requisição recebida.

2.4.1 Método HTTP

O campo `Method` informa qual método HTTP foi usado na requisição.

```
func hello(w http.ResponseWriter, r *http.Request) {  
    fmt.Println(r.Method)  
}
```

Se o cliente fizer uma requisição usando GET, a saída será:

```
GET
```

Nas próximas seções veremos como usar esse campo para diferenciar operações como criação, consulta, atualização e remoção de recursos.

2.4.2 URL

O campo `URL` contém informações sobre o endereço solicitado pelo cliente.

```
func hello(w http.ResponseWriter, r *http.Request) {  
    fmt.Println(r.URL.Path)  
}
```

Se a requisição for realizada para:

```
http://localhost:8080/users
```

a saída será:

```
/users
```

Além do caminho, URL também permite acessar parâmetros de consulta (*query parameters*), assunto que será estudado em detalhes no próximo capítulo.

2.4.3 Cabeçalhos

Os cabeçalhos enviados pelo cliente ficam disponíveis no campo `Header`.

```
func hello(w http.ResponseWriter, r *http.Request) {  
    fmt.Println(r.Header.Get("User-Agent"))  
}
```

Ao acessar a aplicação por meio de um navegador, uma saída semelhante a esta poderá ser produzida:

```
Mozilla/5.0 ...
```

Os cabeçalhos permitem transmitir informações adicionais sobre a requisição, como tipo de conteúdo, credenciais de autenticação, idioma preferencial e diversas outras configurações.

2.4.4 Corpo da Requisição

Quando o cliente envia dados no corpo da requisição, eles ficam disponíveis no campo `Body`. Esse campo representa um fluxo de leitura, não uma string pronta.

```
func hello(w http.ResponseWriter, r *http.Request) {  
    data, err := io.ReadAll(r.Body)  
    if err != nil {  
        http.Error(w, err.Error(), http.StatusInternalServerError)  
        return  
    }  
  
    fmt.Println(string(data))  
}
```

Se o corpo da requisição contiver:

```
Nome=Maria
```

esse conteúdo poderá ser lido e processado pela aplicação. Em geral, o corpo da requisição é consumido uma vez durante o processamento do handler.

Nas seções dedicadas ao formato JSON veremos como converter esse corpo para estruturas da linguagem Go e como lidar com entradas maiores de forma mais controlada.

2.4.5 Outras Informações

Além dos campos apresentados, `http.Request` oferece diversos outros recursos úteis.

Alguns dos mais utilizados são:

- `Host`: host informado pelo cliente;
- `RemoteAddr`: endereço remoto da conexão;
- `Cookie()`: recupera cookies enviados pelo cliente;
- `Context()`: obtém o contexto associado à requisição;
- `FormValue()`: acessa dados enviados por formulários HTML.

Esses recursos serão explorados gradualmente ao longo do livro, conforme surgirem situações em que seu uso se torne necessário.

2.4.6 O Ciclo de Vida da Requisição

Sempre que um cliente envia uma requisição, o fluxo pode ser resumido assim:

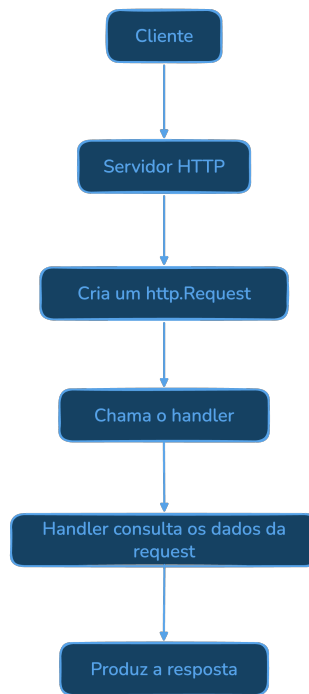
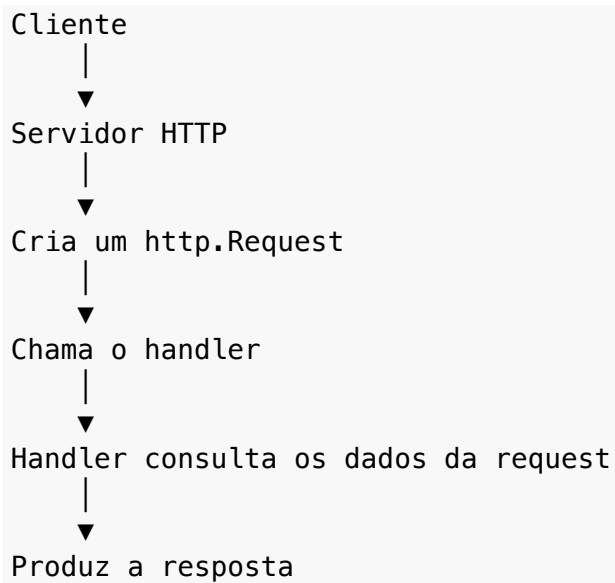


Figura 2.2: Fluxo de um Requisição



O objeto `http.Request` representa uma requisição específica. Depois que a resposta é enviada, aquele valor deixa de ser usado pelo servidor. Uma nova requisição receberá uma nova instância de `http.Request`.

2.4.7 Considerações

A estrutura `http.Request` representa aquilo que o cliente comunicou ao servidor em uma requisição específica. É por meio dela que um handler descobre qual recurso foi solicitado, quais dados foram enviados e quais informações adicionais acompanham a chamada.

Embora possua muitos campos e métodos, na prática a maior parte das aplicações utiliza apenas um subconjunto deles. A biblioteca `net/http` organiza esses recursos de forma que cada aplicação acesse apenas aquilo de que realmente precisa.

2.5 ResponseWriter

Se `http.Request` representa as informações que o cliente envia ao servidor, `http.ResponseWriter` representa o caminho usado pelo handler para construir a resposta enviada de volta ao cliente.

É por meio desse objeto que um handler escreve o corpo da resposta, define cabeçalhos HTTP e informa o código de status retornado.

De forma simplificada, a interface `http.ResponseWriter` possui os seguintes métodos:

```
type ResponseWriter interface {  
    Header() http.Header  
    Write([]byte) (int, error)  
    WriteHeader(statusCode int)  
}
```

Esses três métodos cobrem as partes essenciais de uma resposta HTTP: cabeçalhos, corpo e código de status.

O servidor fornece uma implementação dessa interface sempre que executa um handler.

```
func hello(w http.ResponseWriter, r *http.Request) {  
    // escreve a resposta  
}
```

O parâmetro `w` é usado durante o processamento da requisição para construir a resposta enviada ao cliente.

2.5.1 Escrevendo o Corpo da Resposta

A forma mais simples de responder a uma requisição é escrever diretamente no `ResponseWriter`.

```
1 package main
2
3 import (
4     "fmt"
5     "net/http"
6 )
7
8 func hello(w http.ResponseWriter, r *http.Request) {
9     fmt.Fprintln(w, "Olá, mundo!")
10 }
```

Nesse exemplo, `fmt.Fprintln` escreve a mensagem no corpo da resposta HTTP.

Ao acessar a aplicação, o cliente receberá um corpo semelhante a:

Olá, mundo!

Internamente, `fmt.Fprintln` utiliza o método `Write` da interface `ResponseWriter`.

Também é possível escrever usando esse método diretamente.

```
func hello(w http.ResponseWriter, r *http.Request) {
    w.Write([]byte("Olá, mundo!"))
}
```

Na prática, ambas as abordagens produzem o mesmo corpo de resposta. Se nenhum código de status tiver sido definido antes da escrita, a biblioteca envia automaticamente `200 OK`.

2.5.2 Definindo o Código de Status

Além do conteúdo da resposta, normalmente é necessário informar o resultado da operação por meio de um código de status HTTP.

Isso é feito usando o método `WriteHeader`.

```
func hello(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusCreated)

    fmt.Fprintln(w, "Usuário criado.")
}
```

Nesse caso, o cliente receberá o código HTTP **201 Created**.

É importante observar que `WriteHeader` deve ser chamado **antes** da escrita do corpo da resposta. Depois que os dados começam a ser enviados ao cliente, o status já foi definido e não pode mais ser alterado.

Se `WriteHeader` não for chamado explicitamente antes de `Write` ou de uma função como `fmt.Fprintln`, a biblioteca envia automaticamente o código:

200 OK

Essa é a resposta padrão quando o handler escreve uma resposta sem definir outro status.

2.5.3 Definindo Cabeçalhos

Os cabeçalhos da resposta são acessados por meio do método `Header()`.

```
func hello(w http.ResponseWriter, r *http.Request) {  
    w.Header().Set("Content-Type", "text/plain")  
  
    fmt.Fprintln(w, "Olá, mundo!")  
}
```

Nesse exemplo, o servidor informa ao cliente que o conteúdo retornado é texto simples.

Da mesma forma que o código de status, os cabeçalhos devem ser definidos antes de `WriteHeader` ou antes da primeira escrita do corpo da resposta.

2.5.4 Construindo uma Resposta Completa

O exemplo a seguir reúne os principais elementos de uma resposta HTTP.

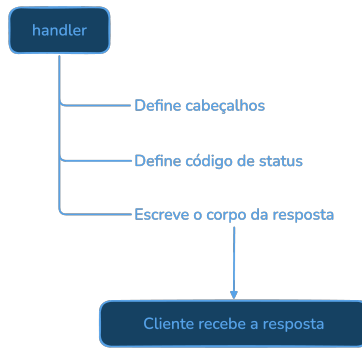
```
func hello(w http.ResponseWriter, r *http.Request) {  
    w.Header().Set("Content-Type", "text/plain")  
    w.WriteHeader(http.StatusOK)  
  
    fmt.Fprintln(w, "Servidor em funcionamento.")  
}
```

A resposta produzida será composta por:

- código de status 200 OK;
- cabeçalho `Content-Type: text/plain`;
- corpo contendo a mensagem "Servidor em funcionamento."

2.5.5 O Ciclo da Resposta

Durante o processamento de uma requisição, o `ResponseWriter` é usado para construir a resposta passo a passo.



Após o envio da resposta, aquele `ResponseWriter` deixa de ser usado. Cada nova requisição receberá seu próprio `ResponseWriter`.

2.5.6 Considerações

`http.ResponseWriter` é o principal mecanismo usado por um handler para se comunicar com o cliente. Apesar da interface simples, ele oferece o necessário para construir respostas HTTP completas.

Nas próximas seções utilizaremos esse recurso para retornar documentos JSON, informar erros de validação, definir cabeçalhos específicos e implementar diferentes códigos de status.

2.6 Headers

Além do corpo da mensagem, requisições e respostas HTTP podem transportar informações adicionais por meio dos **headers**, ou cabeçalhos HTTP. Eles funcionam como metadados: descrevem características da comunicação, informam preferências do cliente ou fornecem instruções sobre como a mensagem deve ser interpretada.

Os headers fazem parte do protocolo HTTP e aparecem em praticamente todas as requisições e respostas.

Uma requisição típica pode ser representada assim:

```
GET /users HTTP/1.1
Host: localhost:8080
Accept: application/json
User-Agent: curl/8.7.1
Authorization: Bearer <token>
```

Nesse exemplo:

- `Host` identifica o servidor de destino;
- `Accept` informa quais formatos de resposta o cliente aceita;
- `User-Agent` identifica a aplicação cliente;
- `Authorization` transporta as credenciais de autenticação.

Da mesma forma, uma resposta HTTP também contém seus próprios headers.

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 48

{"id":1,"name":"Maria"}
```

Nesse caso, o servidor informa que o conteúdo retornado é um documento JSON e indica seu tamanho em bytes.

2.6.1 Acessando Headers da Requisição

Na estrutura `http.Request`, os headers enviados pelo cliente ficam disponíveis no campo `Header`.

```
func hello(w http.ResponseWriter, r *http.Request) {
    fmt.Println(r.Header)
}
```

Esse campo possui o tipo `http.Header`, definido como um mapa de listas de valores.

```
type Header map[string][]string
```

Essa representação permite que um mesmo header possua mais de um valor, conforme previsto pelo protocolo HTTP.

Na maioria das situações, entretanto, basta usar o método `Get`.

```
func hello(w http.ResponseWriter, r *http.Request) {
    userAgent := r.Header.Get("User-Agent")

    fmt.Println(userAgent)
}
```

Se o cliente enviar o cabeçalho:

```
User-Agent: curl/8.7.1
```

a variável `userAgent` conterá:

```
curl/8.7.1
```

Caso o header não exista, `Get` retorna uma string vazia. Em geral, prefira `Get` em vez de acessar o mapa diretamente, pois ele lida melhor com a forma canônica dos nomes de headers.

2.6.2 Definindo Headers da Resposta

Os headers enviados pelo servidor são configurados por meio do método `Header()` do `ResponseWriter`.


```
func hello(w http.ResponseWriter, r *http.Request) {  
    w.Header().Set("Content-Type", "text/plain")  
  
    fmt.Fprintln(w, "Olá, mundo!")  
}
```

Nesse exemplo, o cliente receberá a resposta com o seguinte header:

Content-Type: text/plain

O método Set define o valor do header, substituindo qualquer valor anterior associado ao mesmo nome.

Também é possível usar Add quando for necessário manter valores anteriores e adicionar mais um valor ao mesmo header.

```
w.Header().Add("Cache-Control", "no-cache")  
w.Header().Add("Cache-Control", "no-store")
```

2.6.3 Headers Mais Utilizados

Embora o protocolo HTTP defina muitos headers, alguns aparecem com bastante frequência no desenvolvimento de APIs.

Header	Finalidade
Content-Type	Informa o formato do conteúdo enviado.
Accept	Indica os formatos aceitos pelo cliente.
Authorization	Envia credenciais de autenticação.
User-Agent	Identifica a aplicação cliente.
Host	Identifica o servidor de destino.
Content-Length	Indica o tamanho do corpo da mensagem.
Cache-Control	Define políticas de cache.

Ao longo deste livro, outros headers serão apresentados conforme se tornarem necessários.

2.6.4 Quando Definir os Headers

Assim como o código de status, os headers da resposta devem ser definidos antes do envio do corpo.

```
func hello(w http.ResponseWriter, r *http.Request) {  
    w.Header().Set("Content-Type", "application/json")  
    w.WriteHeader(http.StatusOK)  
  
    fmt.Fprintln(w, `{"status":"ok"}`)  
}
```

Após a primeira chamada a `WriteHeader` ou após a primeira escrita no corpo da resposta, os headers são enviados ao cliente e não podem mais ser alterados para aquela resposta.

2.6.5 Considerações

Headers são um dos mecanismos mais importantes do protocolo HTTP. Eles permitem transportar metadados sem misturá-los ao conteúdo da mensagem, tornando a comunicação entre cliente e servidor mais flexível e padronizada.

Embora muitos exemplos deste capítulo utilizem apenas `Content-Type`, nas próximas seções veremos o uso de outros headers, como `Accept` para negociação de conteúdo, `Authorization` para autenticação, `Origin` e `Access-Control-Allow-Origin` para CORS, além de headers personalizados usados para rastreamento e observabilidade.

2.7 Status Codes

Ao processar uma requisição HTTP, não basta enviar apenas o corpo da resposta. O servidor também precisa informar ao cliente o resultado da operação realizada. Essa informação é transmitida por meio dos **status codes**, ou códigos de status HTTP.

Cada código possui um significado bem definido e permite que clientes, navegadores e outras aplicações compreendam rapidamente se a requisição foi processada com sucesso, se exige alguma ação adicional ou se ocorreu algum tipo de erro.

Por exemplo, uma requisição bem-sucedida pode produzir uma resposta como esta:

```
HTTP/1.1 200 OK
Content-Type: text/plain
```

Olá, mundo!

Nesse caso, o código 200 informa que a operação foi concluída com sucesso.

Já uma tentativa de acessar um recurso inexistente pode resultar em:

```
HTTP/1.1 404 Not Found
Content-Type: text/plain
```

Página não encontrada.

Embora ambas as respostas tenham corpo, é o código de status que comunica ao cliente o resultado geral da operação.

2.7.1 Categorias de Status Codes

Os códigos HTTP são organizados em cinco categorias, identificadas pelo primeiro dígito.

Categoria	Significado
1xx	Respostas informativas.
2xx	Operação realizada com sucesso.

Categoria	Significado
3xx	Redirecionamentos.
4xx	Problemas relacionados à requisição.
5xx	Problemas ocorridos no servidor.

Na construção de APIs, as categorias 2xx, 4xx e 5xx são as mais utilizadas no dia a dia.

2.7.2 Principais Códigos de Sucesso

O código de sucesso mais comum é 200 OK.

```
w.WriteHeader(http.StatusOK)
```

Entretanto, existem outros códigos apropriados para situações diferentes.

Código	Constante	Utilização
200 OK	http.StatusOK	Requisição processada com sucesso.
201 Created	http.StatusCreated	Um novo recurso foi criado.
204 No Content	http.StatusNoContent	Operação realizada sem corpo na resposta.

Por exemplo, após cadastrar um usuário, normalmente usamos:

```
w.WriteHeader(http.StatusCreated)
```

Já uma operação de exclusão pode retornar:

```
w.WriteHeader(http.StatusNoContent)
```

Nesse caso, nenhum corpo deve ser enviado na resposta. Em Go, isso significa chamar `WriteHeader` e encerrar o handler sem escrever conteúdo no `ResponseWriter`.

2.7.3 Principais Erros do Cliente

Quando a requisição enviada pelo cliente apresenta algum problema, usamos códigos da categoria 4xx.

Os mais frequentes em APIs são:

Código	Constante	Situação
400 Bad Request	http.StatusBadRequest	Requisição inválida.
401 Unauthorized	http.StatusUnauthorized	Autenticação ausente ou inválida.
403 Forbidden	http.StatusForbidden	Acesso recusado ao recurso.
404 Not Found	http.StatusNotFound	Recurso inexistente.
409 Conflict	http.StatusConflict	Conflito de estado da aplicação.

Código	Constante	Situação
--------	-----------	----------

Por exemplo, caso um usuário solicite uma lista inexistente, a resposta pode ser:

```
http.Error(w, "Lista não encontrada.", http.StatusNotFound)
```

A função `http.Error` define o código de status e escreve uma mensagem simples no corpo da resposta. Ela é útil para exemplos e erros simples; em APIs JSON, construiremos respostas de erro mais padronizadas nos próximos capítulos.

2.7.4 Erros do Servidor

Quando o problema ocorre durante o processamento no servidor, usamos códigos da categoria 5xx.

O mais comum é:

Código	Constante	Situação
500 Internal Server Error	<code>http.StatusInternalServerError</code>	Erro inesperado no servidor.

Exemplo:

```
http.Error(w, "Erro interno.", http.StatusInternalServerError)
```

Em aplicações reais, detalhes internos do erro normalmente são registrados em logs, enquanto o cliente recebe apenas uma mensagem genérica.

2.7.5 Utilizando as Constantes da Biblioteca

Embora seja possível informar diretamente o valor numérico do código, a biblioteca `net/http` disponibiliza constantes para os status HTTP.

```
http.StatusOK  
http.StatusCreated  
http.StatusNoContent  
http.StatusBadRequest  
http.StatusUnauthorized  
http.StatusForbidden  
http.StatusNotFound  
http.StatusConflict  
http.StatusInternalServerError
```

O uso dessas constantes torna o código mais legível e reduz a possibilidade de erros.

Por exemplo:

```
w.WriteHeader(http.StatusCreated)
```

é mais claro do que:

```
w.WriteHeader(201)
```

2.7.6 Escolhendo o Código Correto

A escolha do código de status deve refletir o resultado da operação realizada. Como vimos na seção sobre `ResponseWriter`, o status precisa ser definido antes da primeira escrita no corpo da resposta.

Por exemplo:

Operação	Código recomendado
Consultar um recurso existente	200 OK
Criar um novo recurso	201 Created
Atualizar um recurso	200 OK ou 204 No Content
Remover um recurso	204 No Content
Dados inválidos enviados pelo cliente	400 Bad Request
Recurso não encontrado	404 Not Found
Erro inesperado da aplicação	500 Internal Server Error

Usar corretamente os códigos HTTP torna a API mais previsível e facilita sua integração com outras aplicações.

2.7.7 Considerações

Status codes são uma parte fundamental da comunicação HTTP. Eles permitem que o cliente compreenda rapidamente o resultado de uma operação, independentemente do conteúdo do corpo da resposta.

Ao longo deste livro, utilizaremos constantemente esses códigos para representar situações como criação de recursos, validação de dados, autenticação, autorização e tratamento de erros. Escolher o código adequado para cada cenário é uma das práticas mais importantes no desenvolvimento de APIs bem projetadas.

2.8 Métodos HTTP

O protocolo HTTP define um conjunto de métodos que indicam a intenção de uma requisição. Em outras palavras, além de informar qual recurso deseja acessar, o cliente também especifica qual operação pretende realizar sobre esse recurso.

Uma mesma URL pode representar operações diferentes dependendo do método utilizado.

Por exemplo:

```
GET    /users
POST   /users
```

Embora ambas as requisições usem o mesmo caminho (`/users`), seus objetivos são distintos. A primeira consulta usuários existentes, enquanto a segunda solicita a criação de um novo usuário.

Essa separação entre recurso e operação é uma das ideias centrais no desenho de APIs HTTP.

2.8.1 Os Principais Métodos

Embora o protocolo HTTP defina diversos métodos, alguns aparecem com muito mais frequência no desenvolvimento de APIs.

Método	Finalidade
GET	Consultar recursos.
POST	Criar novos recursos.
PUT	Atualizar completamente um recurso.
PATCH	Atualizar parcialmente um recurso.
DELETE	Remover um recurso.
HEAD	Obter apenas os cabeçalhos da resposta.
OPTIONS	Consultar os métodos suportados pelo servidor.

Ao longo deste livro, concentraremos nossos estudos principalmente nos métodos GET, POST, PUT, PATCH e DELETE.

2.8.2 GET

O método GET é usado para recuperar informações do servidor.

Exemplos:

```
GET /users
GET /users/10
GET /lists
GET /lists/3
```

Em uma API HTTP bem desenhada, requisições GET não devem modificar o estado da aplicação. Seu objetivo é apenas consultar informações. Por isso, GET é considerado um método seguro.

2.8.3 POST

O método POST normalmente é usado para criar novos recursos.

Por exemplo:

```
POST /users
POST /lists
POST /lists/5/items
```

Nessas requisições, os dados do novo recurso normalmente são enviados no corpo da mensagem. Após uma criação bem-sucedida, é comum retornar o código:

```
201 Created
```

2.8.4 PUT

O método PUT representa a substituição completa de um recurso existente.

Exemplo:

```
PUT /users/10
```

Nesse caso, espera-se que o cliente envie uma representação completa do recurso.

Embora muitas APIs usem PUT para qualquer tipo de atualização, sua semântica mais precisa indica a substituição integral do recurso. Repetir a mesma requisição PUT deve produzir o mesmo estado final.

2.8.5 PATCH

Quando apenas parte das informações precisa ser modificada, normalmente usamos o método PATCH.

Exemplo:

```
PATCH /users/10
```

Se apenas o endereço de e-mail de um usuário for alterado, não há necessidade de reenviar todos os demais campos.

Essa abordagem reduz o volume de dados transmitidos e torna a intenção da requisição mais clara.

2.8.6 DELETE

O método DELETE solicita a remoção de um recurso.

Por exemplo:

```
DELETE /lists/5  
DELETE /items/18
```

Após uma remoção bem-sucedida, muitas APIs retornam:

```
204 No Content
```

indicando que a operação foi realizada e que não há conteúdo adicional a ser enviado. Assim como PUT, DELETE normalmente é tratado como idempotente: repetir a mesma requisição não deveria produzir efeitos adicionais depois que o recurso já foi removido.

2.8.7 Identificando o Método

Na biblioteca net/http, o método usado pelo cliente está disponível no campo Method da estrutura http.Request.

```
func handler(w http.ResponseWriter, r *http.Request) {  
    switch r.Method {  
    case http.MethodGet:  
        fmt.Fprintln(w, "Consulta")  
  
    case http.MethodPost:  
        fmt.Fprintln(w, "Criação")  
  
    default:  
        w.Header().Set("Allow", "GET, POST")  
        http.Error(w, "Método não permitido", http.StatusMethodNotAllowed)  
    }  
}
```

A biblioteca também disponibiliza constantes para os métodos definidos pelo protocolo.

```
1 http.MethodGet  
2 http.MethodPost  
3 http.MethodPut  
4 http.MethodPatch  
5 http.MethodDelete  
6 http.MethodHead  
7 http.MethodOptions
```

Assim como ocorre com os códigos de status, é recomendável usar essas constantes em vez de escrever os nomes dos métodos como strings. No exemplo anterior, métodos diferentes de GET e POST recebem 405 Method Not Allowed, acompanhado do header Allow, que informa quais métodos são aceitos naquele recurso.

2.8.8 Métodos e REST

Em APIs HTTP, os métodos possuem um papel fundamental, pois representam as operações realizadas sobre os recursos da aplicação.

Por exemplo, considere o recurso users.

Método	Recurso	Operação
GET	/users	Listar usuários
GET	/users/{id}	Consultar um usuário
POST	/users	Criar um usuário
PUT	/users/{id}	Atualizar um usuário
PATCH	/users/{id}	Atualizar parcialmente um usuário

Método	Recurso	Operação
DELETE	/users/{id}	Remover um usuário

Observe que a URL representa o recurso, enquanto o método define a operação a ser executada sobre ele. Essa combinação produz APIs mais intuitivas, consistentes e alinhadas ao protocolo HTTP.

2.8.9 Considerações

Os métodos HTTP são um dos pilares da comunicação entre clientes e servidores. Eles permitem expressar a intenção de cada requisição de forma padronizada, tornando as APIs mais previsíveis e interoperáveis.

Nas próximas seções veremos como associar diferentes métodos a diferentes rotas usando a biblioteca `net/http`, construindo uma API capaz de responder corretamente às operações realizadas pelos clientes.

Capítulo 3

Requisições e Respostas

No capítulo anterior, estudamos os elementos fundamentais de uma aplicação HTTP em Go. Vimos como iniciar um servidor com o pacote `net/http`, implementar handlers, receber requisições e construir respostas para os clientes.

Neste capítulo, vamos aprofundar a comunicação entre clientes e servidores. O foco será entender as informações transportadas por uma requisição HTTP e a forma como uma API pode estruturar suas respostas.

Uma requisição HTTP é mais do que uma URL. Ela pode conter parâmetros no caminho, parâmetros de consulta, cabeçalhos, cookies e um corpo com dados enviados pelo cliente. Cada elemento tem uma finalidade específica e deve ser usado de maneira adequada para que a API seja clara, consistente e alinhada aos padrões da Web.

Da mesma forma, uma resposta HTTP não se resume ao código de status retornado pelo servidor. A definição correta dos cabeçalhos, do tipo de conteúdo e da estrutura dos dados enviados tem papel importante na interoperabilidade entre sistemas e na facilidade de consumo da API.

Ao longo deste capítulo, veremos como acessar e utilizar esses componentes com a biblioteca padrão da linguagem Go. Também discutiremos boas práticas para construir respostas padronizadas, estabelecendo uma base que será usada durante o desenvolvimento da **GoList**.

Os conceitos apresentados aqui aparecem em praticamente todas as APIs HTTP, independentemente da linguagem ou do framework utilizado. Compreendê-los desde o início permitirá que os próximos capítulos avancem para funcionalidades mais complexas sem perder a clareza da comunicação entre cliente e servidor.

3.1 Path Parameters

Em uma API HTTP, é comum que uma mesma estrutura de rota seja usada para acessar recursos diferentes. Por exemplo, uma aplicação pode possuir milhares de usuários, mas não faz sentido criar manualmente uma URL diferente para cada um deles.

Para resolver esse problema, usamos **path parameters**, ou parâmetros de caminho, que permitem incorporar valores diretamente na URL.

Considere o seguinte endereço:

```
GET /users/42
```

Nesse exemplo, `/users` identifica o conjunto de usuários, enquanto `42` representa o identificador do usuário que se deseja consultar.

Da mesma forma:

```
GET /lists/10
```

indica que o cliente deseja acessar a lista de identificador `10`.

Os parâmetros de caminho fazem parte da própria URL e ajudam a identificar qual recurso será manipulado pela requisição.

3.1.1 Path Parameters e Recursos

Em APIs HTTP, path parameters são amplamente usados para representar recursos específicos.

Alguns exemplos comuns são:

```
GET    /users/15
PUT    /users/15
DELETE /users/15

GET    /lists/8
POST   /lists/8/items

PATCH /items/32
```

Observe que o identificador faz parte do caminho da URL. Isso torna as rotas mais intuitivas e facilita sua compreensão tanto por pessoas quanto por ferramentas.

3.1.2 Obtendo o Caminho da Requisição

Na estrutura `http.Request`, o caminho solicitado pelo cliente está disponível no campo `URL.Path`.

```
1 func handler(w http.ResponseWriter, r *http.Request) {
2     fmt.Println(r.URL.Path)
3 }
```

Se o cliente fizer a requisição:

```
GET /users/42
```

a saída será:

```
/users/42
```

Nesse ponto do código, `URL.Path` contém o caminho completo. A extração de valores como `42` depende da estratégia de roteamento usada pela aplicação.

3.1.3 Extrair Parâmetros Manualmente

Uma forma simples de entender a ideia consiste em dividir o caminho usando o pacote `strings`. No exemplo abaixo, o handler será registrado no padrão `/users/`, de modo que requisições cujo caminho comece com `/users/` serão encaminhadas para essa função.

```
1 package main
2
3 import (
4     "fmt"
5     "net/http"
6     "strconv"
7     "strings"
8 )
9
10 func users(w http.ResponseWriter, r *http.Request) {
11     parts := strings.Split(r.URL.Path, "/")
12
13     if len(parts) != 3 || parts[1] != "users" {
14         http.NotFound(w, r)
15         return
16     }
17
18     fmt.Fprintf(w, "Usuário: %s\n", id)
19 }
20
21 func main() {
22     http.HandleFunc("/users/", users)
23
24     if err := http.ListenAndServe(":8080", nil); err != nil {
25         panic(err)
26     }
27 }
```

Observe que o padrão registrado em `http.HandleFunc` é `/users/`. Por isso, neste exemplo, o caminho da requisição deve começar com `/users/`, como em `/users/42`.

Para a requisição:

```
GET /users/42
```

a resposta será:

```
Usuário: 42
```

Embora essa abordagem funcione em exemplos pequenos, ela rapidamente se torna difícil de manter à medida que o número de rotas aumenta.

3.1.4 Extraíndo Valores Numéricos

Em muitas APIs, identificadores são valores inteiros. Após extrair o parâmetro da URL, normalmente é necessário convertê-lo para um tipo numérico.

```
id, err := strconv.Atoi(parts[2])
if err != nil {
    http.Error(w, "Identificador inválido.", http.StatusBadRequest)
    return
}

fmt.Fprintf(w, "Usuário: %d\n", id)
```

Caso o cliente envie:

```
GET /users/abc
```

a conversão falhará, permitindo que a aplicação retorne uma resposta adequada ao cliente.

3.1.5 Boas Práticas

Path parameters devem ser usados para identificar recursos específicos da aplicação.

Por exemplo:

```
/users/15
/lists/8
/items/42
```

Em contrapartida, informações usadas para filtrar, ordenar ou paginar resultados normalmente pertencem à **query string**, que será estudada na próxima seção.

Por exemplo:

```
GET /users?page=2
GET /users?name=Maria
GET /lists?completed=true
```

Essa separação torna a API mais consistente e facilita seu uso.

3.1.6 Considerações

Path parameters permitem representar recursos específicos de forma natural, tornando as URLs mais expressivas e previsíveis.

Nesta seção, usamos a extração manual apenas para compreender como os valores fazem parte do caminho da requisição. Quando estudarmos roteamento, veremos formas mais adequadas de associar padrões de rota a handlers e obter esses valores sem espalhar manipulação manual de strings pela aplicação.

3.2 Query Parameters

Enquanto os **path parameters** identificam um recurso específico, os **query parameters**, ou parâmetros de consulta, são usados para modificar ou refinar uma requisição. Eles permitem informar filtros, critérios de ordenação, paginação e outras opções sem alterar o recurso solicitado.

Os query parameters aparecem após o caractere ? na URL.

Por exemplo:

```
GET /users?page=2
```

Nesse caso, o recurso continua sendo /users, mas o parâmetro page indica que o cliente deseja consultar a segunda página de resultados.

É possível enviar vários parâmetros na mesma requisição, separados pelo caractere &.

```
GET /pagination?page=2&limit=20&active=true
```

Essa requisição informa:

- página 2;
- limite de 20 registros por página;
- filtro active com valor true.

3.2.1 Quando Utilizar Query Parameters

Query parameters são apropriados para informações que modificam a forma como um recurso é consultado, mas não alteram sua identidade.

Alguns exemplos comuns são:

```
GET /users?name=Maria
GET /users?city=Vitória
GET /users?sort=name
GET /users?page=3
GET /users?limit=50
```

Em APIs HTTP, é recomendável usar:

- **path parameters** para identificar recursos;
- **query parameters** para filtrar ou configurar a consulta.

Essa distinção torna as URLs mais consistentes e intuitivas.

3.2.2 Obtendo os Query Parameters

Na estrutura `http.Request`, os parâmetros de consulta são acessados por meio do método `URL.Query()`. No exemplo abaixo, o handler pode ser registrado no path `/query`.

```
func queryHandler(w http.ResponseWriter, r *http.Request) {
    parameters := r.URL.Query()

    for k, p := range parameters {
        fmt.Fprintf(w, "%s: %s\n", k, p[0])
    }
}
```

Se a requisição for:

```
GET /query?page=2&limit=20&active=true
```

a resposta será semelhante a:

```
page: 2
limit: 20
active: true
```

O método retorna um valor do tipo `url.Values`, que representa um mapa de chaves para listas de valores. Por isso, no exemplo acima, `p` é uma lista de valores (`[]string`) e `p[0]` acessa o primeiro valor recebido para cada parâmetro.

Para obter um parâmetro específico, usamos o método `Get`. O exemplo abaixo aplica esse método em um handler de paginação, registrado no path `/pagination`.

Como os valores dos query parameters são sempre recebidos como texto, o handler converte `page` e `limit` para números inteiros, e `active` para um valor booleano. A biblioteca `strconv` possui funções para essas conversões.

```
func paginationHandler(w http.ResponseWriter, r *http.Request) {
    query := r.URL.Query()

    page, err := strconv.Atoi(query.Get("page"))
    if err != nil {
        page = 1
    }

    limit, err := strconv.Atoi(query.Get("limit"))
    if err != nil {
        limit = 10
    }

    active, err := strconv.ParseBool(query.Get("active"))
    if err != nil {
        active = false
    }

    fmt.Fprintf(w, "Página: %d\nLimite: %d\nAtivo: %v", page, limit, active)
}
```


Se a requisição for:

```
GET /pagination?page=2&limit=20&active=true
```

a resposta será:

```
Página: 2  
Limite: 20  
Ativo: true
```

Quando algum parâmetro estiver ausente ou não puder ser convertido, o exemplo usa valores padrão: página 1, limite 10 e active como false.

3.2.3 Múltiplos Valores

Um mesmo parâmetro pode aparecer mais de uma vez na URL.

```
GET /users?role=admin&role=manager
```

Nessa situação, o método `Get` retorna apenas o primeiro valor.

Para recuperar todos eles, acessamos diretamente o mapa retornado por `URL.Query()`.

```
roles := r.URL.Query()["role"]  
  
fmt.Println(roles)
```

A saída será semelhante a:

```
[admin manager]
```

Esse recurso é útil para filtros que aceitam múltiplas opções.

3.2.4 Considerações

Query parameters oferecem uma maneira simples e padronizada de personalizar consultas sem alterar a identificação do recurso solicitado. Eles são amplamente usados para implementar filtros, paginação, ordenação e critérios de pesquisa em APIs HTTP.

Ao usar a estrutura `http.Request`, a biblioteca `net/http` disponibiliza mecanismos simples para acessar esses parâmetros, permitindo que a aplicação trate seus valores de forma segura e consistente. Nas próximas seções, eles serão empregados com frequência na implementação das operações de consulta da **GoList**.

3.3 Body

Em muitas requisições HTTP, o caminho da URL e os parâmetros de consulta não são suficientes para transportar todas as informações necessárias. Quando o cliente precisa enviar dados mais estruturados ao servidor, usamos o **corpo da requisição**, ou *request body*.

O corpo da requisição pode conter documentos JSON, dados de formulários, arquivos ou qualquer outro formato aceito pela aplicação.

Por exemplo, considere uma requisição para criar um novo usuário.

```
POST /users HTTP/1.1
Content-Type: application/json

{
  "name": "Maria",
  "email": "maria@example.com"
}
```

Nesse caso, o corpo da requisição contém as informações necessárias para criar o usuário.

Embora o protocolo permita corpo em diferentes métodos, na prática ele aparece principalmente em requisições POST, PUT e PATCH, que normalmente enviam dados ao servidor.

3.3.1 Acessando o Corpo da Requisição

Na estrutura `http.Request`, o corpo da requisição está disponível no campo `Body`.

```
type Request struct {
    ...
    Body io.ReadCloser
    ...
}
```

Como o corpo é representado por um fluxo de dados, ou *stream*, ele normalmente é lido uma única vez durante o processamento do handler. Em handlers de servidor, o próprio `net/http` gerencia o fechamento desse corpo após o processamento da requisição.

O exemplo a seguir lê todo o conteúdo enviado pelo cliente. Essa abordagem é útil para exemplos pequenos.

```
1 package main
2
3 import (
4     "fmt"
5     "io"
6     "net/http"
7 )
8
9 func bodyHandler(w http.ResponseWriter, r *http.Request) {
10     body, err := io.ReadAll(r.Body)
11     if err != nil {
12         http.Error(w, "Erro ao ler a requisição.", http.StatusInternalServerError)
13         return
14     }
15
16     fmt.Fprintf(w, "Body: %s\n", string(body))
17 }
```

Para testar esse handler, podemos enviar uma requisição POST com curl. Como o handler foi registrado no path /body/, a URL usada no teste será `http://localhost:8080/body/`.

```
curl -X POST http://localhost:8080/body/ \
-d "Olá, Go!"
```

A resposta será:

```
Body: Olá, Go!
```

3.3.2 Convertendo o Corpo para uma Estrutura

Em APIs modernas, o formato mais comum para troca de dados é JSON.

Suponha a seguinte estrutura:

```
type User struct {
    Name string `json:"name"`
    Email string `json:"email"`
}

func (u User) String() string {
    return fmt.Sprintf("%s (%s)", u.Name, u.Email)
}
```

O corpo pode ser convertido diretamente para essa estrutura usando o pacote `encoding/json`.

```
func userHandler(w http.ResponseWriter, r *http.Request) {  
    var user User  
  
    if err := json.NewDecoder(r.Body).Decode(&user); err != nil {  
        http.Error(w, "JSON inválido.", http.StatusBadRequest)  
        return  
    }  
  
    fmt.Fprintf(w, "User: %s\n", user)  
}
```

Nesse exemplo, `json.NewDecoder(r.Body).Decode(&user)` lê o corpo da requisição e preenche os campos da variável `user`. O método `String` define como esse valor será formatado quando usado com `%s`.

Para testar esse handler, podemos enviar um documento JSON com `curl`:

```
curl -X POST http://localhost:8080/user \  
-H "Content-Type: application/json" \  
-d '{"name":"Maria","email":"maria@example.com"}'
```

A resposta será:

```
User: Maria (maria@example.com)
```

Essa abordagem será utilizada com frequência ao longo deste livro.

3.3.3 Lendo o Corpo Apenas Uma Vez

Como `Body` representa um fluxo de dados, seu conteúdo é consumido durante a leitura.

Considere o seguinte código:

```
body1, _ := io.ReadAll(r.Body)  
body2, _ := io.ReadAll(r.Body)
```

A primeira leitura obterá todo o conteúdo enviado pelo cliente.

Na segunda leitura, o corpo já terá sido consumido e `body2` estará vazio.

Por esse motivo, normalmente o corpo é lido apenas uma vez, sendo armazenado em memória quando necessário ou convertido diretamente para uma estrutura.

3.3.4 Tratando Erros

Sempre que dados externos são recebidos, existe a possibilidade de erro durante a leitura ou interpretação do conteúdo.

Por isso, é importante validar o resultado da operação.

```
body, err := io.ReadAll(r.Body)
if err != nil {
    http.Error(w, "Erro ao processar a requisição.", http.StatusInternalServerError)
    return
}
```

Da mesma forma, ao converter JSON para estruturas Go, falhas de sintaxe ou dados incompatíveis devem ser tratadas adequadamente.

3.3.5 Quando Utilizar o Body

O corpo da requisição deve ser usado quando o cliente precisa enviar dados para o servidor.

Alguns exemplos são:

Operação	Corpo da requisição
Criar um usuário	Dados do usuário
Atualizar uma lista	Novos dados da lista
Alterar um item	Campos modificados
Enviar um arquivo	Conteúdo do arquivo

Em contrapartida, informações usadas apenas para identificar recursos ou filtrar consultas devem permanecer na URL, usando **path parameters** ou **query parameters**.

3.3.6 Considerações

O corpo da requisição é o principal mecanismo usado para transmitir dados estruturados do cliente para o servidor. Em aplicações modernas, ele normalmente contém documentos JSON que representam os recursos manipulados pela API.

Embora a leitura do corpo possa ser feita manualmente, na maioria das APIs o conteúdo é convertido diretamente para estruturas da linguagem usando `encoding/json`. Nas próximas seções exploraremos esse processo em detalhes, aprendendo a serializar e desserializar objetos Go de forma simples e segura.

3.4 Headers

Os **headers**, ou cabeçalhos HTTP, permitem transmitir metadados entre cliente e servidor sem colocá-los na URL nem no corpo da mensagem. Eles descrevem características da requisição ou da resposta, informam preferências do cliente, transportam credenciais de autenticação e controlam diversos aspectos da comunicação HTTP.

Uma requisição HTTP pode conter vários headers.

```
POST /users HTTP/1.1
Host: localhost:8080
Content-Type: application/json
```

```
Accept: application/json
Authorization: Bearer <token>

{
  "name": "Maria",
  "email": "maria@example.com"
}
```

Nesse exemplo:

- Host identifica o servidor de destino;
- Content-Type informa o formato do corpo da requisição;
- Accept indica os formatos de resposta aceitos pelo cliente;
- Authorization transporta as credenciais de autenticação.

Headers não substituem o corpo da mensagem. Eles fornecem informações sobre como a requisição ou a resposta deve ser interpretada.

3.4.1 Lendo Headers da Requisição

Na estrutura `http.Request`, os headers enviados pelo cliente estão disponíveis no campo `Header`. Na prática, normalmente queremos acessar apenas alguns headers específicos. Para isso, usamos o método `Get`.

```
func requestHeadersHandler(w http.ResponseWriter, r *http.Request) {
    authorization := r.Header.Get("Authorization")
    accept := r.Header.Get("Accept")
    userAgent := r.Header.Get("User-Agent")

    fmt.Fprintf(w, "Authorization: %s\n", authorization)
    fmt.Fprintf(w, "Accept: %s\n", accept)
    fmt.Fprintf(w, "User-Agent: %s\n", userAgent)
}
```

Esse handler pode ser registrado no path `/headers/request`. Para testá-lo, podemos enviar alguns headers com `curl`:

```
curl http://localhost:8080/headers/request \
-H "Authorization: Bearer abc123" \
-H "Accept: application/json"
```

A resposta será semelhante a:

```
Authorization: Bearer abc123
Accept: application/json
User-Agent: curl/8.0.0
```

Caso um header não exista, o método retorna uma string vazia. Em geral, prefira `Get` em vez de acessar o mapa diretamente, pois ele lida melhor com a forma canônica dos nomes de headers.

3.4.2 Definindo Headers da Resposta

Os headers enviados pelo servidor são configurados usando o método `Header()` do `ResponseWriter`.

```
func responseHeadersHandler(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "text/plain")
    w.Header().Set("X-App-Version", "1.0")
    w.Header().Set("X-App-Id", "abc123")

    fmt.Fprintln(w, "Resposta com headers definidos pelo servidor.")
}
```

Esse handler pode ser registrado no path `/headers/response`. Para visualizar os headers da resposta, use `curl` com a opção `-i`:

```
curl -i http://localhost:8080/headers/response
```

A resposta incluirá headers semelhantes a estes:

```
Content-Type: text/plain
X-App-Id: abc123
X-App-Version: 1.0
```

O método `Set` define o valor do header, substituindo qualquer valor anterior associado ao mesmo nome. Quando for necessário adicionar mais de um valor para o mesmo header, use `Add`.

É importante definir os headers antes de escrever o corpo da resposta. Após a primeira chamada a `WriteHeader` ou após a primeira escrita no corpo, os headers são enviados ao cliente e não podem mais ser alterados para aquela resposta.

3.4.3 Headers Mais Utilizados

Durante o desenvolvimento de APIs, alguns headers aparecem com bastante frequência.

Header	Finalidade
Content-Type	Informa o formato do corpo da mensagem.
Accept	Indica os formatos aceitos pelo cliente.
Authorization	Transporta credenciais de autenticação.
User-Agent	Identifica a aplicação cliente.
Host	Identifica o servidor de destino.
Content-Length	Informa o tamanho do corpo da mensagem.
Cache-Control	Define políticas de cache.

Ao longo deste livro, outros headers serão apresentados conforme novos recursos forem incorporados à **GoList**.

3.4.4 Múltiplos Valores

Alguns headers podem possuir mais de um valor.

Por exemplo:

```
Accept: application/json  
Accept: text/plain
```

Como o tipo `http.Header` é definido como um mapa de listas de valores, é possível recuperar todos eles.

```
values := r.Header.Values("Accept")  
  
fmt.Println(values)
```

A saída será semelhante a:

```
[application/json text/plain]
```

Na maioria das situações, entretanto, `Get` é suficiente, pois retorna o primeiro valor de forma simples.

3.4.5 Considerações

Headers desempenham um papel fundamental na comunicação HTTP. Eles permitem transportar metadados sobre a requisição e a resposta sem interferir no conteúdo principal da mensagem.

Nas próximas seções utilizaremos diversos headers para implementar autenticação, negociação de conteúdo, CORS, controle de cache e outras funcionalidades comuns em APIs modernas. Mesmo quando seu uso não é explícito no código, eles fazem parte de praticamente toda comunicação entre cliente e servidor.

3.5 Cookies

Embora o protocolo HTTP seja, por natureza, **sem estado** (*stateless*), muitas aplicações precisam manter algum vínculo entre diferentes requisições. Um exemplo comum é reconhecer um usuário autenticado durante sua navegação pela aplicação.

Os **cookies** são um dos mecanismos usados para esse propósito. Eles permitem que o servidor envie pequenas informações ao navegador do cliente. Depois de armazenadas, essas informações podem ser enviadas automaticamente em requisições seguintes para o mesmo servidor.

O funcionamento básico é simples: na resposta, o servidor envia o header `Set-Cookie`. O navegador armazena esse valor e passa a incluí-lo automaticamente nas próximas requisições por meio do header `Cookie`.

3.5.1 Criando um Cookie

Na biblioteca `net/http`, um cookie é representado pela estrutura `http.Cookie`.

O exemplo a seguir cria um cookie chamado username.

```
func createCookieHandler(w http.ResponseWriter, r *http.Request) {  
    cookie := &http.Cookie{  
        Name:     "username",  
        Value:    "Maria",  
    }  
  
    http.SetCookie(w, cookie)  
  
    fmt.Fprintln(w, "Cookie criado.")  
}
```

Para testar com curl e salvar o cookie recebido, use a opção -c:

```
curl -i -c cookies.txt http://localhost:8080/
```

Após essa resposta, o cliente poderá armazenar o cookie e enviá-lo nas próximas requisições ao servidor.

3.5.2 Lendo um Cookie

Os cookies enviados pelo cliente podem ser recuperados usando o método Cookie da estrutura http.Request.

```
func readCookieHandler(w http.ResponseWriter, r *http.Request) {  
    cookie, err := r.Cookie("username")  
    if err != nil {  
        http.Error(w, "Cookie não encontrado.", http.StatusNotFound)  
        return  
    }  
  
    fmt.Fprintf(w, "Value: %s\n", cookie.Value)  
}
```

Se este handler for registrado no path /me, o navegador enviará automaticamente o cookie nas próximas requisições. Com curl, esse envio precisa ser indicado manualmente. Para isso, usamos a opção -b, lendo o cookie salvo no teste anterior:

```
curl -b cookies.txt http://localhost:8080/me
```

Se o cookie for enviado na requisição, a resposta será:

```
Value: Maria
```

Quando o cliente envia o cookie de volta ao servidor, o header Cookie carrega o nome e o valor do cookie.

Caso o cookie não esteja presente na requisição, o método retornará um erro. Neste exemplo, qualquer erro ao obter o cookie resulta em uma resposta 404 Not Found.

3.5.3 Configurando um Cookie

Além do nome e do valor, um cookie pode possuir configurações que controlam quando ele será enviado novamente pelo cliente e por quanto tempo será armazenado.

```
cookie := &http.Cookie{
    Name:     "username",
    Value:    "Maria",
    Path:     "/",
    MaxAge:   3600,
    HttpOnly: true,
    Secure:   false,
    SameSite: http.SameSiteLaxMode,
}
```

Os campos mais utilizados são:

Campo	Finalidade
Name	Nome do cookie.
Value	Valor armazenado.
Path	Caminhos em que o cookie será enviado.
MaxAge	Tempo de vida em segundos.
Expires	Data de expiração.
HttpOnly	Impede acesso via JavaScript.
Secure	Envia o cookie apenas em conexões HTTPS.
SameSite	Controla o envio em navegações entre sites.

Essas configurações permitem controlar o comportamento do cookie e aumentar sua segurança. Em aplicações reais, HttpOnly, Secure e SameSite costumam ser especialmente importantes para cookies ligados a autenticação.

3.5.4 Cookies e Autenticação

Uma das aplicações mais comuns dos cookies é o gerenciamento de sessões.

Após autenticar um usuário, o servidor pode gerar um identificador de sessão e enviá-lo em um cookie. O ideal é que o cookie contenha um identificador opaco, e não dados sensíveis do usuário.

```
Set-Cookie: session=abc123xyz
```

Nas requisições seguintes, o navegador enviará automaticamente:

```
Cookie: session=abc123xyz
```

Com esse identificador, o servidor pode localizar a sessão correspondente e reconhecer o usuário sem que ele precise informar suas credenciais novamente a cada requisição.

Embora muitas APIs usem tokens enviados no header `Authorization`, cookies continuam sendo

amplamente empregados em aplicações web e em diversos mecanismos de autenticação baseados em sessão.

3.5.5 Removendo um Cookie

Para remover um cookie, o servidor envia outro cookie com o mesmo nome e tempo de vida expirado. Quando o cookie tiver sido criado com um Path específico, a remoção deve usar um caminho compatível.

```
func removeCookieHandler(w http.ResponseWriter, r *http.Request) {  
    cookie := &http.Cookie{  
        Name:  "username",  
        Value: "",  
        Path:  "/",  
        MaxAge: -1,  
    }  
  
    http.SetCookie(w, cookie)  
  
    fmt.Fprintln(w, "Cookie removido.")  
}
```

Se este handler for registrado no path / remove, podemos testá-lo com o mesmo arquivo de cookies usado anteriormente:

```
curl -i -b cookies.txt -c cookies.txt http://localhost:8080/remove
```

Ao receber essa resposta, o cliente removerá o cookie armazenado com aquele nome, desde que as regras usadas na remoção sejam compatíveis com as regras usadas na criação.

3.5.6 Considerações

Cookies constituem um mecanismo simples e eficiente para manter informações entre diferentes requisições HTTP. Apesar de serem frequentemente associados ao gerenciamento de sessões, eles também podem ser usados para armazenar preferências do usuário, configurações da aplicação e outros dados de pequeno porte.

Ao longo deste livro, os cookies serão revisitados no capítulo de autenticação, onde veremos como usá-los para implementar sessões de usuário e compararemos essa abordagem com o uso de **JSON Web Tokens (JWT)**, bastante comum no desenvolvimento de APIs modernas.

3.6 Content-Type

Uma mensagem HTTP pode transportar diferentes tipos de conteúdo. O corpo pode conter um documento JSON, um formulário HTML, uma imagem, um arquivo PDF ou dados binários. Para que cliente e servidor interpretem corretamente esse corpo, o protocolo HTTP utiliza o header **Content-Type**.

O Content-Type informa o formato dos dados presentes no corpo da mensagem.

Por exemplo, uma requisição que envia um documento JSON normalmente possui o seguinte header:

```
POST /users HTTP/1.1
Content-Type: application/json

{
  "name": "Maria",
  "email": "maria@example.com"
}
```

Ao receber essa requisição, o servidor sabe que o corpo contém um documento JSON e pode processá-lo adequadamente.

Da mesma forma, respostas com corpo devem informar o formato do conteúdo retornado.

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 1,
  "name": "Maria"
}
```

Nesse caso, o cliente sabe que a resposta deve ser interpretada como JSON.

3.6.1 Definindo o Content-Type

Na biblioteca `net/http`, o header é definido por meio do método `Header()` do `ResponseWriter`.

```
func textHandler(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "text/plain; charset=utf-8")

    fmt.Fprintln(w, "Olá, mundo!")
}
```

Nesse exemplo, a resposta será enviada como texto simples em UTF-8, preservando caracteres acentuados.

Para retornar um documento JSON, basta alterar o valor do header.

```
func jsonHandler(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")

    fmt.Fprintln(w, `{"status":"ok"}`)
}
```

É importante definir o `Content-Type` antes de chamar `WriteHeader` ou antes da primeira escrita no corpo da resposta.

3.6.2 Obtendo o Content-Type da Requisição

O servidor também pode consultar o formato dos dados enviados pelo cliente.

```
func contentTypeHandler(w http.ResponseWriter, r *http.Request) {  
    contentType := r.Header.Get("Content-Type")  
  
    fmt.Println(contentType)  
}
```

Se o cliente enviar:

Content-Type: application/json

a variável contentType conterá:

application/json

Essa informação pode ser usada para validar se o cliente enviou os dados no formato esperado. Como o header pode conter parâmetros, como charset, é melhor separar o tipo principal antes de comparar.

```
mediaType, _, err := mime.ParseMediaType(r.Header.Get("Content-Type"))  
if err != nil || mediaType != "application/json" {  
    http.Error(w, "Content-Type inválido.", http.StatusUnsupportedMediaType)  
    return  
}
```

Assim, valores como application/json; charset=utf-8 continuam sendo aceitos corretamente.

3.6.3 Tipos Mais Utilizados

O protocolo HTTP usa tipos de mídia, também conhecidos como *MIME types*, para identificar formatos de conteúdo. No desenvolvimento de APIs, alguns aparecem com bastante frequência.

Content-Type	Descrição
application/json	Documento JSON.
text/plain	Texto simples.
text/html	Documento HTML.
application/xml	Documento XML.
application/octet-stream	Dados binários.
multipart/form-data	Envio de formulários com arquivos.
application/x-www-form-urlencoded	Formulários HTML tradicionais.

Durante este livro, usaremos principalmente application/json, reservando multipart/form-data para o capítulo dedicado ao upload de arquivos.

3.6.4 Charset

Alguns formatos permitem informar também parâmetros adicionais, como a codificação de caracteres utilizada.

Por exemplo:

```
Content-Type: text/plain; charset=utf-8
```

O parâmetro `charset` indica que o texto está codificado em UTF-8, permitindo que caracteres acentuados e símbolos especiais sejam interpretados corretamente.

Embora esse parâmetro seja comum em conteúdos textuais, normalmente ele não é necessário para documentos JSON, pois JSON usa UTF-8 por padrão em APIs modernas.

3.6.5 Content-Type e APIs HTTP

Em APIs HTTP modernas, o formato mais utilizado para troca de dados é JSON.

Por esse motivo, é uma boa prática que:

- clientes enviem requisições com `Content-Type: application/json` quando o corpo for JSON;
- servidores respondam com `Content-Type: application/json` quando retornarem JSON.

Essa padronização simplifica a integração entre sistemas e reduz ambiguidades na interpretação das mensagens.

3.6.6 Considerações

O header `Content-Type` informa como o corpo de uma mensagem HTTP deve ser interpretado. Seu uso correto permite que cliente e servidor processem os dados de forma consistente, independentemente do formato utilizado.

Nas próximas seções, quando estudarmos o pacote `encoding/json`, usaremos esse header em praticamente todas as requisições e respostas da **GoList**, tornando JSON o principal formato de comunicação da aplicação.

3.7 Respostas Padronizadas

Uma API não deve apenas executar operações corretamente; ela também precisa responder de maneira consistente. Quando respostas semelhantes seguem um padrão comum, torna-se mais simples desenvolver clientes, tratar erros e compreender o comportamento da aplicação.

Considere, por exemplo, uma API em que cada endpoint retorna um formato diferente.

```
{  
  "name": "Maria"  
}
```

Outro endpoint responde:

```
{
  "success": true,
  "result": {
    "name": "Maria"
  }
}
```

E um terceiro retorna:

```
{
  "data": {
    "name": "Maria"
  }
}
```

Embora todas essas respostas sejam válidas, essa falta de padronização dificulta o consumo da API, pois o cliente precisa conhecer um formato diferente para cada operação.

Uma alternativa é adotar uma estrutura consistente e flexível para as respostas da aplicação. Neste livro, usaremos a ideia de três campos principais: data, meta e error.

3.7.1 Estrutura Base

A estrutura base pode ser entendida da seguinte forma:

```
{
  "data": ...,
  "meta": ...,
  "error": ...
}
```

O campo data contém o resultado principal da operação. O campo meta contém informações adicionais, como paginação. O campo error contém detalhes sobre falhas.

Na resposta final, campos sem valor podem ser omitidos. Assim, uma resposta de sucesso não precisa retornar error: null, e uma resposta de erro não precisa retornar data: null.

3.7.2 Respostas de Sucesso

Em uma resposta de sucesso, os dados principais ficam no campo data.

```
{
  "data": {
    "id": 1,
    "name": "Maria"
  }
}
```

Quando a operação retorna uma coleção, o mesmo campo pode ser usado.

```
{
  "data": [
    {
      "id": 1,
      "name": "Maria"
    },
    {
      "id": 2,
      "name": "João"
    }
  ]
}
```

Quando houver informações adicionais, como paginação, elas podem ser incluídas em meta.

```
{
  "data": [
    {
      "id": 1,
      "name": "Maria"
    },
    {
      "id": 2,
      "name": "João"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10
  }
}
```

Essa padronização facilita o desenvolvimento de clientes, pois a estrutura da resposta permanece previsível sem obrigar a presença de campos vazios.

3.7.3 Respostas de Erro

Da mesma forma, respostas de erro devem usar o campo error.

```
{
  "error": {
    "code": "USER_NOT_FOUND",
    "message": "Usuário não encontrado."
  }
}
```

Além do código de status HTTP, esse formato fornece informações para que aplicações clientes identifiquem e tratem cada situação de forma adequada. Como data e meta não possuem valor nesse caso, eles podem ser omitidos da resposta.

3.7.4 Criando Estruturas em Go

Uma forma simples de representar esse padrão consiste em definir uma estrutura única de resposta.

```
type Response struct {  
    Data    any          `json:"data,omitempty"`  
    Meta    any          `json:"meta,omitempty"`  
    Error *ErrorDetail `json:"error,omitempty"`  
}  
  
type ErrorDetail struct {  
    Code    string `json:"code"`  
    Message string `json:"message"`  
}
```

A tag `omitempty` faz com que campos sem valor sejam omitidos na serialização JSON. Nesse modelo, quando `Data`, `Meta` ou `Error` forem `nil`, eles não aparecerão na resposta final. Com isso, a aplicação mantém uma estrutura comum sem precisar enviar campos nulos em todas as respostas.

3.7.5 Enviando uma Resposta JSON

Após definir a estrutura, o handler pode convertê-la para JSON na resposta HTTP.

```
func handler(w http.ResponseWriter, r *http.Request) {  
    w.Header().Set("Content-Type", "application/json")  
  
    response := Response{  
        Data: map[string]any{  
            "id": 1,  
            "name": "Maria",  
        },  
    }  
  
    if err := json.NewEncoder(w).Encode(response); err != nil {  
        http.Error(w, "Erro ao gerar resposta.", http.StatusInternalServerError)  
        return  
    }  
}
```

A resposta produzida será:

```
{  
  "data": {  
    "id": 1,  
    "name": "Maria"  
  }  
}
```

}

3.7.6 Criando Funções Auxiliares

À medida que a aplicação cresce, torna-se inconveniente repetir o mesmo código em todos os handlers.

Uma alternativa é criar funções auxiliares.

```
func WriteResponse(w http.ResponseWriter, status int, response Response) error {
    body, err := json.Marshal(response)
    if err != nil {
        http.Error(w, "Erro ao gerar resposta.", http.StatusInternalServerError)
        return err
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)

    _, err = w.Write(body)
    return err
}
```

Nesse helper, a resposta é convertida para JSON antes de WriteHeader. Assim, se a serialização falhar, ainda é possível retornar um erro 500 Internal Server Error.

Com esse helper central, podemos criar funções específicas para sucesso e erro.

```
func WriteJSON(w http.ResponseWriter, status int, data, meta any) error {
    return WriteResponse(w, status, Response{
        Data: data,
        Meta: meta,
    })
}

func WriteError(w http.ResponseWriter, status int, code, message string) error {
    return WriteResponse(w, status, Response{
        Error: &ErrorDetail{
            Code:    code,
            Message: message,
        },
    })
}
```

O helper WriteResponse serializa a estrutura, define o Content-Type, escreve o status e envia o corpo da resposta. O erro retornado por essas funções representa uma falha técnica ao gerar ou escrever a resposta, não um erro de negócio da API. Para respostas sem corpo, como 204 No

Content, o handler deve escrever apenas o status, sem encapsular dados em JSON.

O uso dessas funções torna os handlers mais simples.

```
func handler(w http.ResponseWriter, r *http.Request) {
    user := User{
        ID: 1,
        Name: "Maria",
    }

    if err := WriteJSON(w, http.StatusOK, user, nil); err != nil {
        return
    }
}
```

Em caso de erro:

```
WriteError(w, http.StatusNotFound, "USER_NOT_FOUND", "Usuário não encontrado.")
```

3.7.7 Considerações

A padronização das respostas melhora a experiência de quem utiliza uma API. Clientes passam a conhecer previamente o formato das mensagens, reduzindo código repetitivo e facilitando o tratamento de erros.

Ao longo deste livro, a **GoList** adotará um formato consistente para suas respostas HTTP. Inicialmente utilizaremos uma estrutura simples, suficiente para os exemplos apresentados. Nos capítulos posteriores, à medida que novos recursos forem incorporados à aplicação, esse modelo poderá ser refinado para atender necessidades como validação, autenticação, observabilidade e metadados mais completos.

3.8 Desafios Preparatórios

Antes de iniciar o projeto da **GoList**, vale consolidar os conceitos estudados neste capítulo em pequenos exercícios. A ideia aqui não é construir a aplicação final, mas praticar os blocos que serão usados nela: query parameters, corpo da requisição, headers, validação de Content-Type, códigos de status e respostas padronizadas.

Esses desafios devem ser resolvidos usando apenas a biblioteca padrão de Go. O objetivo é preparar o terreno para o projeto, sem introduzir banco de dados, roteadores externos, camadas de arquitetura ou persistência.

3.8.1 Desafio 1: Consulta com Query Parameters

Crie um handler `searchHandler` registrado no path `/search`.

Esse handler deve ler os seguintes query parameters:

- q

- page
- limit

A requisição de teste pode ser:

```
GET /search?q=go&page=2&limit=10
```

O handler deve aplicar valores padrão quando page ou limit não forem informados ou não puderem ser convertidos:

- page: 1
- limit: 10

A resposta deve usar a estrutura padronizada com data e, se desejar, meta.

Exemplo de resposta:

```
{
  "data": {
    "q": "go"
  },
  "meta": {
    "page": 2,
    "limit": 10
  }
}
```

3.8.2 Desafio 2: Leitura de Body JSON

Crie um handler `createUserHandler` registrado no path `/users`.

Esse handler deve receber um corpo JSON com os dados de um usuário.

```
{
  "name": "Maria",
  "email": "maria@example.com"
}
```

O corpo deve ser convertido para uma estrutura Go.

```
type User struct {
    Name string `json:"name"`
    Email string `json:"email"`
}
```

A resposta deve retornar o usuário dentro do campo data.

```
{
  "data": {
    "name": "Maria",
    "email": "maria@example.com"
  }
}
```

```
}
}
```

3.8.3 Desafio 3: Validação de Content-Type

Antes de ler o corpo da requisição no handler `/users`, valide se o cliente enviou o header correto.

`Content-Type: application/json`

Use `mime.ParseMediaType` para aceitar também variações como:

`application/json; charset=utf-8`

Caso o `Content-Type` não seja JSON, retorne uma resposta padronizada de erro com status 415 `Unsupported Media Type`.

```
{
  "error": {
    "code": "UNSUPPORTED_MEDIA_TYPE",
    "message": "Content-Type inválido."
  }
}
```

3.8.4 Desafio 4: Resposta Padronizada

Implemente uma estrutura única para as respostas da aplicação.

```
type Response struct {
    Data    any          `json:"data,omitempty"`
    Meta    any          `json:"meta,omitempty"`
    Error *ErrorDetail `json:"error,omitempty"`
}

type ErrorDetail struct {
    Code    string `json:"code"`
    Message string `json:"message"`
}
```

Em seguida, crie funções auxiliares para escrever respostas de sucesso e erro.

```
func WriteJSON(w http.ResponseWriter, status int, data, meta any) error
func WriteError(w http.ResponseWriter, status int, code, message string) error
```

Essas funções devem definir `Content-Type: application/json`, escrever o status correto e retornar erros técnicos de serialização ou escrita quando eles ocorrerem.

3.8.5 Desafio 5: Erros de Validação

No handler `/users`, valide os campos recebidos.

Se `name` ou `email` estiverem vazios, retorne status `400 Bad Request` com uma resposta padronizada.

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Nome e email são obrigatórios."
  }
}
```

3.8.6 Testes Sugeridos

Teste a consulta com query parameters:

```
curl "http://localhost:8080/search?q=go&page=2&limit=10"
```

Teste a criação de usuário com JSON válido:

```
curl -i -X POST http://localhost:8080/users \
  -H "Content-Type: application/json" \
  -d '{"name":"Maria","email":"maria@example.com"}'
```

Teste a validação de `Content-Type`:

```
curl -i -X POST http://localhost:8080/users \
  -H "Content-Type: text/plain" \
  -d 'Maria'
```

Teste a validação dos campos obrigatórios:

```
curl -i -X POST http://localhost:8080/users \
  -H "Content-Type: application/json" \
  -d '{"name":"","email":""}'
```

3.8.7 Considerações

Esses desafios funcionam como um ensaio geral antes do início da **GoList**. Ao resolvê-los, você pratica a entrada e saída de dados em uma API HTTP sem ainda se preocupar com banco de dados, autenticação, organização em pacotes ou regras de negócio mais elaboradas.

No próximo capítulo, esses mesmos conceitos começarão a ser usados dentro de um projeto real.

Capítulo 4

Roteamento

Nos capítulos anteriores, desenvolvemos os conceitos fundamentais para a construção de aplicações HTTP em Go. Aprendemos como criar um servidor, implementar *handlers*, acessar informações da requisição e produzir respostas adequadas. Entretanto, ainda existe uma limitação importante: como determinar qual função deve processar cada recurso da aplicação?

É justamente essa responsabilidade que pertence ao **roteamento**.

O roteamento consiste no processo de associar uma requisição HTTP ao código responsável por tratá-la. Em outras palavras, quando o servidor recebe uma requisição para uma determinada URL, ele precisa identificar qual *handler* deverá ser executado. Essa decisão normalmente leva em consideração fatores como o caminho (*path*) solicitado e, em muitos casos, o método HTTP utilizado.

No contexto da **GoList**, apresentada no **Apêndice A**, a API precisa expor recursos como usuários, listas de compras e itens. Cada um desses recursos será acessado por meio de rotas específicas.

```
GET    /lists
POST   /lists
GET    /lists/15
PUT    /lists/15
DELETE /lists/15

POST   /users
GET    /users/8
```

Cada uma dessas requisições possui um comportamento diferente. Embora algumas compartilhem o mesmo caminho, como GET `/lists` e POST `/lists`, elas representam operações distintas e devem ser tratadas por funções diferentes. A rota, portanto, não identifica apenas uma URL: ela expressa a combinação entre um caminho, um método HTTP e a ação esperada dentro da aplicação.

Em aplicações muito pequenas, seria possível registrar apenas um único *handler* e utilizar diversas estruturas `if` ou `switch` para decidir qual lógica executar. Essa abordagem, porém, rapidamente se torna difícil de manter conforme novos recursos são adicionados.

O roteamento resolve esse problema permitindo que cada grupo de requisições seja direcionado

para o *handler* apropriado. Dessa forma, a responsabilidade de localizar o código correto deixa de estar espalhada pela aplicação e passa a ser centralizada em um componente específico.

A biblioteca padrão de Go fornece esse mecanismo por meio do **ServeMux**, responsável por registrar rotas e encaminhar automaticamente cada requisição para o *handler* correspondente. Desde o Go 1.22, esse mecanismo foi significativamente aprimorado, passando a oferecer suporte nativo a restrições por método HTTP e parâmetros de caminho, recursos que anteriormente exigiam implementações manuais ou o uso de bibliotecas externas.

Neste capítulo estudaremos como organizar as rotas de uma aplicação utilizando apenas a biblioteca padrão. Veremos como registrar recursos, restringir rotas a determinados métodos HTTP, agrupar funcionalidades relacionadas e estruturar o projeto de forma que a evolução da API permaneça simples e organizada. Todos esses conceitos serão aplicados gradualmente na **GoList**, usando a visão de projeto descrita no **Apêndice A** como referência.

4.1 ServeMux

O **ServeMux** (*Serve Multiplexer*) é o roteador fornecido pela biblioteca padrão de Go. Sua principal responsabilidade é associar padrões de rota a *handlers*, encaminhando cada requisição para a função responsável por processá-la.

Em outras palavras, o ServeMux atua como um ponto central de distribuição das requisições recebidas pelo servidor HTTP. Sempre que uma nova requisição chega, ele procura um padrão registrado compatível e determina qual *handler* deve ser executado.

O exemplo mais simples consiste em registrar uma rota utilizando a função `http.HandleFunc`, que utiliza o *ServeMux* padrão da aplicação.

```
1 package main
2
3 import (
4     "fmt"
5     "net/http"
6 )
7
8 func hello(w http.ResponseWriter, r *http.Request) {
9     fmt.Fprintln(w, "Olá, mundo!")
10 }
11
12 func main() {
13     http.HandleFunc("/", hello)
14
15     http.ListenAndServe(":8080", nil)
16 }
```

Nesse exemplo, a função `hello` é registrada para atender ao caminho `/`. O segundo argumento de `ListenAndServe` é `nil`, indicando que o servidor utilizará o `DefaultServeMux`, instância global criada automaticamente pelo pacote `net/http`.

Embora conveniente para exemplos pequenos, utilizar o roteador global não costuma ser a melhor opção em projetos que crescem com o tempo. Na GoList, por exemplo, as rotas serão organizadas de forma explícita. Para isso, é preferível criar uma instância própria de ServeMux.

```
1 package main
2
3 import (
4     "fmt"
5     "net/http"
6 )
7
8 func hello(w http.ResponseWriter, r *http.Request) {
9     fmt.Fprintln(w, "Olá, mundo!")
10 }
11
12 func main() {
13     mux := http.NewServeMux()
14
15     mux.HandleFunc("/", hello)
16
17     http.ListenAndServe(":8080", mux)
18 }
```

Essa abordagem torna mais explícita a estrutura da aplicação, facilita a realização de testes e evita que o roteamento dependa de estado global.

Um único ServeMux pode registrar diversas rotas da aplicação.

```
mux := http.NewServeMux()

mux.HandleFunc("/", home)
mux.HandleFunc("/users", users)
mux.HandleFunc("/lists", lists)
mux.HandleFunc("/items", items)
```

Quando uma requisição é recebida, o roteador compara o caminho solicitado com os padrões registrados e executa o *handler* correspondente.

A partir do **Go 1.22**, o ServeMux passou a oferecer recursos de roteamento mais completos. Além dos caminhos tradicionais, tornou-se possível registrar rotas considerando também o método HTTP e definir parâmetros diretamente no padrão da rota.

```
mux.HandleFunc("GET /users", listUsers)
mux.HandleFunc("POST /users", createUser)
mux.HandleFunc("GET /users/{id}", getUser)
mux.HandleFunc("DELETE /users/{id}", deleteUser)
```

Antes dessa versão, era comum registrar apenas o caminho e verificar manualmente o método

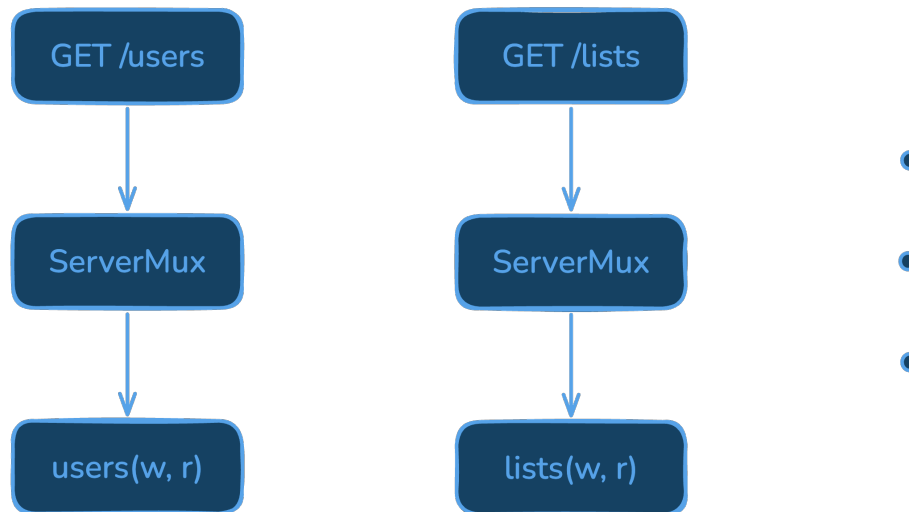


Figura 4.1: Serve Multiplexer

HTTP dentro do *handler*.

```
func users(w http.ResponseWriter, r *http.Request) {  
    switch r.Method {  
    case http.MethodGet:  
        listUsers(w, r)  
  
    case http.MethodPost:  
        createUser(w, r)  
  
    default:  
        http.Error(w, "Método não permitido.", http.StatusMethodNotAllowed)  
    }  
}
```

Embora essa técnica ainda funcione, ela concentra múltiplas responsabilidades em uma única função e dificulta a organização da aplicação à medida que o número de recursos aumenta. Em uma API como a GoList, isso faria com que um mesmo *handler* precisasse decidir entre listar, criar, consultar, atualizar ou remover recursos.

As melhorias introduzidas no ServeMux reduziram significativamente a necessidade de bibliotecas externas de roteamento para grande parte das APIs. Hoje, utilizando apenas a biblioteca padrão, é possível construir aplicações organizadas, legíveis e suficientemente flexíveis para muitos projetos.

Nos próximos tópicos veremos como aproveitar esses recursos para registrar rotas específicas para cada método HTTP, trabalhar com parâmetros de caminho e organizar as rotas da GoList de maneira clara e progressiva.

4.2 Métodos HTTP

Uma API raramente executa apenas uma operação sobre um recurso. Normalmente, um mesmo caminho pode representar diferentes ações, dependendo do método HTTP utilizado.

Considere o recurso `/lists`, que representa as listas de compras da GoList. A URL pode permanecer a mesma, mas o significado da requisição muda conforme o método usado.

Método	Recurso	Operação
GET	<code>/lists</code>	Listar listas
POST	<code>/lists</code>	Criar uma nova lista
GET	<code>/lists/{id}</code>	Consultar uma lista
PUT	<code>/lists/{id}</code>	Atualizar uma lista
DELETE	<code>/lists/{id}</code>	Remover uma lista

Essa separação faz parte das convenções usadas em APIs HTTP e permite que a interface seja consistente e previsível.

Como vimos no tópico anterior, a partir do **Go 1.22** o `ServeMux` passou a considerar o método HTTP como parte do padrão da rota. Dessa forma, cada operação pode possuir seu próprio *handler*.

```
mux := http.NewServeMux()

mux.HandleFunc("GET /lists", listLists)
mux.HandleFunc("POST /lists", createList)

mux.HandleFunc("GET /lists/{id}", getList)
mux.HandleFunc("PUT /lists/{id}", updateList)
mux.HandleFunc("DELETE /lists/{id}", deleteList)
```

Agora cada função possui uma responsabilidade mais clara, tornando o código mais simples de compreender, testar e evoluir.

Outra vantagem importante é que o próprio `ServeMux` passa a rejeitar automaticamente métodos incompatíveis. Se uma requisição for enviada para um caminho existente utilizando um método não registrado, a biblioteca padrão responderá com o código **405 Method Not Allowed**, sem que seja necessário escrever código adicional.

Por exemplo, considerando apenas a rota abaixo:

```
mux.HandleFunc("GET /lists", listLists)
```

As seguintes requisições terão comportamentos distintos.

Requisição	Resultado
GET <code>/lists</code>	Executa <code>listLists</code>

Requisição	Resultado
POST /lists	Retorna 405 Method Not Allowed
DELETE /lists	Retorna 405 Method Not Allowed

Caso o caminho solicitado não exista, a resposta continuará sendo **404 Not Found**, como ocorre tradicionalmente.

Essa distinção é importante. O código **404** indica que o recurso não foi encontrado, enquanto o **405** informa que o recurso existe, mas não aceita o método utilizado.

Na API da GoList, praticamente todas as rotas serão registradas utilizando esse novo formato. Isso permitirá separar claramente cada operação em uma função específica, deixando o roteamento mais declarativo e eliminando a necessidade de verificar manualmente `r.Method` na maioria dos *handlers*.

A adoção do roteamento por método representa uma das maiores evoluções do ServeMux. Além de reduzir código repetitivo, ela aproxima a biblioteca padrão das funcionalidades oferecidas pelos principais roteadores da comunidade, mantendo a simplicidade e a filosofia da linguagem Go.

4.3 Agrupamento de rotas

À medida que uma API evolui, o número de rotas cresce rapidamente. Uma aplicação que inicialmente possui apenas dois ou três recursos pode, em pouco tempo, passar a registrar dezenas de padrões diferentes.

Considere uma versão simplificada das rotas iniciais da GoList.

```
GET    /users
POST   /users

GET    /lists
POST   /lists
GET    /lists/{id}
PUT    /lists/{id}
DELETE /lists/{id}

POST   /lists/{id}/items
GET    /items/{id}
PATCH /items/{id}
DELETE /items/{id}
```

Registrar todas essas rotas em sequência funciona, mas logo torna o código difícil de navegar.

```
mux := http.NewServeMux()

mux.HandleFunc("GET /users", listUsers)
mux.HandleFunc("POST /users", createUser)

mux.HandleFunc("GET /lists", listLists)
mux.HandleFunc("POST /lists", createList)
mux.HandleFunc("GET /lists/{id}", getList)
mux.HandleFunc("PUT /lists/{id}", updateList)
mux.HandleFunc("DELETE /lists/{id}", deleteList)

mux.HandleFunc("POST /lists/{id}/items", createItem)
mux.HandleFunc("GET /items/{id}", getItem)
mux.HandleFunc("PATCH /items/{id}", updateItem)
mux.HandleFunc("DELETE /items/{id}", deleteItem)
```

Embora a biblioteca padrão não forneça um mecanismo próprio de agrupamento de rotas, semelhante ao encontrado em alguns roteadores externos, é perfeitamente possível organizar os registros de maneira clara utilizando apenas recursos da linguagem.

Uma prática bastante comum consiste em agrupar os registros por recurso.

```
mux := http.NewServeMux()

// Usuários
mux.HandleFunc("GET /users", listUsers)
mux.HandleFunc("POST /users", createUser)

// Listas
mux.HandleFunc("GET /lists", listLists)
mux.HandleFunc("POST /lists", createList)
mux.HandleFunc("GET /lists/{id}", getList)
mux.HandleFunc("PUT /lists/{id}", updateList)
mux.HandleFunc("DELETE /lists/{id}", deleteList)

// Itens
mux.HandleFunc("POST /lists/{id}/items", createItem)
mux.HandleFunc("GET /items/{id}", getItem)
mux.HandleFunc("PATCH /items/{id}", updateItem)
mux.HandleFunc("DELETE /items/{id}", deleteItem)
```

Mesmo sendo uma alteração simples, essa organização facilita a leitura do código e torna mais rápido localizar uma determinada rota.

Em aplicações maiores, é comum dar um passo além e separar o registro das rotas em funções específicas para cada recurso.

```
func registerUserRoutes(mux *http.ServeMux) {
    mux.HandleFunc("GET /users", listUsers)
    mux.HandleFunc("POST /users", createUser)
}

func registerListRoutes(mux *http.ServeMux) {
    mux.HandleFunc("GET /lists", listLists)
    mux.HandleFunc("POST /lists", createList)
    mux.HandleFunc("GET /lists/{id}", getList)
    mux.HandleFunc("PUT /lists/{id}", updateList)
    mux.HandleFunc("DELETE /lists/{id}", deleteList)
}

func registerItemRoutes(mux *http.ServeMux) {
    mux.HandleFunc("POST /lists/{id}/items", createItem)
    mux.HandleFunc("GET /items/{id}", getItem)
    mux.HandleFunc("PATCH /items/{id}", updateItem)
    mux.HandleFunc("DELETE /items/{id}", deleteItem)
}
```

A função principal passa então a ter apenas a responsabilidade de criar o roteador e registrar os grupos de rotas.

```
func main() {
    mux := http.NewServeMux()

    registerUserRoutes(mux)
    registerListRoutes(mux)
    registerItemRoutes(mux)

    http.ListenAndServe(":8080", mux)
}
```

Essa abordagem apresenta algumas vantagens importantes:

- cada função registra apenas as rotas de um único recurso;
- a responsabilidade de cada parte da aplicação fica bem definida;
- novas rotas podem ser adicionadas sem modificar funções não relacionadas;
- o arquivo `main.go` permanece pequeno e fácil de compreender.

É importante observar que esse agrupamento é apenas uma forma de organização do código. O `ServeMux` continua sendo uma única estrutura contendo todas as rotas da aplicação. Não existe qualquer hierarquia ou sub-roteador sendo criado; apenas distribuímos o registro das rotas em funções menores para melhorar a legibilidade.

Esse estilo de organização é bastante utilizado em aplicações desenvolvidas exclusivamente com a biblioteca padrão. Ele não muda o funcionamento do roteador, mas cria uma divisão lógica que ajuda o projeto a crescer sem transformar o registro das rotas em uma lista difícil de acompanhar.

No próximo tópico, veremos como levar essa mesma ideia um passo adiante, distribuindo o registro das rotas em arquivos e, mais tarde, em pacotes próprios.

4.4 Organização das rotas

O agrupamento de rotas melhora significativamente a legibilidade do código, mas, à medida que uma API cresce, torna-se igualmente importante definir onde essas rotas serão registradas.

Nos exemplos apresentados até aqui, todo o roteamento foi declarado no arquivo `main.go`. Essa abordagem funciona bem para aplicações pequenas, porém rapidamente se torna difícil de manter quando novos recursos passam a fazer parte do projeto.

Imagine que a GoList evolua para oferecer gerenciamento de usuários, listas, membros, itens, autenticação, upload de arquivos e webhooks. Concentrar todas essas rotas em um único arquivo produziria um código extenso e de difícil navegação.

Uma solução simples consiste em distribuir o registro das rotas entre diferentes arquivos, ainda dentro de uma estrutura pequena.

```
golist/  
├── go.mod  
├── main.go  
├── routes.go  
├── users.go  
├── lists.go  
└── items.go
```

Nesse primeiro momento, `main.go` pode ficar responsável por inicializar o servidor, enquanto `routes.go` concentra a função que registra os grupos de rotas.

```
func registerRoutes(mux *http.ServeMux) {  
    registerUserRoutes(mux)  
    registerListRoutes(mux)  
    registerItemRoutes(mux)  
}
```

Os demais arquivos podem reunir os *handlers* e o registro das rotas relacionadas a cada recurso.

```
func registerUserRoutes(mux *http.ServeMux) {  
    mux.HandleFunc("GET /users", listUsers)  
    mux.HandleFunc("POST /users", createUser)  
    mux.HandleFunc("GET /users/{id}", getUser)  
}
```

```
func registerListRoutes(mux *http.ServeMux) {
    mux.HandleFunc("GET /lists", listLists)
    mux.HandleFunc("POST /lists", createList)
    mux.HandleFunc("GET /lists/{id}", getList)
    mux.HandleFunc("PUT /lists/{id}", updateList)
    mux.HandleFunc("DELETE /lists/{id}", deleteList)
}
```

```
func registerItemRoutes(mux *http.ServeMux) {
    mux.HandleFunc("POST /lists/{id}/items", createItem)
    mux.HandleFunc("GET /items/{id}", getItem)
    mux.HandleFunc("PATCH /items/{id}", updateItem)
    mux.HandleFunc("DELETE /items/{id}", deleteItem)
}
```

A função principal permanece responsável apenas pela inicialização da aplicação.

```
func main() {
    mux := http.NewServeMux()

    registerRoutes(mux)

    http.ListenAndServe(":8080", mux)
}
```

Essa organização oferece diversas vantagens. O arquivo principal permanece pequeno, novas funcionalidades podem ser adicionadas sem modificar código não relacionado e cada arquivo passa a ter uma responsabilidade mais fácil de entender.

À medida que a aplicação cresce, é comum evoluir ainda mais essa estrutura, movendo cada grupo de rotas para um pacote específico. No caso da GoList, essa evolução acompanhará a arquitetura modular apresentada no Apêndice A.

```
internal/
├── modules/
│   ├── users/
│   │   └── transport/
│   │       └── http.go
│   ├── lists/
│   │   └── transport/
│   │       └── http.go
│   └── items/
│       └── transport/
│           └── http.go
```


Nesse modelo, cada módulo passa a reunir o código HTTP relacionado ao seu próprio recurso. O registro das rotas e os *handlers* continuam próximos, mas deixam de ficar misturados com recursos de outras partes da aplicação.

Essa separação segue um princípio importante do desenvolvimento de software: **cada componente deve possuir uma única responsabilidade**. O código de usuários não precisa dividir o mesmo arquivo com o código de listas, itens ou autenticação apenas porque todos eles são acessados por HTTP.

É importante destacar que não existe uma estrutura “oficial” para organizar rotas em aplicações Go. A linguagem procura oferecer mecanismos simples e flexíveis, permitindo que cada projeto adote a organização mais adequada às suas necessidades.

Ao longo deste livro, a GoList seguirá uma evolução incremental. Inicialmente, utilizaremos uma estrutura bastante simples para manter o foco nos conceitos de HTTP e da biblioteca padrão. Conforme novos recursos forem sendo introduzidos, reorganizaremos gradualmente o projeto, aproximando-o da estrutura modular descrita no Apêndice A. Dessa forma, o leitor acompanhará não apenas a implementação das funcionalidades, mas também a evolução natural da arquitetura de uma API escrita em Go.

4.5 Rotas da GoList

Depois de estudar o ServeMux, o roteamento por método e algumas formas de organizar o registro das rotas, podemos aplicar essas ideias à primeira versão da **GoList**.

Como definido no Apêndice A, nesta etapa do desenvolvimento a aplicação disponibilizará recursos relacionados aos principais elementos do domínio: usuários, listas de compras e itens.

O contrato HTTP inicial da API será composto pelas seguintes rotas.

Usuários

```
POST   /users
GET    /users
GET    /users/{id}
PUT    /users/{id}
DELETE /users/{id}
```

Listas

```
POST   /lists
GET    /lists
GET    /lists/{id}
PUT    /lists/{id}
DELETE /lists/{id}
```

Itens

```
POST   /lists/{id}/items
GET    /items/{id}
```

```
PATCH /items/{id}
DELETE /items/{id}
```

Cada rota representa uma operação específica da aplicação. Embora alguns caminhos sejam semelhantes, o método HTTP define qual ação será executada.

Por exemplo, o recurso `/lists` admite diferentes operações.

Método	Rota	Descrição
POST	<code>/lists</code>	Cria uma nova lista de compras.
GET	<code>/lists</code>	Lista todas as listas disponíveis.
GET	<code>/lists/{id}</code>	Obtém os dados de uma lista específica.
PUT	<code>/lists/{id}</code>	Atualiza uma lista existente.
DELETE	<code>/lists/{id}</code>	Remove uma lista.

Utilizando o `ServeMux`, essas rotas podem ser registradas de forma direta.

```
func registerListRoutes(mux *http.ServeMux) {
    mux.HandleFunc("GET /lists", listLists)
    mux.HandleFunc("POST /lists", createList)
    mux.HandleFunc("GET /lists/{id}", getList)
    mux.HandleFunc("PUT /lists/{id}", updateList)
    mux.HandleFunc("DELETE /lists/{id}", deleteList)
}
```

Da mesma forma, as rotas dos demais recursos ficam agrupadas em funções próprias.

```
func registerUserRoutes(mux *http.ServeMux) {
    mux.HandleFunc("GET /users", listUsers)
    mux.HandleFunc("POST /users", createUser)
    mux.HandleFunc("GET /users/{id}", getUser)
    mux.HandleFunc("PUT /users/{id}", updateUser)
    mux.HandleFunc("DELETE /users/{id}", deleteUser)
}
```

```
func registerItemRoutes(mux *http.ServeMux) {
    mux.HandleFunc("POST /lists/{id}/items", createItem)
    mux.HandleFunc("GET /items/{id}", getItem)
    mux.HandleFunc("PATCH /items/{id}", updateItem)
    mux.HandleFunc("DELETE /items/{id}", deleteItem)
}
```

Uma função intermediária pode reunir o registro de todos os grupos de rotas.

```
func registerRoutes(mux *http.ServeMux) {  
    registerUserRoutes(mux)  
    registerListRoutes(mux)  
    registerItemRoutes(mux)  
}
```

Com isso, a função principal permanece responsável apenas por criar o roteador, registrar as rotas e iniciar o servidor.

```
func main() {  
    mux := http.NewServeMux()  
  
    registerRoutes(mux)  
  
    http.ListenAndServe(":8080", mux)  
}
```

Essa estrutura ainda é simples, mas já estabelece uma base importante: cada recurso possui um conjunto claro de rotas, cada operação aponta para um *handler* específico e o roteamento fica concentrado em pontos fáceis de localizar.

Ao longo dos capítulos, um espelho simplificado do projeto será mantido de forma incremental nos diretórios de código do volume. Para este capítulo, esse código ficará em `vol_2-desenvolvimento-de-apis/codigo/cap04/go-list/`. Essa pasta não representa ainda a versão final da GoList. Ela servirá como ponto de acompanhamento para o código construído passo a passo, de acordo com os conceitos apresentados em cada capítulo.

À medida que novos capítulos forem desenvolvidos, a GoList evoluirá naturalmente. Serão adicionadas rotas para autenticação, compartilhamento de listas, gerenciamento de membros, upload de anexos, notificações em tempo real, webhooks e outras funcionalidades. Quando isso acontecer, a organização do roteamento também evoluirá junto com o projeto, sem abandonar os princípios apresentados neste capítulo.

4.6 Atividade: rotas da GoList

Nesta atividade, você implementará o roteamento inicial da **GoList** usando apenas os recursos apresentados neste capítulo.

O objetivo não é construir ainda as regras da aplicação, nem persistir dados, nem responder JSON estruturado. O foco é criar a primeira forma executável da API: um servidor HTTP com rotas organizadas por recurso e *handlers* simples o suficiente para confirmar que cada endereço foi registrado corretamente.

Implemente uma aplicação em Go com um `http.ServeMux` e registre as rotas de usuários, listas de compras e itens.

Usuários

POST /users

```
GET    /users
GET    /users/{id}
PUT    /users/{id}
DELETE /users/{id}
```

Listas

```
POST   /lists
GET    /lists
GET    /lists/{id}
PUT    /lists/{id}
DELETE /lists/{id}
```

Itens

```
POST   /lists/{id}/items
GET    /items/{id}
PATCH /items/{id}
DELETE /items/{id}
```

Separe o registro das rotas em funções pequenas, uma para cada grupo de recurso.

```
func registerUserRoutes(mux *http.ServeMux) {}
func registerListRoutes(mux *http.ServeMux) {}
func registerItemRoutes(mux *http.ServeMux) {}
func registerRoutes(mux *http.ServeMux) {}
```

A função `main` deve criar o `ServeMux`, chamar `registerRoutes` e iniciar o servidor na porta 8080.

```
func main() {
    mux := http.NewServeMux()

    registerRoutes(mux)

    http.ListenAndServe(":8080", mux)
}
```

Neste momento, os *handlers* podem retornar apenas texto simples, como `create user`, `list lists` ou `update item`. Essa resposta mínima ajuda a testar o roteamento sem antecipar assuntos que serão tratados nos próximos capítulos, como leitura do corpo da requisição, validação, serialização JSON, regras de domínio e persistência.

Depois de iniciar o servidor, teste algumas rotas com `curl`.

```
curl -i http://localhost:8080/users
curl -i -X POST http://localhost:8080/lists
curl -i http://localhost:8080/lists/123
curl -i -X PATCH http://localhost:8080/items/456
```

Observe também o comportamento quando o método HTTP não corresponde à rota registrada.

```
curl -i -X DELETE http://localhost:8080/users
```

Como a rota DELETE /users não foi registrada, o servidor não deve encaminhar essa requisição para o mesmo *handler* usado por GET /users.

A solução de referência desta atividade está em `vol_2-desenvolvimento-de-apis/codigo/cap04/go-list/`.

Capítulo 5

Json

Até este ponto, nossas aplicações trocaram informações utilizando texto simples. Embora isso seja suficiente para compreender o funcionamento do protocolo HTTP, aplicações reais normalmente precisam transmitir dados estruturados entre clientes e servidores.

É nesse contexto que o **JSON** (*JavaScript Object Notation*) se tornou um dos formatos de troca de dados mais utilizados no desenvolvimento de APIs HTTP.

Sua popularidade decorre de algumas características importantes:

- possui uma sintaxe simples e legível;
- é independente de linguagem de programação;
- é suportado por praticamente todas as plataformas;
- representa naturalmente objetos, listas, números, textos e valores booleanos.

Considere o cadastro de um usuário na GoList. Em vez de enviar informações separadas em parâmetros da URL ou em texto livre no corpo da requisição, o cliente pode enviar um documento JSON contendo os dados necessários.

```
{  
  "name": "Maria Oliveira",  
  "email": "maria@example.com",  
  "active": true  
}
```

Da mesma forma, a resposta da API também pode utilizar JSON para representar o recurso criado.

```
{  
  "id": 15,  
  "name": "Maria Oliveira",  
  "email": "maria@example.com",  
  "active": true  
}
```

Observe que a estrutura do documento é composta por pares **chave-valor**, permitindo representar dados de forma organizada. Além disso, o formato suporta listas, possibilitando que uma API retorne coleções inteiras de recursos.

```
[
  {
    "id": 1,
    "name": "Lista do Mercado"
  },
  {
    "id": 2,
    "name": "Churrasco"
  }
]
```

No ecossistema Go, o suporte a JSON é fornecido pela biblioteca padrão por meio do pacote `encoding/json`. Esse pacote permite converter valores da linguagem em documentos JSON e realizar o processo inverso, transformando documentos JSON recebidos em valores Go.

Essa conversão ocorre de forma natural para os principais tipos da linguagem. Valores como textos, números, booleanos, listas, mapas e `structs` possuem representações diretas em JSON. Por isso, é comum representar os dados de entrada e saída de uma API utilizando `structs`, deixando que o pacote `encoding/json` faça a conversão entre o código Go e o documento enviado pela rede.

Ao longo deste capítulo veremos como utilizar o pacote `encoding/json` para realizar essas conversões. Estudaremos como serializar valores Go em JSON, interpretar documentos recebidos nas requisições, personalizar os nomes dos campos utilizando *struct tags*, criar objetos específicos para transporte de dados (*DTOs*) e produzir respostas HTTP padronizadas utilizando JSON.

Ao final do capítulo, a API da **GoList** será capaz de receber e retornar informações estruturadas, aproximando-se da forma como aplicações web e APIs HTTP são desenvolvidas na prática.

5.1 `encoding/json`

A biblioteca padrão de Go fornece suporte ao formato JSON por meio do pacote `encoding/json`. Ele é responsável por converter valores da linguagem para documentos JSON e realizar a operação inversa, transformando documentos JSON em valores Go.

Na prática, muitas APIs desenvolvidas em Go utilizam esse pacote para receber dados enviados pelos clientes e produzir respostas para requisições HTTP.

Para utilizá-lo, basta importá-lo normalmente.

```
import "encoding/json"
```

Embora o pacote possua diversas funções e interfaces, duas delas concentram boa parte do trabalho inicial realizado em aplicações web:

- `json.Marshal`, que converte um valor Go em JSON;
- `json.Unmarshal`, que converte um documento JSON em um valor Go.

Essas duas funções serão estudadas detalhadamente nas próximas seções.

5.1.1 Tipos suportados

O pacote `encoding/json` conhece os principais tipos da linguagem e sabe como representá-los em JSON.

Considere a seguinte estrutura, que poderia representar um usuário da GoList.

```
type User struct {  
    ID      int  
    Name    string  
    Email   string  
    Active  bool  
}
```

Ao ser convertida para JSON, cada campo torna-se um atributo do objeto.

```
{  
  "ID": 1,  
  "Name": "Maria",  
  "Email": "maria@example.com",  
  "Active": true  
}
```

O mesmo acontece com outros tipos fundamentais.

Go	JSON
bool	true ou false
string	string
int, float64	number
struct	object
slice e array	array
map[string]T	object
nil	null

Essa correspondência é um dos motivos pelos quais structs são amplamente utilizadas para representar dados recebidos e enviados por uma API.

5.1.2 Exportação dos campos

Um detalhe importante é que o pacote `encoding/json` somente acessa **campos exportados** de uma struct, ou seja, aqueles cujo nome começa com letra maiúscula.

Considere o exemplo abaixo.

```
type User struct {  
    ID      int  
    Name    string  
    email   string  
    password string  
}
```

Ao converter esse valor para JSON, apenas os campos exportados serão incluídos.

```
{  
  "ID": 1,  
  "Name": "Maria"  
}
```

Os campos `email` e `password` são ignorados porque não são exportados.

Essa regra está relacionada ao próprio mecanismo de visibilidade da linguagem Go e vale para diversos pacotes da biblioteca padrão que utilizam reflexão (*reflection*), incluindo `encoding/json`.

5.1.3 Leitura e escrita

Em uma API HTTP, o pacote `encoding/json` normalmente é utilizado em dois momentos distintos.

Quando o cliente envia uma requisição contendo JSON, o servidor precisa interpretar esse documento e convertê-lo para uma `struct`.

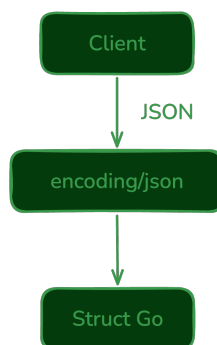


Figura 5.1: JSON para struct Go

Quando a aplicação produz uma resposta, ocorre o processo inverso.

Esses dois fluxos estão presentes em grande parte das operações de uma API HTTP.

5.1.4 Simplicidade da biblioteca padrão

Apesar de bastante completo, o pacote `encoding/json` possui uma API pequena para os usos mais comuns. Na maioria das aplicações introdutórias, poucas funções são suficientes para realizar as conversões necessárias.

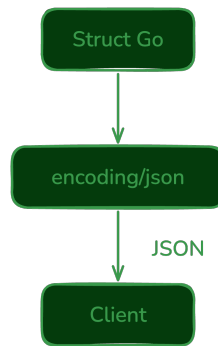


Figura 5.2: JSON para struct Go

Essa simplicidade está alinhada com a filosofia da linguagem Go: oferecer ferramentas poderosas por meio de interfaces reduzidas e fáceis de compreender.

Nas próximas seções estudaremos como utilizar `json.Marshal` e `json.Unmarshal` para converter valores entre Go e JSON, personalizar os nomes dos campos por meio de *struct tags* e produzir respostas HTTP padronizadas para a API da GoList.

5.2 Marshal

A função `json.Marshal` é utilizada para converter um valor Go em um documento JSON. Esse processo é conhecido como **serialização**, pois transforma uma estrutura de dados em uma sequência de bytes que pode ser armazenada, transmitida ou enviada em uma resposta HTTP.

Sua assinatura é bastante simples.

```
func Marshal(v any) ([]byte, error)
```

O parâmetro `v` representa o valor que será convertido para JSON. Em caso de sucesso, a função retorna um `[]byte` contendo o documento JSON gerado. Caso ocorra algum problema durante a conversão, um valor do tipo `error` também será retornado.

5.2.1 Convertendo uma struct

Considere a seguinte estrutura, representando um usuário da GoList.

```
1 package main
2
3 import (
4     "encoding/json"
5     "fmt"
6 )
7
8 type User struct {
9     ID      int
10    Name    string
11    Email   string
12    Active  bool
13 }
14
15 func main() {
16     user := User{
17         ID:      1,
18         Name:    "Maria Oliveira",
19         Email:   "maria@example.com",
20         Active:  true,
21     }
22
23     data, err := json.Marshal(user)
24     if err != nil {
25         panic(err)
26     }
27
28     fmt.Println(string(data))
29 }
```

A saída produzida será:

```
{"ID":1,"Name":"Maria Oliveira","Email":"maria@example.com","Active":true}
```

Observe que o resultado é um documento JSON válido representado como uma sequência de bytes. Para exibi-lo na tela, foi necessário convertê-lo para string.

5.2.2 Convertendo outros tipos

O `Marshal` não está limitado a structs. Ele pode serializar diversos tipos suportados pelo pacote `encoding/json`.

Uma slice de inteiros, por exemplo:

```
numbers := []int{10, 20, 30}

data, err := json.Marshal(numbers)
if err != nil {
    panic(err)
}

fmt.Println(string(data))
```

Produz:

```
[10,20,30]
```

Também é possível converter mapas.

```
list := map[string]any{
    "id":    1,
    "name":  "Lista do Mercado",
    "shared": true,
}

data, err := json.Marshal(list)
if err != nil {
    panic(err)
}

fmt.Println(string(data))
```

Resultado:

```
{"id":1,"name":"Lista do Mercado","shared":true}
```

5.2.3 JSON formatado

O JSON produzido por `Marshal` é compacto, não contendo espaços ou quebras de linha desnecessárias. Esse formato é ideal para transmissão pela rede, pois reduz a quantidade de dados enviados.

Entretanto, durante o desenvolvimento pode ser útil gerar um documento mais legível. Para isso, a biblioteca padrão fornece a função `json.MarshalIndent`.

```
data, err := json.MarshalIndent(user, "", "  ")
if err != nil {
    panic(err)
}

fmt.Println(string(data))
```

O resultado passa a ser:

```
{
  "ID": 1,
  "Name": "Maria Oliveira",
  "Email": "maria@example.com",
  "Active": true
}
```

O primeiro parâmetro adicional define o prefixo de cada linha, enquanto o segundo especifica a indentação utilizada.

Embora seja útil para depuração, documentação e arquivos de configuração, `MarshalIndent` normalmente não é utilizado em respostas de APIs, já que aumenta o tamanho do documento transmitido.

5.2.4 Tratando erros

Nem todos os valores Go podem ser representados em JSON. Alguns tipos, como funções, canais e números especiais (NaN e Infinity), não possuem representação válida e fazem com que `Marshal` retorne um erro.

```
1 package main
2
3 import (
4     "encoding/json"
5     "fmt"
6 )
7
8 func main() {
9     _, err := json.Marshal(func() {})
10
11     fmt.Println(err)
12 }
```

Saída:

```
json: unsupported type: func()
```

Por esse motivo, mesmo quando o erro parece improvável, é uma boa prática verificar o valor retornado por `Marshal`.

5.2.5 Utilização em APIs

Em aplicações HTTP, `json.Marshal` pode ser utilizado para produzir o corpo das respostas enviadas ao cliente.

O fluxo normalmente ocorre da seguinte forma:

Nos próximos tópicos veremos que, em muitas situações, nem será necessário chamar `Marshal` diretamente. O próprio pacote `encoding/json` oferece um `Encoder` capaz de escrever o JSON

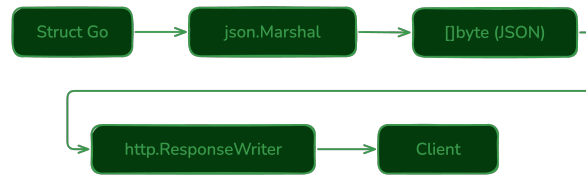


Figura 5.3: Marshal Structs

diretamente na resposta HTTP, simplificando ainda mais a implementação de uma API.

5.3 Unmarshal

Enquanto `json.Marshal` converte valores Go em JSON, a função `json.Unmarshal` realiza o processo inverso: ela interpreta um documento JSON e preenche um valor Go com os dados encontrados.

Esse processo é conhecido como **desserialização** (*deserialization*) e é utilizado em APIs para interpretar o corpo (*body*) das requisições enviadas pelos clientes.

Sua assinatura é a seguinte:

```
func Unmarshal(data []byte, v any) error
```

O primeiro parâmetro contém o documento JSON que será interpretado. O segundo representa o valor que receberá os dados.

Diferentemente de `Marshal`, o segundo argumento deve apontar para o valor que será preenchido. Por isso, na maioria dos usos, passamos um **ponteiro** para `Unmarshal`.

5.3.1 Convertendo JSON para uma struct

Considere o seguinte documento JSON, representando um usuário da GoList.

```
{
  "ID": 1,
  "Name": "Maria Oliveira",
  "Email": "maria@example.com",
  "Active": true
}
```

Podemos convertê-lo para uma struct da seguinte forma.

```
1 package main
2
3 import (
4     "encoding/json"
5     "fmt"
6 )
7
8 type User struct {
9     ID      int
10    Name    string
11    Email   string
12    Active  bool
13 }
14
15 func main() {
16     data := []byte(`{
17         "ID": 1,
18         "Name": "Maria Oliveira",
19         "Email": "maria@example.com",
20         "Active": true
21     }`)
22
23     var user User
24
25     err := json.Unmarshal(data, &user)
26     if err != nil {
27         panic(err)
28     }
29
30     fmt.Printf("%+v\n", user)
31 }
```

Saída:

```
{ID:1 Name:Maria Oliveira Email:maria@example.com Active:true}
```

Observe que foi utilizado `&user` em vez de `user`. Como a função precisa preencher os campos da estrutura, é necessário fornecer seu endereço de memória.

5.3.2 Convertendo listas

O `Unmarshal` também é capaz de interpretar arrays JSON.

```
[
    "Arroz",
    "Feijão",
    "Leite"
]
```


O código correspondente é bastante simples.

```
var items []string

err := json.Unmarshal(data, &items)
if err != nil {
    panic(err)
}
```

Após a conversão, a variável `items` conterá os três elementos presentes no documento JSON.

5.3.3 Convertendo mapas

Quando a estrutura do JSON não é conhecida previamente, pode ser conveniente utilizar um mapa.

```
jsonData := []byte(`{
    "name": "Lista do Mercado",
    "shared": true
}`)

var data map[string]any

err := json.Unmarshal(jsonData, &data)
if err != nil {
    panic(err)
}

fmt.Println(data["name"])
fmt.Println(data["shared"])
```

Essa abordagem oferece maior flexibilidade, mas perde a segurança de tipos proporcionada pelas structs. Em APIs, normalmente prefere-se utilizar estruturas bem definidas.

5.3.4 Tratando erros

Se o documento recebido não for um JSON válido, `Unmarshal` retornará um erro.

```
data := []byte(`{"name":"Maria"}`)

err := json.Unmarshal(data, &user)

fmt.Println(err)
```

Saída:

```
unexpected end of JSON input
```

Também podem ocorrer erros quando o conteúdo do JSON é incompatível com os tipos definidos

na estrutura.

```
{
  "ID": "abc",
  "Name": "Maria"
}
```

Como ID deveria conter um número inteiro, a conversão falhará.

```
json: cannot unmarshal string into Go struct field User.ID of type int
```

Por esse motivo, toda chamada a `json.Unmarshal` deve verificar o erro retornado antes de utilizar os dados obtidos.

5.3.5 Utilização em APIs

Em aplicações HTTP, `Unmarshal` pode ser utilizado para interpretar o corpo das requisições.

O fluxo normalmente ocorre da seguinte forma:

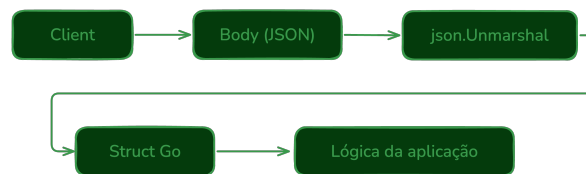


Figura 5.4: Ummarshal Structs

Na prática, o JSON recebido no corpo da requisição é lido, convertido para uma `struct` e então verificado antes de ser utilizado pela aplicação.

Nos próximos tópicos veremos como controlar o nome dos campos utilizando *struct tags* e como definir estruturas específicas para transportar dados entre clientes e servidores, conhecidas como **DTOs**.

5.4 Struct tags

Até agora, vimos que o pacote `encoding/json` utiliza automaticamente os nomes dos campos de uma `struct` para gerar os atributos do documento JSON.

Considere a seguinte estrutura, representando um usuário da GoList.

```
type User struct {
    ID      int
    Name    string
    Email   string
    Active  bool
}
```

Ao convertê-la para JSON, o resultado será:

```
{
  "ID": 1,
  "Name": "Maria Oliveira",
  "Email": "maria@example.com",
  "Active": true
}
```

Embora esse comportamento seja correto, ele nem sempre corresponde ao formato utilizado em APIs HTTP. É comum que documentos JSON usem nomes menores e independentes da convenção de Go, como `id`, `name`, `email` e `active`.

Para personalizar a forma como um campo é representado em JSON, Go utiliza **struct tags**.

Uma *struct tag* é uma anotação associada a um campo da estrutura. Ela fornece informações adicionais para pacotes que utilizam reflexão (*reflection*), como `encoding/json`, `database/sql` e diversas bibliotecas externas.

Sua sintaxe consiste em uma string delimitada por crases (```), posicionada logo após a declaração do campo.

```
type User struct {
    ID      int    `json:"id"`
    Name    string `json:"name"`
    Email   string `json:"email"`
    Active  bool   `json:"active"`
}
```

Agora, ao utilizar `json.Marshal`, o resultado será:

```
{
  "id": 1,
  "name": "Maria Oliveira",
  "email": "maria@example.com",
  "active": true
}
```

Observe que apenas a representação em JSON foi alterada. Os nomes dos campos na linguagem Go permanecem inalterados.

5.4.1 Leitura e escrita

As *struct tags* são utilizadas tanto por `Marshal` quanto por `Unmarshal`.

Considere o seguinte JSON.

```
{
  "id": 10,
  "name": "Carlos",
  "email": "carlos@example.com",
  "active": true
}
```

```
}
```

A conversão ocorre normalmente.

```
var user User

err := json.Unmarshal(data, &user)
if err != nil {
    panic(err)
}
```

Mesmo que o JSON utilize `id` e a estrutura possua o campo `ID`, a associação é feita por meio da tag `json:"id"`.

5.4.2 Ignorando campos

Em algumas situações, determinados campos da estrutura não devem fazer parte do JSON.

Isso é bastante comum para informações internas da aplicação, como senhas, identificadores temporários ou dados utilizados apenas durante o processamento.

Para impedir que um campo seja serializado ou desserializado, utiliza-se a tag `-`.

```
type User struct {
    ID        int    `json:"id"`
    Name      string `json:"name"`
    Password  string `json:"- "`
}
```

Ao converter essa estrutura para JSON, o campo `Password` será ignorado.

```
{
  "id": 1,
  "name": "Maria Oliveira"
}
```

Da mesma forma, se o cliente enviar um atributo `password`, ele também será desconsiderado durante o `Unmarshal`.

5.4.3 Omitindo campos vazios

Outro recurso bastante utilizado é a opção `omitempty`.

Ela faz com que um campo seja omitido do JSON quando possuir o valor zero (*zero value*) do seu tipo.

```
type User struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Email string `json:"email,omitempty"`
}
```

Se o campo Email estiver vazio, o resultado será:

```
{
  "id": 1,
  "name": "Maria Oliveira"
}
```

Sem `omitempty`, o campo seria incluído normalmente.

```
{
  "id": 1,
  "name": "Maria Oliveira",
  "email": ""
}
```

Esse recurso é útil para evitar o envio de informações sem significado, mas deve ser usado com cuidado: em alguns contratos de API, um campo vazio e um campo ausente podem ter significados diferentes.

5.4.4 Múltiplas opções

As opções podem ser combinadas na própria tag.

```
type User struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Email string `json:"email,omitempty"`
}
```

O primeiro elemento define o nome do atributo (`email`), enquanto os demais configuram o comportamento da serialização.

5.4.5 Boas práticas

Algumas recomendações tornam o uso de *struct tags* mais consistente em APIs:

- utilize nomes consistentes no contrato JSON da API;
- evite expor nomes internos utilizados pela aplicação;
- utilize `omitempty` apenas quando a ausência do campo possuir o mesmo significado que seu valor zero;
- utilize `json:"-"` para impedir que informações sensíveis sejam expostas.

As *struct tags* permitem desacoplar a representação dos dados em Go da representação utilizada

pela API. Assim, é possível manter nomes idiomáticos na linguagem e, ao mesmo tempo, produzir documentos JSON consistentes com as convenções adotadas por clientes e outros serviços.

Nos próximos tópicos veremos como aproveitar esse recurso na definição de **DTOs** (*Data Transfer Objects*), estruturas criadas especificamente para representar os dados trocados entre clientes e servidores.

5.5 DTOs

Até este ponto, utilizamos `structs` para representar os dados manipulados pela aplicação. Em muitos casos, porém, a estrutura utilizada internamente não é a mesma que deve ser enviada ou recebida pela API.

Para resolver esse problema, é comum utilizar **DTOs** (*Data Transfer Objects*).

Um DTO é uma estrutura criada para transportar dados entre processos, como entre um cliente HTTP e uma API. Seu objetivo é definir quais informações fazem parte de uma requisição ou resposta, independentemente de como esses dados são armazenados ou processados internamente.

Considere a seguinte estrutura representando um usuário da GoList.

```
type User struct {  
    ID          int  
    Name        string  
    Email        string  
    PasswordHash string  
    CreatedAt    time.Time  
    UpdatedAt    time.Time  
}
```

Essa estrutura contém informações importantes para a aplicação, mas nem todas fazem parte do contrato público da API.

Por exemplo, uma resposta para consulta de usuário não deve retornar a senha criptografada. Em muitos casos, também não precisa retornar campos de auditoria, como datas internas de criação e atualização.

Em vez disso, podemos criar um DTO específico para essa resposta.

```
type UserResponse struct {  
    ID    int    `json:"id"`  
    Name  string `json:"name"`  
    Email string `json:"email"`  
}
```

Ao serializar um valor desse tipo, apenas os campos definidos no DTO serão enviados ao cliente.

Da mesma forma, uma requisição de criação de usuário também possui um formato próprio.

```
type CreateUserRequest struct {  
    Name      string `json:"name"`  
    Email     string `json:"email"`  
    Password  string `json:"password"`  
}
```

Observe que esse DTO possui o campo Password, enquanto a estrutura User possui PasswordHash. Isso acontece porque a senha informada pelo cliente será utilizada pela aplicação para gerar um hash antes de ser armazenada.

O fluxo completo pode ser representado da seguinte forma.

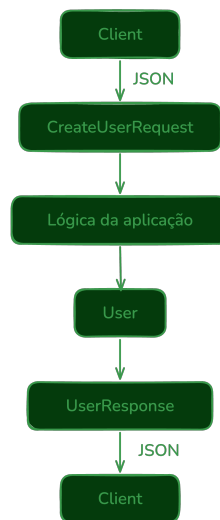


Figura 5.5: DTOs

Cada estrutura possui uma responsabilidade diferente:

- **CreateUserRequest** representa os dados recebidos na requisição;
- **User** representa a entidade utilizada pela aplicação;
- **UserResponse** representa os dados enviados na resposta.

Essa separação oferece diversas vantagens.

Primeiramente, ela reduz o risco de expor informações internas. Campos como senhas criptografadas, identificadores técnicos ou informações administrativas deixam de fazer parte das estruturas usadas diretamente nas respostas.

Além disso, os DTOs permitem que a API evolua sem impactar diretamente o modelo interno da aplicação. Alterações nessa estrutura interna nem sempre exigem mudanças no formato das requisições e respostas.

Outro benefício importante é preparar o caminho para validação. Como cada DTO representa uma operação específica, torna-se mais simples definir quais campos são esperados em cada contexto. Um cadastro de usuário, por exemplo, exige senha, enquanto uma atualização parcial pode permitir apenas a alteração do nome.

Na GoList, utilizaremos DTOs nas operações em que a separação entre entrada, saída e modelo

interno tornar o código mais claro. Cada recurso poderá possuir estruturas próprias para representar as informações recebidas e enviadas, mantendo uma separação entre o modelo da aplicação e o contrato exposto aos clientes.

Embora aplicações muito simples possam utilizar a mesma `struct` para todas essas responsabilidades, essa prática tende a gerar acoplamento conforme a API cresce. A utilização de DTOs ajuda a manter o código mais organizado, reduz a chance de exposição acidental de dados e facilita a evolução do contrato HTTP.

5.6 Respostas JSON

Grande parte das APIs HTTP utiliza JSON como formato padrão para suas respostas. Sempre que uma operação é concluída com sucesso ou quando ocorre um erro, o servidor pode enviar um documento JSON descrevendo o resultado da requisição.

Nos tópicos anteriores, vimos que `json.Marshal` converte uma estrutura Go em um documento JSON. Embora essa função possa ser utilizada diretamente, a biblioteca padrão oferece uma alternativa prática para aplicações HTTP: o tipo `json.Encoder`.

Considere a seguinte estrutura de resposta.

```
type UserResponse struct {  
    ID    int    `json:"id"`  
    Name  string  `json:"name"`  
    Email string  `json:"email"`  
}
```

Para enviá-la como resposta HTTP, basta criar um codificador associado ao `ResponseWriter`.

```
func getUser(w http.ResponseWriter, r *http.Request) {  
    user := UserResponse{  
        ID:    1,  
        Name:  "Maria Oliveira",  
        Email: "maria@example.com",  
    }  
  
    w.Header().Set("Content-Type", "application/json")  
  
    if err := json.NewEncoder(w).Encode(user); err != nil {  
        return  
    }  
}
```

O método `Encode` converte a estrutura para JSON e escreve o resultado diretamente na resposta HTTP. Isso evita a criação intermediária de um `[]byte`, tornando o código mais simples.

A resposta enviada ao cliente será semelhante à seguinte.


```
{  
  "id": 1,  
  "name": "Maria Oliveira",  
  "email": "maria@example.com"  
}
```

5.6.1 Definindo o tipo do conteúdo

Quando uma API retorna JSON, deve informar esse fato por meio do cabeçalho Content-Type.

```
w.Header().Set("Content-Type", "application/json")
```

Esse cabeçalho permite que clientes, navegadores e outras aplicações interpretem corretamente o conteúdo recebido.

Embora alguns clientes consigam identificar automaticamente o formato da resposta, definir o Content-Type é uma boa prática e deve fazer parte das respostas JSON da API.

5.6.2 Códigos de status

Além do corpo da resposta, uma API também deve retornar um código de status HTTP adequado para cada situação.

Uma criação bem-sucedida, por exemplo, normalmente utiliza o código **201 Created**.

```
w.Header().Set("Content-Type", "application/json")  
w.WriteHeader(http.StatusCreated)  
  
if err := json.NewEncoder(w).Encode(user); err != nil {  
    return  
}
```

Já uma consulta realizada com sucesso utiliza, em geral, o código **200 OK**, que é o comportamento padrão do ResponseWriter quando nenhum outro status é informado.

Outros códigos serão utilizados conforme o resultado da operação.

Situação	Status
Consulta realizada com sucesso	200 OK
Recurso criado	201 Created
Requisição inválida	400 Bad Request
Não autenticado	401 Unauthorized
Recurso não encontrado	404 Not Found
Erro interno da aplicação	500 Internal Server Error

Nos capítulos seguintes estudaremos esses códigos com mais profundidade e veremos como padronizar as respostas da API.

5.6.3 Respostas de erro

Assim como respostas de sucesso, erros também podem ser representados em JSON.

Em vez de retornar apenas uma mensagem em texto simples:

```
Usuário não encontrado.
```

é comum responder com uma estrutura como:

```
{  
  "error": "Usuário não encontrado."  
}
```

Isso facilita o processamento da resposta pelo cliente, que pode acessar diretamente o campo `error` sem precisar interpretar texto livre.

Uma possível implementação é:

```
type ErrorResponse struct {  
    Error string `json:"error"`  
}  
  
func writeError(w http.ResponseWriter, status int, message string) {  
    w.Header().Set("Content-Type", "application/json")  
    w.WriteHeader(status)  
  
    if err := json.NewEncoder(w).Encode(ErrorResponse{  
        Error: message,  
    }); err != nil {  
        return  
    }  
}
```

Essa função centraliza a geração de respostas de erro, evitando repetição de código e ajudando a manter o mesmo formato em diferentes handlers.

5.6.4 Padronização das respostas

À medida que uma API evolui, torna-se importante manter consistência entre as respostas produzidas. Clientes diferentes, como aplicações web, aplicativos móveis ou outros serviços, esperam encontrar informações em formatos previsíveis.

Por esse motivo, é comum definir estruturas específicas para respostas de sucesso e de erro, centralizando sua geração em funções auxiliares.

Na GoList, adotaremos essa abordagem ao longo dos próximos capítulos. Isso permitirá que a API produza respostas JSON consistentes, facilitando tanto a manutenção do código quanto a integração com clientes externos.

Com isso, concluímos os fundamentos do trabalho com JSON em Go. A partir do próximo capítulo, utilizaremos esses conceitos continuamente para receber dados enviados pelos clientes, validar

informações e produzir respostas estruturadas utilizando apenas a biblioteca padrão.

Capítulo 6

Validação

Receber um documento JSON válido não significa, necessariamente, receber uma requisição correta.

O pacote `encoding/json` consegue verificar se o corpo da requisição possui uma sintaxe válida e se os valores recebidos podem ser convertidos para os tipos definidos em uma `struct`. Entretanto, ele não sabe quais dados são obrigatórios para a GoList, quais tamanhos são aceitáveis, quais valores fazem sentido para o domínio ou quais regras precisam ser respeitadas antes de uma operação ser executada.

Essa responsabilidade pertence à aplicação.

O processo de verificar se os dados recebidos são aceitáveis é chamado de **validação**.

Considere a criação de uma nova lista de compras na GoList. O seguinte documento JSON é sintaticamente válido:

```
{
  "name": "",
  "description": "Compras do mês"
}
```

Mesmo assim, uma lista sem nome não atende às regras da aplicação. O JSON pode ser interpretado, mas a operação não deve prosseguir.

Da mesma forma, o documento abaixo também é válido:

```
{
  "name": "Compras do mês",
  "description": "..."
}
```

Ainda assim, talvez o domínio determine que o nome da lista não possa ultrapassar 100 caracteres ou que cada usuário não possa ter duas listas com o mesmo nome. Essas restrições não pertencem ao formato JSON; elas pertencem à lógica da aplicação.

Em uma API, a validação normalmente começa logo depois da desserialização da requisição. Primeiro, o servidor tenta transformar o JSON recebido em um DTO. Depois, a aplicação verifica se os dados presentes nesse DTO atendem às regras esperadas.

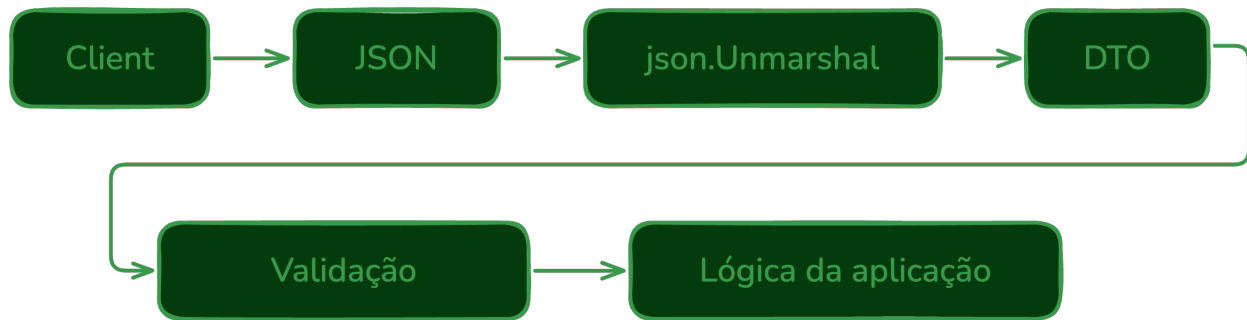


Figura 6.1: Validação dos dados

Separar essas etapas torna o fluxo mais claro. Antes de validar o conteúdo, a aplicação precisa garantir que a requisição pode ser interpretada. Depois disso, pode verificar campos obrigatórios, tamanhos, valores mínimos, formatos e regras específicas da GoList.

Essa distinção ajuda a organizar o código e também melhora as respostas enviadas ao cliente.

Um primeiro grupo de problemas está relacionado à estrutura da requisição. Um JSON malformatado ou um valor incompatível com o tipo esperado impede que a desserialização seja concluída.

Por exemplo:

```
{
  "name": "Mercado",
  "active": "sim"
}
```

Se o campo `active` for do tipo `bool`, a conversão falhará antes mesmo de qualquer validação de domínio ser executada. A aplicação não chegou a receber um DTO válido para analisar.

Um segundo grupo de problemas ocorre quando o JSON é válido, mas os dados não atendem às regras da aplicação.

```
{
  "name": "",
  "active": true
}
```

Nesse caso, a desserialização é realizada com sucesso, mas a aplicação deve rejeitar a requisição porque o nome da lista é obrigatório.

O fluxo apresentado na imagem pode ser entendido em etapas. Primeiro, o cliente envia um documento JSON. Em seguida, a aplicação usa `json.Unmarshal` ou `json.NewDecoder` para transformar esse documento em um DTO. Depois disso, a validação analisa os dados presentes no DTO. Somente se essa etapa for concluída com sucesso a requisição avança para a lógica da aplicação.

Cada etapa responde a uma pergunta diferente.

Na desserialização, a aplicação pergunta: **o corpo da requisição pode ser interpretado?**

Na validação do DTO, pergunta: **os dados mínimos foram enviados e possuem valores aceitáveis?**

Nas regras de domínio, pergunta: **essa operação faz sentido para a GoList neste momento?**

Essa separação evita que regras diferentes fiquem misturadas no mesmo ponto do código. Também ajuda a produzir respostas de erro mais claras. Problemas de formato indicam que o cliente enviou uma requisição incompatível com a API. Já violações de validação indicam que o formato está correto, mas o conteúdo não atende aos requisitos da aplicação.

Ao longo deste capítulo, veremos como implementar validações utilizando apenas a biblioteca padrão da linguagem. Estudaremos como verificar campos obrigatórios, tratar tipos inválidos, aplicar regras de domínio, acumular erros de validação e produzir respostas claras e consistentes. No final, reuniremos esses conceitos nas validações de listas e itens da **GoList**, garantindo que apenas dados válidos avancem para a lógica da aplicação.

6.1 Campos obrigatórios

Uma das validações mais comuns em qualquer API consiste em verificar se os campos obrigatórios foram informados pelo cliente. Um documento JSON pode ser sintaticamente válido, pode ser convertido para um DTO e, ainda assim, não conter os dados mínimos necessários para que a operação faça sentido.

Considere o DTO utilizado para criar um usuário na GoList.

```
type CreateUserRequest struct {  
    Name      string `json:"name"`  
    Email     string `json:"email"`  
    Password  string `json:"password"`  
}
```

Para cadastrar um usuário, a aplicação precisa receber nome, e-mail e senha. Entretanto, nada impede que o cliente envie uma requisição sem o campo name.

```
{  
  "email": "maria@example.com",  
  "password": "123456"  
}
```

Esse JSON é válido. O pacote `encoding/json` conseguirá interpretar o documento e preencher a struct sem retornar erro.

O resultado será semelhante a:

```
CreateUserRequest{  
    Name:      "",  
    Email:     "maria@example.com",  
    Password:  "123456",  
}
```

Observe que o campo Name recebeu seu **valor zero**. Para o tipo string, o valor zero é a string vazia.

Esse comportamento é importante: o pacote `encoding/json` não considera a ausência de um campo um erro. Quando um atributo não aparece no JSON, o campo correspondente na struct simplesmente permanece com o valor padrão do seu tipo.

O mesmo resultado ocorreria se o cliente enviasse o campo com uma string vazia.

```
{
  "name": "",
  "email": "maria@example.com",
  "password": "123456"
}
```

Para a validação de campo obrigatório, tanto a ausência do atributo quanto uma string vazia indicam que o dado necessário não foi fornecido corretamente.

Por esse motivo, a aplicação deve realizar a validação explicitamente.

```
func validateCreateUser(req CreateUserRequest) error {
    if req.Name == "" {
        return errors.New("o nome é obrigatório")
    }

    if req.Email == "" {
        return errors.New("o e-mail é obrigatório")
    }

    if req.Password == "" {
        return errors.New("a senha é obrigatória")
    }

    return nil
}
```

Após desserializar a requisição, o *handler* executa a validação antes de continuar com a lógica da aplicação.


```
var req CreateUserRequest

if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
    http.Error(w, "JSON inválido.", http.StatusBadRequest)
    return
}

if err := validateCreateUser(req); err != nil {
    http.Error(w, err.Error(), http.StatusBadRequest)
    return
}
```

Caso algum campo obrigatório não tenha sido informado corretamente, a API responde com **400 Bad Request**, indicando que a requisição enviada pelo cliente não atende ao contrato esperado.

Essa etapa deve acontecer antes de qualquer regra mais profunda da aplicação. Não faz sentido tentar criar um usuário, consultar banco de dados ou executar outras operações se os dados mínimos da requisição não foram fornecidos.

6.1.1 Espaços em branco

Verificar apenas se uma string é diferente de "" nem sempre é suficiente. Um cliente pode enviar um campo contendo apenas espaços em branco.

Considere a seguinte requisição:

```
{
  "name": "   ",
  "email": "maria@example.com",
  "password": "123456"
}
```

Nesse caso, Name não é exatamente uma string vazia, mas também não representa um nome válido para a aplicação.

Uma forma simples de tratar esse problema é utilizar `strings.TrimSpace`, que remove espaços, quebras de linha e tabulações no início e no fim da string.

```
func validateCreateUser(req CreateUserRequest) error {
    if strings.TrimSpace(req.Name) == "" {
        return errors.New("o nome é obrigatório")
    }

    if strings.TrimSpace(req.Email) == "" {
        return errors.New("o e-mail é obrigatório")
    }

    if strings.TrimSpace(req.Password) == "" {
        return errors.New("a senha é obrigatória")
    }

    return nil
}
```

Assim, valores como " " e "\t\n" também passam a ser rejeitados.

Essa validação é especialmente útil em campos de texto fornecidos pelo usuário, como nomes, títulos, descrições curtas e e-mails.

6.1.2 Campos obrigatórios em outros tipos

Campos obrigatórios não se restringem a `string`. Para tipos numéricos e booleanos, a validação depende do significado do valor zero dentro da aplicação.

Por exemplo, suponha que um item da lista possua uma quantidade.

```
type CreateItemRequest struct {
    Name      string `json:"name"`
    Quantity  int    `json:"quantity"`
}
```

Se a aplicação exige que a quantidade seja informada e que seja maior que zero, a validação pode verificar diretamente essa condição.

```
if req.Quantity <= 0 {
    return errors.New("a quantidade deve ser maior que zero")
}
```

Nesse exemplo, uma quantidade ausente e uma quantidade igual a 0 resultam no mesmo valor final: 0.

Isso acontece porque 0 é o valor zero de `int`. Portanto, com a estrutura acima, a aplicação não consegue distinguir estes dois documentos:

```
{
  "name": "Leite"
```

```
}  
  
{  
  "name": "Leite",  
  "quantity": 0  
}
```

Em ambos os casos, `Quantity` terá o valor `0`.

Em muitas operações isso não é um problema, pois a aplicação simplesmente rejeita quantidades menores ou iguais a zero. Quando for necessário diferenciar um campo ausente de um campo informado com valor zero, uma alternativa é utilizar ponteiros.

```
type CreateItemRequest struct {  
    Name      string `json:"name"`  
    Quantity  *int   `json:"quantity"`  
}
```

Agora, se o campo `quantity` não for informado, `Quantity` será `nil`. Caso o cliente envie o campo, mesmo com valor `0`, o ponteiro apontará para o valor recebido.

A validação pode então verificar primeiro a presença do campo.

```
if req.Quantity == nil {  
    return errors.New("a quantidade é obrigatória")  
}  
  
if *req.Quantity <= 0 {  
    return errors.New("a quantidade deve ser maior que zero")  
}
```

Essa técnica é especialmente útil em operações de atualização parcial (*PATCH*), nas quais é importante diferenciar um campo ausente de um campo explicitamente informado.

O mesmo raciocínio vale para campos booleanos. Um campo `bool` ausente recebe `false`, que também é o valor que seria recebido caso o cliente enviasse o campo explicitamente com `false`.

```
type UpdateUserRequest struct {  
    Active bool `json:"active"`  
}
```

Com essa estrutura, a aplicação não consegue distinguir estes dois documentos:

```
{}  
  
{  
  "active": false  
}
```

Se essa diferença for importante, o campo também pode ser representado como ponteiro.

```
type UpdateUserRequest struct {  
    Active *bool `json:"active"`  
}
```

Assim, `nil` indica que o cliente não enviou o campo, enquanto `true` ou `false` indicam que ele foi informado explicitamente.

6.1.3 Validação próxima ao DTO

Em aplicações pequenas, é comum implementar funções específicas de validação, como `validateCreateUser`. À medida que a aplicação cresce, outra abordagem bastante utilizada é definir a validação como um método do próprio DTO.

```
func (r CreateUserRequest) Validate() error {  
    if strings.TrimSpace(r.Name) == "" {  
        return errors.New("o nome é obrigatório")  
    }  
  
    if strings.TrimSpace(r.Email) == "" {  
        return errors.New("o e-mail é obrigatório")  
    }  
  
    if strings.TrimSpace(r.Password) == "" {  
        return errors.New("a senha é obrigatória")  
    }  
  
    return nil  
}
```

Com isso, o *handler* fica mais direto.

```
var req CreateUserRequest  
  
if err := json.NewDecoder(r.Body).Decode(&req); err != nil {  
    http.Error(w, "JSON inválido.", http.StatusBadRequest)  
    return  
}  
  
if err := req.Validate(); err != nil {  
    http.Error(w, err.Error(), http.StatusBadRequest)  
    return  
}
```

Essa organização mantém as regras de validação próximas da estrutura que elas verificam, facilitando a manutenção e a reutilização do código.

Por enquanto, os exemplos retornam apenas o primeiro erro encontrado. Essa abordagem é suficiente para entender o fluxo básico da validação. Mais adiante neste capítulo, veremos como acumular vários erros e devolver uma resposta mais útil para o cliente.

Neste livro, adotaremos validações simples utilizando apenas a biblioteca padrão da linguagem. O objetivo é compreender os princípios envolvidos antes de introduzir bibliotecas externas de validação. Assim, fica claro o que está sendo verificado, em qual momento da requisição a validação acontece e por que a API deve rejeitar dados incompletos antes de executar a operação solicitada.

6.2 Tipos inválidos

Antes de verificar campos obrigatórios ou regras de domínio, a aplicação precisa garantir que o JSON recebido pode ser convertido para o DTO esperado. Essa etapa faz parte da validação estrutural da requisição.

Quando um campo possui um tipo incompatível com a struct de destino, a conversão falha e a aplicação não deve continuar o processamento.

Considere o seguinte DTO.

```
type CreateItemRequest struct {
    Name      string `json:"name"`
    Quantity  int    `json:"quantity"`
    Price     float64 `json:"price"`
}
```

Um cliente pode enviar a seguinte requisição.

```
{
  "name": "Leite",
  "quantity": "dez",
  "price": 7.50
}
```

Esse JSON é sintaticamente válido. Entretanto, o valor informado para *quantity* é uma *string*, enquanto a aplicação espera um número inteiro.

Ao executar a desserialização, o pacote `encoding/json` retornará um erro.

```
var req CreateItemRequest

err := json.NewDecoder(r.Body).Decode(&req)
if err != nil {
    fmt.Println(err)
}
```

A mensagem produzida será semelhante a:

```
json: cannot unmarshal string into Go struct field CreateItemRequest.quantity of type int
```

Nesse caso, o erro não está relacionado às regras de negócio da aplicação. A própria estrutura da requisição é incompatível com o contrato definido pela API.

O mesmo ocorre para outros tipos incompatíveis.

```
{
  "name": true,
  "quantity": 10,
  "price": 7.50
}
```

Resultado:

```
json: cannot unmarshal bool into Go struct field CreateItemRequest.name of type string
```

Ou ainda:

```
{
  "name": "Leite",
  "quantity": 10,
  "price": "sete reais"
}
```

Resultado:

```
json: cannot unmarshal string into Go struct field CreateItemRequest.price of type float64
```

Esses erros devem ser tratados imediatamente, pois a aplicação não pode continuar processando uma requisição cuja estrutura não pôde ser interpretada corretamente. Se a conversão falhou, o DTO não é confiável para as próximas etapas.

Uma implementação típica é a seguinte.

```
var req CreateItemRequest

if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
    http.Error(w, "JSON inválido.", http.StatusBadRequest)
    return
}
```

Observe que não é necessário analisar detalhadamente todas as mensagens produzidas por encoding/json. Em geral, basta informar ao cliente que a requisição possui um formato inválido e interromper seu processamento.

Em APIs maiores, pode ser útil registrar o erro original nos logs para depuração, mas a resposta enviada ao cliente costuma permanecer simples e padronizada.

```
if err := json.NewDecoder(r.Body).Decode(&req); err != nil {  
    log.Printf("erro ao decodificar JSON: %v", err)  
    http.Error(w, "JSON inválido.", http.StatusBadRequest)  
    return  
}
```

Assim, a aplicação preserva detalhes técnicos para investigação interna sem expor mensagens longas ou inconsistentes para quem consome a API.

6.2.1 Campos desconhecidos

Por padrão, o pacote `encoding/json` ignora atributos presentes no JSON que não existam na estrutura de destino.

Considere o seguinte documento.

```
{  
  "name": "Leite",  
  "quantity": 2,  
  "color": "branco"  
}
```

Como `color` não existe em `CreateItemRequest`, ele será simplesmente descartado.

Em algumas aplicações esse comportamento é aceitável. Em outras, pode ser interessante rejeitar a requisição para evitar erros de digitação ou uso incorreto da API.

Isso pode ser feito utilizando o método `DisallowUnknownFields`.

```
decoder := json.NewDecoder(r.Body)  
decoder.DisallowUnknownFields()  
  
var req CreateItemRequest  
  
if err := decoder.Decode(&req); err != nil {  
    http.Error(w, "JSON inválido.", http.StatusBadRequest)  
    return  
}
```

Agora, se o cliente enviar um atributo desconhecido, a conversão falhará.

```
json: unknown field "color"
```

Essa validação torna o contrato da API mais rigoroso e ajuda os clientes a identificar rapidamente erros na construção das requisições.

Se a API decidir aceitar campos extras, essa configuração não é necessária. O importante é que esse comportamento seja uma escolha consciente do projeto.

6.2.2 Erros de sintaxe

Também é responsabilidade do `encoding/json` detectar documentos malformados.

Por exemplo:

```
{  
  "name": "Leite",  
  "quantity": 2,  
}
```

Nesse caso, a desserialização produzirá um erro semelhante a:

```
unexpected end of JSON input
```

Da mesma forma, vírgulas extras, aspas não fechadas ou outros erros de sintaxe impedirão a interpretação do documento.

Esses problemas devem resultar em uma resposta **400 Bad Request**, pois indicam que o cliente enviou uma requisição inválida.

6.2.3 Corpo vazio

Outro caso comum é receber uma requisição sem corpo.

```
POST /items HTTP/1.1
```

```
Content-Type: application/json
```

Ao tentar decodificar um corpo vazio, o decoder retornará um erro semelhante a:

```
EOF
```

Para o cliente, esse caso também deve ser tratado como uma requisição inválida, pois a API esperava receber um documento JSON.

```
if err := json.NewDecoder(r.Body).Decode(&req); err != nil {  
    http.Error(w, "JSON inválido.", http.StatusBadRequest)  
    return  
}
```

Mesmo que a mensagem interna seja EOF, a resposta pode continuar sendo JSON inválido..

6.2.4 Múltiplos documentos JSON

Em uma API, normalmente esperamos receber um único documento JSON no corpo da requisição.

O exemplo abaixo contém dois documentos em sequência:

```
{"name": "Leite"}  
{"name": "Arroz"}
```

Uma chamada simples a `Decode` pode interpretar o primeiro documento e deixar o restante no corpo. Em aplicações mais rigorosas, é possível verificar se existe outro documento depois da primeira decodificação.


```
decoder := json.NewDecoder(r.Body)

if err := decoder.Decode(&req); err != nil {
    http.Error(w, "JSON inválido.", http.StatusBadRequest)
    return
}

var extra any

if err := decoder.Decode(&extra); err != io.EOF {
    http.Error(w, "JSON inválido.", http.StatusBadRequest)
    return
}
```

Nesse caso, a segunda chamada a `Decode` deve encontrar o fim do corpo da requisição. Se encontrar outro documento JSON, a aplicação rejeita a entrada.

Esse cuidado nem sempre é necessário em exemplos iniciais, mas ajuda a reforçar a ideia de contrato: a requisição deve conter exatamente o formato que a API espera.

6.2.5 Validação estrutural versus validação de domínio

É importante distinguir a validação realizada pelo pacote `encoding/json` daquela implementada pela própria aplicação.

A validação estrutural responde perguntas como:

- o JSON possui uma sintaxe válida?
- os tipos dos atributos são compatíveis?
- existem campos desconhecidos?
- o corpo contém um documento JSON esperado?

Já a validação de domínio responde perguntas diferentes:

- o nome foi informado?
- a quantidade é maior que zero?
- o e-mail possui um formato válido?
- o usuário já está cadastrado?

Essa separação torna o código mais organizado e facilita a manutenção da aplicação. Primeiro, garante-se que a requisição pode ser interpretada corretamente. Somente depois são avaliadas as regras específicas do domínio da GoList.

6.3 Regras de domínio

Após verificar que o JSON possui uma estrutura válida e que os campos obrigatórios foram informados corretamente, a aplicação ainda precisa responder a uma pergunta mais importante: **essa operação faz sentido para o domínio do problema?**

Esse conjunto de verificações é conhecido como **validação de domínio**.

Enquanto as validações anteriores garantem que a requisição pode ser interpretada e que os dados mínimos foram enviados, as regras de domínio garantem que aqueles dados representam uma operação válida para a aplicação.

Considere a criação de um item na GoList.

```
type CreateItemRequest struct {  
    Name      string `json:"name"`  
    Quantity  int    `json:"quantity"`  
}
```

O seguinte JSON é perfeitamente válido.

```
{  
  "name": "Leite",  
  "quantity": 0  
}
```

Todos os tipos estão corretos e o campo `quantity` foi enviado. Entretanto, se a GoList determinar que a quantidade de um item deve ser maior que zero, essa requisição deve ser rejeitada.

Uma possível validação seria:

```
func (r CreateItemRequest) Validate() error {  
    if strings.TrimSpace(r.Name) == "" {  
        return errors.New("o nome é obrigatório")  
    }  
  
    if r.Quantity <= 0 {  
        return errors.New("a quantidade deve ser maior que zero")  
    }  
  
    return nil  
}
```

Observe que essa regra não está relacionada ao JSON nem à linguagem Go. Ela existe porque faz parte do funcionamento da GoList.

Esse é um ponto importante: o valor `0` é perfeitamente aceitável para um `int` em Go, mas pode não ser aceitável para a operação de criar um item na aplicação.

6.3.1 Regras dependem da aplicação

As regras de domínio variam conforme o problema que está sendo resolvido.

Na GoList, por exemplo, poderíamos definir regras como:

- o nome da lista deve possuir entre 3 e 100 caracteres;
- um item não pode possuir quantidade negativa;
- o preço deve ser maior que zero;

- não pode existir duas listas com o mesmo nome para o mesmo usuário;
- um usuário não pode ser adicionado duas vezes à mesma lista.

Nenhuma dessas regras pode ser validada pelo pacote `encoding/json`, pois elas fazem parte da lógica da aplicação.

Algumas dessas regras são simples e podem ser verificadas apenas com os dados recebidos na requisição. Outras dependem do estado atual da aplicação, como informações armazenadas no banco de dados.

6.3.2 Regras simples

Algumas regras podem ser verificadas apenas analisando os dados recebidos no DTO.

Por exemplo, limitar o tamanho do nome.

```
if len(r.Name) > 100 {  
    return errors.New("o nome deve possuir no máximo 100 caracteres")  
}
```

Ou verificar se o preço é positivo.

```
if r.Price <= 0 {  
    return errors.New("o preço deve ser maior que zero")  
}
```

Essas validações normalmente são rápidas e independem de qualquer recurso externo. Por isso, é comum mantê-las próximas ao DTO.

```
func (r CreateItemRequest) Validate() error {  
    if strings.TrimSpace(r.Name) == "" {  
        return errors.New("o nome é obrigatório")  
    }  
  
    if r.Quantity <= 0 {  
        return errors.New("a quantidade deve ser maior que zero")  
    }  
  
    return nil  
}
```

Esse tipo de validação ainda está relacionado à entrada da requisição. Ela verifica se os dados enviados pelo cliente são aceitáveis antes de passar a operação para a camada de aplicação.

6.3.3 Validações que dependem de outros dados

Nem todas as regras podem ser verificadas apenas observando a requisição.

Imagine o cadastro de um novo usuário.

```
{
  "name": "Maria",
  "email": "maria@example.com"
}
```

Mesmo que todos os campos estejam corretos, a aplicação talvez exija que o endereço de e-mail seja único.

Nesse caso, é necessário consultar o banco de dados antes de concluir a operação.

```
exists, err := repository.EmailExists(req.Email)
if err != nil {
    return err
}

if exists {
    return errors.New("e-mail já cadastrado")
}
```

Essa validação depende do estado atual da aplicação e, por isso, normalmente é realizada na camada de serviço, onde existe acesso aos repositórios.

O mesmo raciocínio vale para listas de compras. Uma requisição para criar uma lista pode estar bem formada, mas ainda assim violar uma regra da GoList.

```
{
  "name": "Mercado"
}
```

Se o usuário já possui uma lista chamada **"Mercado"**, a aplicação pode rejeitar a operação.

```
exists, err := repository.ExistsByName(userID, req.Name)
if err != nil {
    return err
}

if exists {
    return errors.New("já existe uma lista com esse nome")
}
```

Essa regra não deve ficar no DTO, pois o DTO não deveria consultar banco de dados nem conhecer detalhes de persistência.

6.3.4 Onde validar?

Uma dúvida bastante comum é onde essas regras devem ser implementadas.

Uma boa prática consiste em dividir as responsabilidades:

- o **DTO** valida aspectos relacionados à entrada dos dados, como campos obrigatórios, tamanhos mínimos e formatos simples;
- a **camada de serviço** valida regras que dependem do domínio da aplicação ou de informações armazenadas em outros sistemas, como banco de dados.

Essa divisão evita que os DTOs assumam responsabilidades que não lhes pertencem e mantém as regras de negócio concentradas na camada responsável por executá-las.

Em outras palavras, o DTO deve responder se a entrada parece válida. O serviço deve responder se a operação pode acontecer.

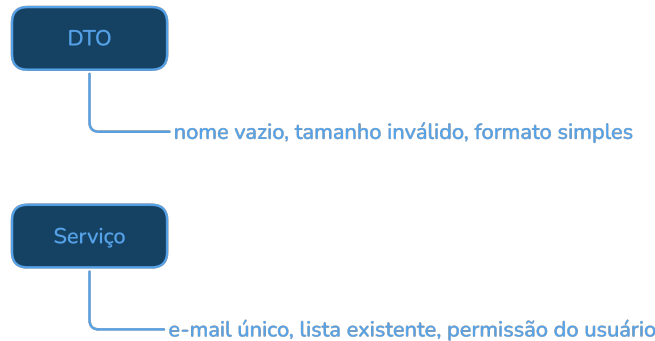
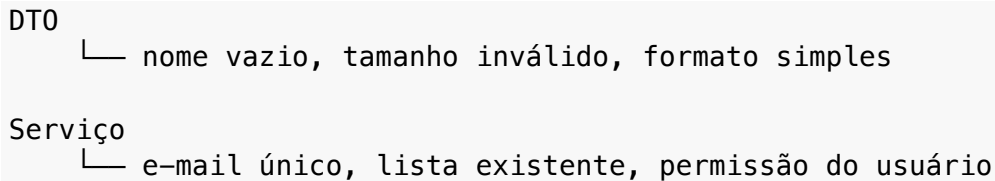


Figura 6.2: DTO e Serviço



6.3.5 Validação em camadas

O fluxo completo de uma requisição pode ser entendido em camadas:

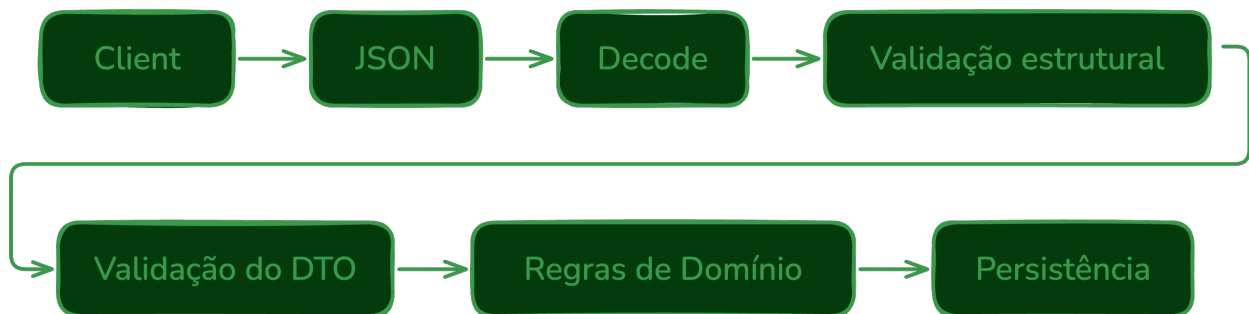


Figura 6.3: Validação em camadas

Cada etapa elimina um conjunto específico de problemas antes que a requisição avance para a próxima fase.

Essa organização torna a aplicação mais robusta e facilita a manutenção do código, pois cada camada possui uma responsabilidade claramente definida.

Na GoList, adotaremos essa estratégia ao longo do desenvolvimento da API. As validações relacionadas à entrada dos dados permanecerão próximas aos DTOs, enquanto as regras que dependem do estado da aplicação serão implementadas na camada de serviços.

Essa separação será especialmente importante quando a API passar a lidar com autenticação, permissões e listas compartilhadas, pois muitas decisões dependerão de quem está fazendo a requisição e de quais recursos esse usuário pode acessar.

6.4 Erros de validação

Quando uma requisição não atende às regras da aplicação, a API deve informar claramente ao cliente quais problemas foram encontrados. Essa comunicação é feita por meio dos **erros de validação**.

O objetivo não é apenas indicar que a requisição falhou. Uma boa resposta de validação fornece informações suficientes para que o cliente consiga corrigir os dados enviados.

Por exemplo, considere a seguinte requisição para criação de um usuário.

```
{
  "name": "",
  "email": "maria@example.com",
  "password": "123"
}
```

Suponha que a aplicação exija:

- nome obrigatório;
- senha com pelo menos oito caracteres.

Em vez de responder apenas com uma mensagem genérica:

```
{
  "error": "Dados inválidos."
}
```

é preferível informar quais validações falharam.

```
{
  "errors": [
    "o nome é obrigatório",
    "a senha deve possuir pelo menos 8 caracteres"
  ]
}
```

Dessa forma, o cliente pode corrigir todos os problemas conhecidos de uma única vez, sem precisar realizar múltiplas tentativas.

6.4.1 Acumulando erros

Uma estratégia simples consiste em interromper a validação no primeiro erro encontrado.

```
func (r CreateUserRequest) Validate() error {
    if strings.TrimSpace(r.Name) == "" {
        return errors.New("o nome é obrigatório")
    }

    if len(r.Password) < 8 {
        return errors.New("a senha deve possuir pelo menos 8 caracteres")
    }

    return nil
}
```

Embora essa abordagem seja simples, ela apresenta uma limitação: o cliente descobre apenas um problema por requisição.

Uma alternativa melhor é acumular todos os erros encontrados.

```
func (r CreateUserRequest) Validate() []string {
    var errs []string

    if strings.TrimSpace(r.Name) == "" {
        errs = append(errs, "o nome é obrigatório")
    }

    if strings.TrimSpace(r.Email) == "" {
        errs = append(errs, "o e-mail é obrigatório")
    }

    if len(r.Password) < 8 {
        errs = append(errs, "a senha deve possuir pelo menos 8 caracteres")
    }

    return errs
}
```

O *handler* passa então a verificar se a lista está vazia.

```
errs := req.Validate()

if len(errs) > 0 {
    writeValidationErrors(w, errs)
    return
}
```

Essa abordagem melhora significativamente a experiência de quem consome a API.

Também evita uma sequência incômoda de tentativa e erro, em que o cliente corrige um campo apenas para descobrir outro problema na requisição seguinte.

6.4.2 Padronizando a resposta

Assim como vimos no capítulo anterior, é recomendável que todas as respostas da API possuam um formato consistente.

Uma estrutura simples para representar erros de validação pode conter uma lista de mensagens.

```
type ValidationErrorResponse struct {  
    Errors []string `json:"errors"`  
}
```

Uma função auxiliar pode centralizar o envio da resposta.

```
func writeValidationErrors(w http.ResponseWriter, errs []string) {  
    w.Header().Set("Content-Type", "application/json")  
    w.WriteHeader(http.StatusBadRequest)  
  
    if err := json.NewEncoder(w).Encode(ValidationErrorResponse{  
        Errors: errs,  
    }); err != nil {  
        return  
    }  
}
```

A resposta enviada ao cliente será semelhante à seguinte.

```
{  
  "errors": [  
    "o nome é obrigatório",  
    "o e-mail é obrigatório",  
    "a senha deve possuir pelo menos 8 caracteres"  
  ]  
}
```

6.4.3 Associando erros aos campos

Em APIs maiores, pode ser interessante indicar a qual campo cada erro está relacionado.

Uma forma simples é utilizar um mapa.

```
type ValidationErrorResponse struct {  
    Errors map[string]string `json:"errors"`  
}
```

O resultado passa a ser:

```
{  
  "errors": {  
    "name": "o nome é obrigatório",  
  }  
}
```



```
"email": "o e-mail é obrigatório",
"password": "a senha deve possuir pelo menos 8 caracteres"
}
```

Esse formato facilita a integração com aplicações web e móveis, que podem destacar automaticamente os campos com erro na interface do usuário.

Uma limitação desse formato é que cada campo possui apenas uma mensagem. Se um mesmo campo puder ter mais de um erro, podemos usar uma lista por campo.

```
type ValidationErrorResponse struct {
    Errors map[string][]string `json:"errors"`
}
```

O resultado seria:

```
{
  "errors": {
    "password": [
      "a senha é obrigatória",
      "a senha deve possuir pelo menos 8 caracteres"
    ]
  }
}
```

Outra alternativa é representar cada erro como um objeto.

```
type FieldError struct {
    Field   string `json:"field"`
    Message string `json:"message"`
}

type ValidationErrorResponse struct {
    Errors []FieldError `json:"errors"`
}
```

Esse formato é mais verboso, mas permite adicionar outras informações no futuro, como códigos de erro.

```
{
  "errors": [
    {
      "field": "name",
      "message": "o nome é obrigatório"
    },
    {
      "field": "password",
      "message": "a senha deve possuir pelo menos 8 caracteres"
    }
  ]
}
```

```
}  
]  
}
```

Não existe um único formato obrigatório. O mais importante é escolher um padrão e utilizá-lo de forma consistente em toda a API.

6.4.4 Código de status

Erros de validação representam problemas nos dados enviados pelo cliente. Por esse motivo, a resposta normalmente utiliza o código HTTP **400 Bad Request**.

É importante distinguir esse caso de outros tipos de erro:

Situação	Status HTTP
JSON inválido	400 Bad Request
Campo obrigatório ausente	400 Bad Request
Regra de validação violada	400 Bad Request
Regra de domínio violada	400 Bad Request
Recurso não encontrado	404 Not Found
Erro interno da aplicação	500 Internal Server Error

Embora todos esses casos representem falhas, suas causas são diferentes e devem ser comunicadas adequadamente ao cliente.

Em alguns projetos, violações de regras de domínio podem ser representadas por outros códigos, como **409 Conflict** quando há conflito com o estado atual do recurso. Neste capítulo, manteremos o uso de **400 Bad Request** para simplificar o fluxo e focar na validação dos dados recebidos.

6.4.5 Boas práticas

Ao implementar erros de validação, algumas recomendações tornam a API mais consistente e fácil de utilizar:

- retorne mensagens claras e objetivas;
- sempre utilize o mesmo formato de resposta;
- informe todos os erros encontrados, sempre que possível;
- associe os erros aos respectivos campos quando isso facilitar o consumo da API;
- evite expor detalhes internos da aplicação nas mensagens;
- utilize o código **400 Bad Request** para indicar problemas nos dados enviados pelo cliente.

Na GoList, adotaremos respostas padronizadas para todas as validações realizadas pela aplicação. Isso tornará a API mais previsível e facilitará sua integração com diferentes clientes, como aplicações web, dispositivos móveis e outros serviços HTTP.

Na próxima seção, aplicaremos esse padrão às validações de listas e itens, reunindo campos obrigatórios, regras simples e respostas de erro em um fluxo único.

6.5 Validação de itens e listas

Ao longo deste capítulo, estudamos técnicas de validação de forma isolada: verificamos campos obrigatórios, tratamos tipos inválidos, aplicamos regras de domínio e padronizamos respostas de erro. Na prática, porém, essas verificações são realizadas em conjunto sempre que uma operação é executada.

Na GoList, praticamente todas as requisições passam por um processo semelhante de validação antes que qualquer alteração seja realizada na aplicação.

Considere o cadastro de uma nova lista de compras.

```
type CreateListRequest struct {  
    Name      string `json:"name"`  
    Description string `json:"description,omitempty"`  
}
```

Uma possível implementação da validação do DTO é:

```
func (r CreateListRequest) Validate() []string {  
    var errs []string  
  
    name := strings.TrimSpace(r.Name)  
    description := strings.TrimSpace(r.Description)  
  
    if name == "" {  
        errs = append(errs, "o nome da lista é obrigatório")  
    }  
  
    if len(name) > 100 {  
        errs = append(errs, "o nome da lista deve possuir no máximo 100 caracteres")  
    }  
  
    if len(description) > 500 {  
        errs = append(errs, "a descrição deve possuir no máximo 500 caracteres")  
    }  
  
    return errs  
}
```

Essa função trata apenas regras que podem ser avaliadas com os dados recebidos no corpo da requisição.

Após desserializar o JSON, o *handler* executa a validação.

```
var req CreateListRequest

if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
    http.Error(w, "JSON inválido.", http.StatusBadRequest)
    return
}

if errs := req.Validate(); len(errs) > 0 {
    writeValidationErrors(w, errs)
    return
}
```

Se houver erros, a requisição é interrompida e a resposta padronizada de validação é enviada ao cliente.

O mesmo princípio pode ser aplicado ao cadastro de itens. Na GoList, um item será criado dentro de uma lista, por meio de uma rota como:

POST /lists/{id}/items

O identificador da lista vem da URL. O corpo JSON contém os dados do item.

```
type CreateItemRequest struct {
    Name      string `json:"name"`
    Quantity  int    `json:"quantity"`
    Price     float64 `json:"price"`
}
```

Uma possível implementação seria:

```
func (r CreateItemRequest) Validate() []string {
    var errs []string

    name := strings.TrimSpace(r.Name)

    if name == "" {
        errs = append(errs, "o nome do item é obrigatório")
    }

    if r.Quantity <= 0 {
        errs = append(errs, "a quantidade deve ser maior que zero")
    }

    if r.Price <= 0 {
        errs = append(errs, "o preço deve ser maior que zero")
    }

    return errs
}
```

Essas validações garantem que a aplicação receba apenas dados mínimos consistentes antes de iniciar qualquer processamento.

6.5.1 Validando parâmetros da rota

Nem todos os dados de entrada vêm do corpo JSON. Em algumas rotas, parte da informação vem do caminho.

Na criação de um item, por exemplo, o ID da lista vem de {id}.

POST /lists/{id}/items

Esse valor também precisa ser validado.

```
listID, err := strconv.Atoi(r.PathValue("id"))
if err != nil || listID <= 0 {
    http.Error(w, "ID da lista inválido.", http.StatusBadRequest)
    return
}
```

Essa validação acontece antes da regra de negócio. Primeiro, a aplicação garante que o parâmetro recebido na URL pode ser interpretado como um identificador válido. Depois, a camada de serviço pode verificar se a lista existe e se o usuário tem permissão para acessá-la.

Entretanto, algumas regras não podem ser verificadas apenas analisando a requisição. Considere a criação de uma nova lista chamada “Mercado”.

```
{
  "name": "Mercado"
```

```
}
```

Mesmo que todos os campos sejam válidos, a aplicação pode determinar que um usuário não pode possuir duas listas com o mesmo nome. Para verificar essa condição, é necessário consultar o banco de dados.

```
exists, err := repository.ExistsByName(userID, req.Name)
if err != nil {
    return err
}

if exists {
    return errors.New("já existe uma lista com esse nome")
}
```

Da mesma forma, ao adicionar um item, pode ser necessário verificar se a lista informada realmente existe.

```
exists, err := repository.ListExists(listID)
if err != nil {
    return err
}

if !exists {
    return errors.New("lista não encontrada")
}
```

Observe que essas validações não pertencem ao DTO, pois dependem do estado atual da aplicação. Elas fazem parte da lógica de negócio e normalmente são executadas na camada de serviços.

6.5.2 Fluxo no handler

Juntando as etapas, um *handler* de criação de lista pode seguir este formato:

```
func createList(w http.ResponseWriter, r *http.Request) {
    var req CreateListRequest

    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "JSON inválido.", http.StatusBadRequest)
        return
    }

    if errs := req.Validate(); len(errs) > 0 {
        writeValidationErrors(w, errs)
        return
    }

    // Chamar a camada de serviço para aplicar regras de domínio
    // e persistir a nova lista.
}
```

Para a criação de item, o fluxo inclui também a validação do parâmetro de rota.

```
func createItem(w http.ResponseWriter, r *http.Request) {
    listID, err := strconv.Atoi(r.PathValue("id"))
    if err != nil || listID <= 0 {
        http.Error(w, "ID da lista inválido.", http.StatusBadRequest)
        return
    }

    var req CreateItemRequest

    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "JSON inválido.", http.StatusBadRequest)
        return
    }

    if errs := req.Validate(); len(errs) > 0 {
        writeValidationErrors(w, errs)
        return
    }

    // Chamar a camada de serviço usando listID e req.
}
```

Esses exemplos mantêm o *handler* com uma responsabilidade clara: interpretar a entrada HTTP, validar o formato dos dados recebidos e encaminhar a operação para a camada adequada.

6.5.3 Fluxo completo

O fluxo completo de uma operação na GoList pode ser representado da seguinte forma.

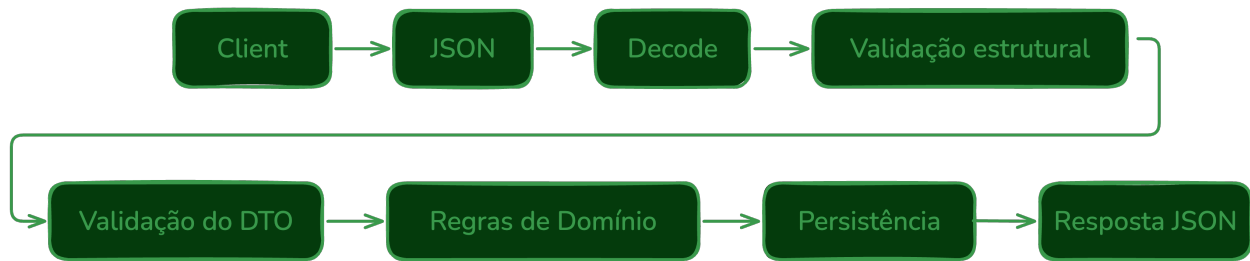


Figura 6.4: Fluxo completo de validação

Cada etapa possui uma responsabilidade específica. Primeiro, verifica-se se a requisição pode ser interpretada corretamente. Em seguida, validam-se os dados informados pelo cliente. Depois, são aplicadas as regras do domínio da aplicação e, somente então, a operação é persistida.

Essa organização evita que dados inconsistentes avancem pelo sistema, reduzindo a ocorrência de erros e tornando a API mais previsível e confiável.

Com isso, concluímos o capítulo sobre validação. A partir do próximo capítulo, utilizaremos esses conceitos continuamente. Sempre que uma nova operação for adicionada à GoList, a aplicação seguirá o mesmo princípio: interpretar a requisição, validar os dados de entrada, aplicar regras de domínio e só então executar a lógica principal.

Capítulo 7

Middleware

Nos capítulos anteriores, vimos como receber requisições HTTP, registrar rotas, interpretar documentos JSON e validar os dados enviados pelo cliente. Com esses recursos, já é possível construir *handlers* capazes de processar operações importantes da API.

Entretanto, conforme a aplicação cresce, algumas responsabilidades começam a aparecer repetidamente em várias rotas. Elas não fazem parte do caso de uso principal, mas precisam ser executadas de forma consistente.

Uma API pode precisar registrar informações sobre cada requisição recebida, recuperar-se de erros inesperados, adicionar cabeçalhos de CORS, associar um identificador único a cada requisição ou verificar se o usuário está autenticado antes de permitir o acesso a determinados recursos.

Essas tarefas são importantes, mas normalmente não pertencem à regra principal de cada *handler*. Elas atravessam várias partes da aplicação e, por isso, recebem o nome de responsabilidades transversais.

Considere a criação de uma lista de compras na GoList. O *handler* responsável por `POST /lists` deveria se concentrar em receber os dados, validar a entrada, acionar a lógica da aplicação e produzir a resposta adequada. Se esse mesmo *handler* também precisar cuidar de log, autenticação, recuperação de *panics* e configuração de cabeçalhos HTTP, sua responsabilidade se torna ampla demais.

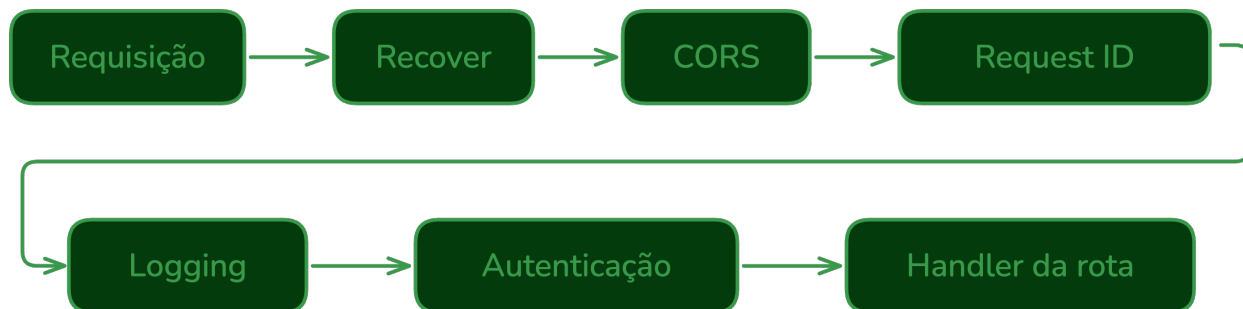


Figura 7.1: Middleware

O **middleware** resolve esse problema criando camadas intermediárias em torno do *handler* final. Em vez de colocar essas responsabilidades dentro de cada rota, envolvemos os *handlers* com

funções reutilizáveis que executam código antes ou depois do processamento principal.

Essa ideia combina muito bem com o modelo da biblioteca padrão de Go. Como um *handler* HTTP é representado pela interface `http.Handler`, um *middleware* pode receber um *handler*, retornar outro *handler* e, nesse processo, adicionar comportamento à requisição.

Com isso, a aplicação passa a ter uma cadeia de processamento mais organizada. Cada *middleware* cuida de uma responsabilidade específica, e o *handler* final permanece focado no caso de uso que aquela rota representa. A ordem dessa cadeia também importa: alguns *middlewares* devem envolver toda a execução, enquanto outros fazem sentido apenas em rotas protegidas.

Ao longo deste capítulo, estudaremos como *middlewares* funcionam e como eles podem ser combinados em cadeia. Veremos *middlewares* para *logging*, recuperação de falhas, CORS, identificação de requisições e autenticação. Todos os exemplos serão desenvolvidos com a biblioteca padrão de Go e aplicados gradualmente à API da **GoList**, preparando a base para uma aplicação mais organizada, observável e segura.

7.1 Conceito

Um **middleware** é uma função que envolve um *handler* HTTP para adicionar algum comportamento ao processamento de uma requisição.

Em vez de colocar toda a lógica dentro do *handler* da rota, criamos uma camada intermediária que pode executar código antes dele, depois dele ou encerrar a requisição antes que ele seja chamado.

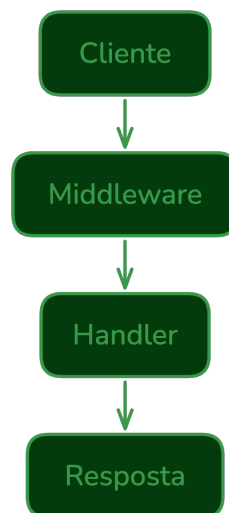


Figura 7.2: Middleware envolve um handler HTTP

Essa camada pode ser usada para tarefas como registrar logs, medir o tempo de execução, adicionar cabeçalhos, verificar autenticação ou recuperar a aplicação de erros inesperados.

O ponto principal é que o *middleware* não substitui o *handler*. Ele o envolve.

7.1.1 Handlers em Go

Para entender middlewares em Go, precisamos lembrar como um *handler* HTTP é representado pela biblioteca padrão.

O pacote `net/http` define a interface `http.Handler`.

```
type Handler interface {  
    ServeHTTP(ResponseWriter, *Request)  
}
```

Qualquer tipo que implemente o método `ServeHTTP` pode processar uma requisição HTTP.

Na prática, muitas vezes usamos funções comuns com a assinatura:

```
func(w http.ResponseWriter, r *http.Request)
```

Essas funções podem ser convertidas para `http.Handler` por meio de `http.HandlerFunc`.

```
func hello(w http.ResponseWriter, r *http.Request) {  
    w.Write([]byte("Olá, mundo!"))  
}  
  
handler := http.HandlerFunc(hello)
```

Isso funciona porque `http.HandlerFunc` é um tipo adaptador: ele permite tratar uma função comum como um valor que implementa `http.Handler`.

7.1.2 A forma de um middleware

Um middleware não é uma interface especial do pacote `net/http`. Em Go, ele costuma ser escrito como uma função que recebe um `http.Handler` e retorna outro `http.Handler`.

```
func middleware(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        // Código executado antes do handler final.  
  
        next.ServeHTTP(w, r)  
  
        // Código executado depois do handler final.  
    })  
}
```

O parâmetro `next` representa o próximo *handler* da cadeia. Ele pode ser o *handler* final da rota ou outro middleware que ainda será executado.

A chamada mais importante é:

```
next.ServeHTTP(w, r)
```

É essa chamada que permite que a requisição continue avançando pelo fluxo da aplicação.

7.1.3 Um exemplo simples

Considere um middleware que escreve uma mensagem no terminal antes de executar o *handler* da rota.

```
func logging(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        fmt.Println("requisição recebida:", r.Method, r.URL.Path)  
  
        next.ServeHTTP(w, r)  
    })  
}
```

Agora, podemos envolver um *handler* comum com esse middleware.

```
func hello(w http.ResponseWriter, r *http.Request) {  
    w.Write([]byte("Olá, mundo!"))  
}  
  
func main() {  
    mux := http.NewServeMux()  
  
    mux.Handle("GET /hello", logging(http.HandlerFunc(hello)))  
  
    http.ListenAndServe(":8080", mux)  
}
```

Quando uma requisição GET /hello for recebida, o fluxo será:

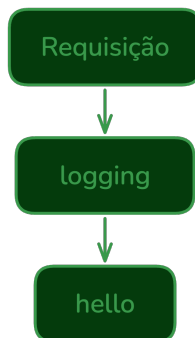


Figura 7.3: Middleware hello

O middleware executa primeiro, registra a requisição e depois chama `next.ServeHTTP`, permitindo que o *handler* `hello` produza a resposta.

7.1.4 Executando código depois do handler

Um middleware também pode executar código após o *handler* final.

```
func logging(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        fmt.Println("início da requisição")  
  
        next.ServeHTTP(w, r)  
  
        fmt.Println("fim da requisição")  
    })  
}
```

Nesse caso, a ordem de execução será:

```
início da requisição  
handler da rota  
fim da requisição
```

Essa característica permite medir tempo de execução, registrar informações depois que o *handler* termina ou executar alguma limpeza ao final do processamento.

7.1.5 Interrompendo a requisição

Nem todo middleware precisa chamar `next.ServeHTTP`.

Em algumas situações, ele pode decidir encerrar a requisição imediatamente. Isso acontece, por exemplo, em um middleware de autenticação.

```
func requireAuth(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        token := r.Header.Get("Authorization")  
  
        if token == "" {  
            http.Error(w, "não autorizado", http.StatusUnauthorized)  
            return  
        }  
  
        next.ServeHTTP(w, r)  
    })  
}
```

Se o cabeçalho `Authorization` não estiver presente, o middleware retorna uma resposta **401 Unauthorized** e interrompe o processamento. O *handler* final não é executado.

Caso o cabeçalho exista, a requisição continua normalmente.

Essa capacidade de decidir se a requisição continua ou não torna middlewares muito úteis para responsabilidades transversais da aplicação.

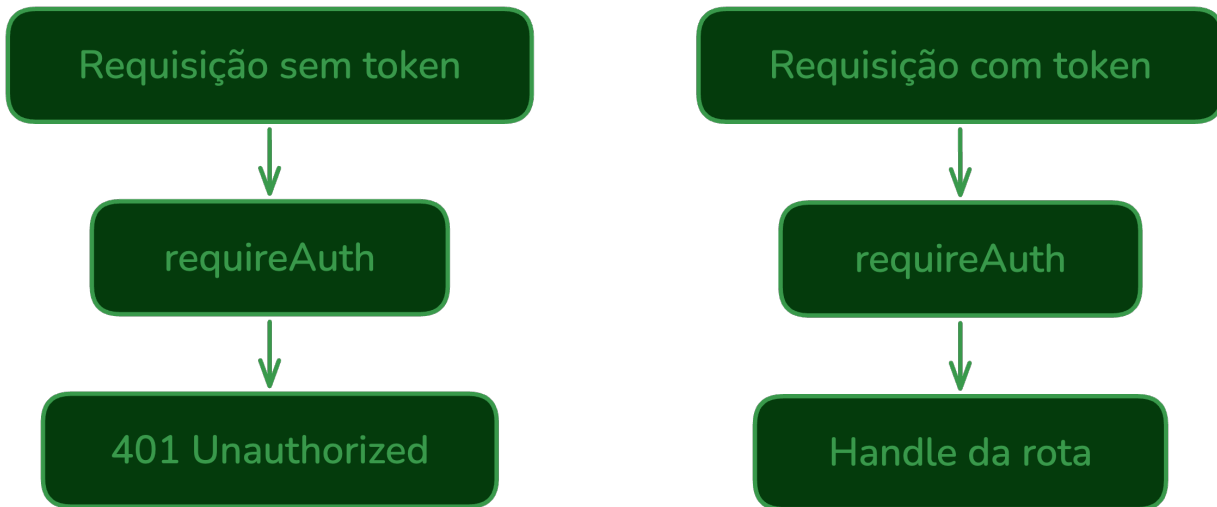


Figura 7.4: Middleware de autorização

7.1.6 Responsabilidades transversais

Middlewares são especialmente adequados para responsabilidades que aparecem em muitas rotas, mas que não pertencem diretamente ao caso de uso de cada uma delas.

Por exemplo:

- registrar informações sobre a requisição;
- recuperar a aplicação de *panics*;
- adicionar cabeçalhos comuns;
- identificar a requisição com um ID único;
- verificar autenticação;
- bloquear métodos ou origens não permitidas.

Essas responsabilidades são chamadas de transversais porque atravessam várias partes da aplicação. Elas não pertencem exclusivamente a uma rota, mas afetam o comportamento geral da API.

Na GoList, isso significa que um *handler* como `CreateList` pode continuar focado em criar uma lista de compras, enquanto middlewares cuidam de tarefas como log, autenticação, CORS, identificação da requisição e tratamento de falhas.

Essa separação torna o código mais organizado e reduz repetição. Em vez de copiar a mesma verificação para vários *handlers*, definimos um middleware uma vez e o aplicamos nas rotas que precisam daquele comportamento.

Nos próximos tópicos, veremos como combinar vários middlewares em uma cadeia e como implementar casos práticos que serão usados na API da **GoList**.

7.2 Cadeia de middlewares

Um único middleware já permite adicionar comportamento a um *handler*. Em uma API real, porém, normalmente precisamos aplicar vários middlewares à mesma requisição.

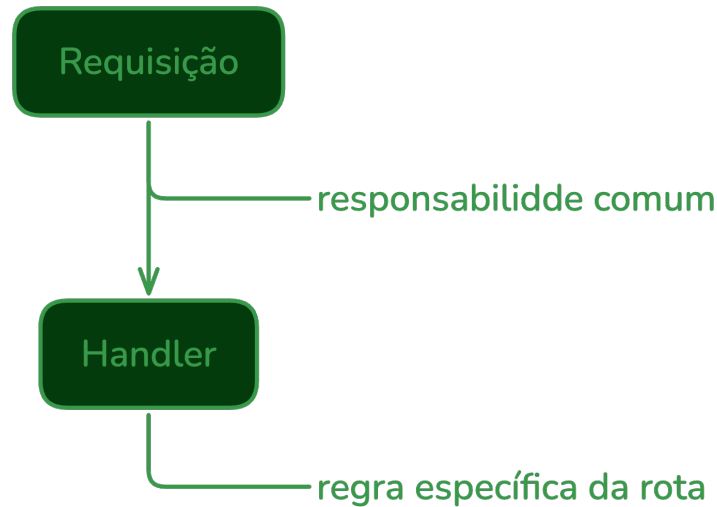


Figura 7.5: Middleware cuidam de tarefas comuns

Por exemplo, uma rota pode precisar:

- recuperar a aplicação de *panics*;
- adicionar cabeçalhos de CORS;
- adicionar um identificador único;
- registrar informações da requisição;
- verificar autenticação;
- executar o *handler* final.

Quando vários middlewares são combinados, formamos uma **cadeia de middlewares**.

Cada middleware recebe um `http.Handler`, envolve esse *handler* e retorna um novo `http.Handler`. Como todos seguem a mesma forma, eles podem ser encaixados uns nos outros.

7.2.1 Composição manual

Considere três middlewares simples.

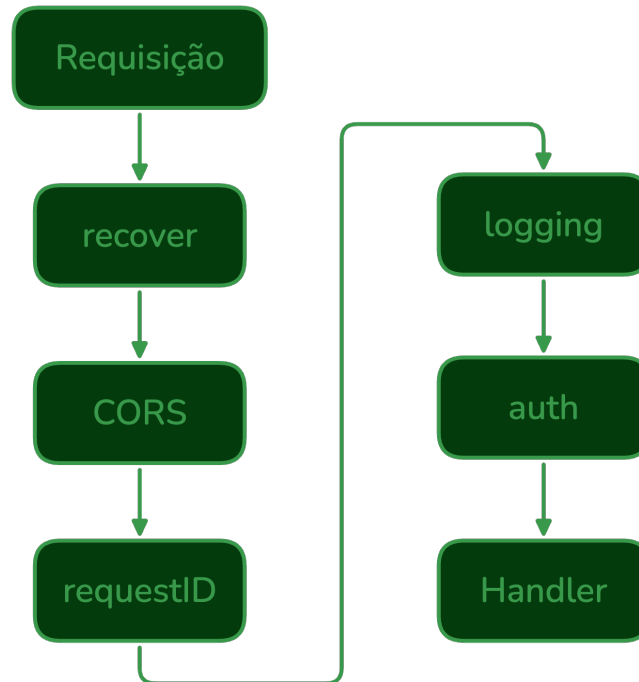


Figura 7.6: Cadeia de Middleware

```
1 func first(next http.Handler) http.Handler {
2     return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
3         fmt.Println("first: antes")
4         next.ServeHTTP(w, r)
5         fmt.Println("first: depois")
6     })
7 }
8
9 func second(next http.Handler) http.Handler {
10    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
11        fmt.Println("second: antes")
12        next.ServeHTTP(w, r)
13        fmt.Println("second: depois")
14    })
15 }
16
17 func third(next http.Handler) http.Handler {
18    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
19        fmt.Println("third: antes")
20        next.ServeHTTP(w, r)
21        fmt.Println("third: depois")
22    })
23 }
```

Agora considere um *handler* final.


```
1 func hello(w http.ResponseWriter, r *http.Request) {  
2     fmt.Println("handler")  
3     w.Write([]byte("Olá, mundo!"))  
4 }
```

Podemos combinar os middlewares manualmente.

```
1 handler := first(second(third(http.HandlerFunc(hello))))
```

O resultado é um único `http.Handler`. Esse valor pode ser registrado normalmente no `ServeMux`.

```
1 mux := http.NewServeMux()  
2 mux.Handle("GET /hello", handler)
```

Embora a composição pareça uma chamada de função comum, cada middleware está envolvendo o próximo.

```
first(  
    second(  
        third(  
            hello  
        )  
    )  
)
```

7.2.2 Ordem de execução

A ordem de execução é um ponto importante.

Com a composição:

```
1 handler := first(second(third(http.HandlerFunc(hello))))
```

O middleware mais externo é executado primeiro. Portanto, antes do *handler* final, a ordem será:

```
first: antes  
second: antes  
third: antes  
handler
```

Depois que o *handler* final termina, a execução volta pelo caminho inverso.

```
third: depois  
second: depois  
first: depois
```

A saída completa será:

```

first: antes
second: antes
third: antes
handler
third: depois
second: depois
first: depois

```

Essa ordem acontece porque cada middleware chama `next.ServeHTTP`. Quando essa chamada retorna, o middleware continua executando o código que vem depois dela.

```

first antes
  second antes
    third antes
      handler
    third depois
  second depois
first depois

```

Entender essa ordem é essencial para definir corretamente a posição de cada middleware.

7.2.3 A ordem importa

Nem todos os middlewares podem ser aplicados em qualquer ordem.

Um middleware de recuperação de *panics*, por exemplo, costuma ficar no início da cadeia. Assim, ele envolve todos os outros middlewares e também o *handler* final.

```

1 handler := recoverMiddleware(cors(requestID(logging(auth(http.HandlerFunc(createLis

```

Nesse caso, se `cors`, `requestID`, `logging`, `auth` ou `createList` causarem um *panic*, o `recoverMiddleware` ainda terá a chance de capturar o erro e produzir uma resposta adequada.

Um middleware de CORS costuma aparecer cedo na cadeia para que a API consiga responder requisições de navegador, inclusive requisições `OPTIONS` de *preflight*, antes de exigir autenticação da rota.

Já um middleware de autenticação normalmente precisa executar antes do *handler* protegido. Se o usuário não estiver autenticado, a requisição deve ser interrompida antes que a regra principal da rota seja executada.

Um middleware de *logging* pode ficar antes da autenticação se quisermos registrar todas as requisições, inclusive as rejeitadas. Também pode ficar depois se quisermos registrar apenas requisições que avançaram para as rotas protegidas. Quando existe um request ID, é comum gerá-lo antes do *logging*, para que o log já inclua esse identificador.

Não existe uma única ordem correta para todos os projetos. A ordem deve refletir o comportamento desejado pela aplicação.

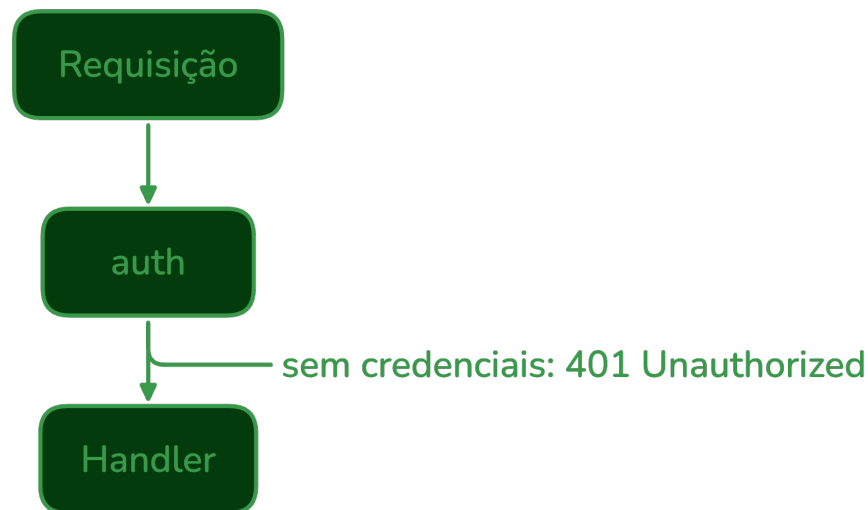


Figura 7.7: Middleware de autenticação

7.2.4 Criando um tipo para middleware

Para deixar o código mais legível, é comum definir um tipo para representar middlewares.

```
1 type Middleware func(http.Handler) http.Handler
```

Esse tipo não muda o funcionamento do código, mas deixa a intenção mais clara. Em vez de repetir a assinatura completa, podemos falar explicitamente em `MiddleWare`.

Com esse tipo, os middlewares continuam sendo escritos da mesma forma.

```
1 func logging(next http.Handler) http.Handler {  
2     return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
3         fmt.Println(r.Method, r.URL.Path)  
4         next.ServeHTTP(w, r)  
5     })  
6 }
```

Como `logging` possui a assinatura `func(http.Handler) http.Handler`, ele é compatível com o tipo `MiddleWare`.

7.2.5 Uma função para encadear middlewares

Compor middlewares manualmente funciona, mas a leitura pode ficar difícil quando a cadeia cresce.

```
1 handler := recoverMiddleware(cors(requestID(logging(auth(http.HandlerFunc(createLis
```

Uma alternativa é criar uma função auxiliar para aplicar vários middlewares a um *handler*.

```

1 func chain(handler http.Handler, middlewares ...Middleware) http.Handler {
2     for i := len(middlewares) - 1; i >= 0; i-- {
3         handler = middlewares[i](handler)
4     }
5
6     return handler
7 }

```

Essa função recebe o *handler* final e uma lista de middlewares. O laço percorre os middlewares de trás para frente para preservar a ordem de leitura.

Assim, este código:

```

1 handler := chain(
2     http.HandlerFunc(createList),
3     recoverMiddleware,
4     cors,
5     requestID,
6     logging,
7     auth,
8 )

```

produz a mesma estrutura que:

```

1 handler := recoverMiddleware(cors(requestID(logging(auth(http.HandlerFunc(createList))))))

```

A diferença é que a primeira forma é mais fácil de ler, especialmente quando a aplicação possui várias rotas. Os middlewares aparecem na chamada de `chain` na ordem em que a requisição entra na aplicação.

7.2.6 Aplicando ao ServeMux

Depois que a cadeia é montada, ela continua sendo apenas um `http.Handler`.

```

1 mux := http.NewServeMux()
2
3 mux.Handle("POST /lists", chain(
4     http.HandlerFunc(createList),
5     recoverMiddleware,
6     cors,
7     requestID,
8     logging,
9     auth,
10 ))

```

O `ServeMux` não precisa saber quantos middlewares existem. Para ele, o valor registrado é simplesmente um *handler* capaz de processar a rota.

Esse detalhe é uma das razões pelas quais middlewares se encaixam tão bem no pacote `net/http`. A aplicação inteira continua usando a mesma abstração: `http.Handler`.

7.2.7 Middlewares globais e por rota

Nem todos os middlewares precisam ser aplicados às mesmas rotas.

Alguns comportamentos fazem sentido para toda a aplicação. É o caso de recuperação de *panics*, CORS, identificação da requisição e *logging*.

```
1 common := []Middleware{
2     recoverMiddleware,
3     cors,
4     requestID,
5     logging,
6 }
```

Outros comportamentos dependem da rota. Um middleware de autenticação, por exemplo, pode ser necessário para criar listas, mas não para cadastrar um novo usuário ou realizar login.

```
1 protected := append([]Middleware{}, common...)
2 protected = append(protected, auth)
3
4 mux.Handle("POST /users", chain(
5     http.HandlerFunc(createUser),
6     common...,
7 ))
8
9 mux.Handle("POST /lists", chain(
10    http.HandlerFunc(createList),
11    protected...,
12 ))
```

Nesse exemplo, `common` representa os middlewares compartilhados pela aplicação, enquanto `protected` adiciona autenticação às rotas que precisam de usuário autenticado.

Esse exemplo mostra a ideia geral, mas também revela um cuidado importante: à medida que a aplicação cresce, a organização das rotas e dos grupos de middlewares precisa permanecer clara.

Na GoList, usaremos middlewares globais para responsabilidades comuns e middlewares específicos para rotas que exigem autenticação. Assim, cada rota recebe apenas os comportamentos necessários, sem duplicar lógica dentro dos *handlers*.

Nos próximos tópicos, implementaremos middlewares reais para *logging*, recuperação de falhas, CORS, request ID e autenticação.

7.3 Logging

Um dos usos mais comuns de middleware é registrar informações sobre cada requisição recebida pela API.

Esses registros ajudam a entender o comportamento da aplicação durante o desenvolvimento e são fundamentais para investigar problemas em produção. Ao observar os logs, conseguimos responder perguntas como:

- qual rota foi acessada?
- qual método HTTP foi utilizado?
- quanto tempo a requisição demorou?
- qual status HTTP foi retornado?
- de qual endereço veio a requisição?

Como essas informações são úteis para praticamente todas as rotas, o logging é um bom exemplo de responsabilidade transversal. Não faz sentido repetir o mesmo código de log dentro de cada *handler*.

Em vez disso, podemos criar um middleware.

7.3.1 Um primeiro middleware de log

O middleware mais simples registra a requisição antes de chamar o próximo *handler*.

```
func logging(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        log.Printf("%s %s", r.Method, r.URL.Path)  
  
        next.ServeHTTP(w, r)  
    })  
}
```

Esse middleware usa `log.Printf`, fornecido pela biblioteca padrão, para escrever uma linha no terminal.

Podemos aplicá-lo a uma rota normalmente.

```
mux := http.NewServeMux()  
  
mux.Handle("GET /lists", logging(http.HandlerFunc(listLists)))
```

Quando uma requisição GET `/lists` for recebida, uma mensagem semelhante será registrada.

```
GET /lists
```

Depois disso, `next.ServeHTTP(w, r)` permite que a requisição continue até o *handler* `listLists`.

Esse exemplo já mostra a ideia central, mas ainda registra poucas informações. Na prática, é comum registrar também a duração da requisição.

7.3.2 Medindo duração

Para medir quanto tempo uma requisição levou, podemos capturar o horário antes de chamar o próximo *handler* e calcular a diferença depois que ele terminar.

```
func logging(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        start := time.Now()  
  
        next.ServeHTTP(w, r)  
  
        duration := time.Since(start)  
  
        log.Printf("%s %s %s", r.Method, r.URL.Path, duration)  
    })  
}
```

Agora o log será escrito depois da execução do *handler*, pois só nesse momento sabemos quanto tempo a requisição levou.

Uma saída possível seria:

```
GET /lists 3.241ms
```

Esse tipo de informação ajuda a identificar rotas lentas e comparar o comportamento da aplicação em diferentes cenários.

7.3.3 Registrando o status da resposta

Outra informação importante é o código de status HTTP retornado pela rota.

O desafio é que `http.ResponseWriter` não expõe um método para consultar o status depois que ele foi enviado. O *handler* pode chamar `WriteHeader`, mas o *middleware* não tem acesso direto ao valor registrado.

Para resolver isso, podemos criar um tipo que envolve `http.ResponseWriter` e guarda o status informado pelo *handler*.

```
type statusResponseWriter struct {  
    http.ResponseWriter  
    status      int  
    wroteHeader bool  
}
```

Esse tipo incorpora `http.ResponseWriter`, ou seja, continua se comportando como um `ResponseWriter` comum. Em seguida, sobrescrevemos o método `WriteHeader`.

```
func (w *statusResponseWriter) WriteHeader(status int) {  
    if w.wroteHeader {  
        return  
    }  
  
    w.status = status  
    w.wroteHeader = true  
    w.ResponseWriter.WriteHeader(status)  
}
```

Na primeira vez que o *handler* chamar `WriteHeader`, nosso tipo registrará o status antes de repassar a chamada para o `ResponseWriter` original.

O campo `wroteHeader` impede que chamadas repetidas alterem o status registrado. Isso acompanha o comportamento do próprio pacote `net/http`, que considera apenas o primeiro status enviado.

Também precisamos sobrescrever o método `Write`.

```
func (w *statusResponseWriter) Write(data []byte) (int, error) {  
    if !w.wroteHeader {  
        w.WriteHeader(http.StatusOK)  
    }  
  
    return w.ResponseWriter.Write(data)  
}
```

Isso é necessário porque, em Go, se o *handler* escrever o corpo da resposta sem chamar `WriteHeader`, o servidor envia automaticamente o status **200 OK**. Ao registrar esse comportamento no nosso tipo, evitamos que uma chamada tardia a `WriteHeader` altere o status que será registrado no log.

Com isso, podemos atualizar o middleware.


```
func logging(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()

        rw := &statusResponseWriter{
            ResponseWriter: w,
            status:          http.StatusOK,
        }

        next.ServeHTTP(rw, r)

        duration := time.Since(start)

        log.Printf(
            "%s %s %d %s",
            r.Method,
            r.URL.Path,
            rw.status,
            duration,
        )
    })
}
```

Observe que passamos `rw` para `next.ServeHTTP`, e não mais `w`.

```
next.ServeHTTP(rw, r)
```

Assim, quando o *handler* escrever o cabeçalho ou o corpo da resposta, o middleware conseguirá guardar o status.

O valor inicial é `http.StatusOK` porque, em Go, se o *handler* escrever o corpo da resposta sem chamar `WriteHeader`, o status padrão será **200 OK**.

```
rw := &statusResponseWriter{
    ResponseWriter: w,
    status:        http.StatusOK,
}
```

Uma saída possível seria:

```
GET /lists 200 4.812ms
POST /lists 400 1.093ms
GET /lists/99 404 782µs
```

Agora os logs mostram método, caminho, status e duração da requisição.

7.3.4 Incluindo o endereço remoto

Também podemos registrar o endereço remoto informado pela requisição.

```
log.Printf(  
    "%s %s %d %s %s",  
    r.Method,  
    r.URL.Path,  
    rw.status,  
    duration,  
    r.RemoteAddr,  
)
```

O campo `RemoteAddr` representa o endereço de rede do cliente conectado ao servidor.

Em ambientes com *reverse proxy*, como Nginx, Caddy ou serviços de nuvem, esse valor pode representar o endereço do proxy, e não necessariamente o endereço original do usuário. Nesses casos, é comum consultar cabeçalhos como `X-Forwarded-For`, desde que eles sejam configurados de forma confiável pela infraestrutura.

Para os exemplos deste capítulo, `RemoteAddr` é suficiente.

7.3.5 Aplicando o middleware globalmente

Como o logging costuma ser útil para toda a API, ele geralmente é aplicado como middleware global.

```
mux := http.NewServeMux()  
  
mux.HandleFunc("GET /lists", listLists)  
mux.HandleFunc("POST /lists", createList)  
mux.HandleFunc("GET /lists/{id}", getList)  
  
handler := logging(mux)  
  
http.ListenAndServe(":8080", handler)
```

Nesse exemplo, o `ServeMux` inteiro é envolvido pelo middleware. Isso significa que todas as requisições que chegarem a esse *handler* passarão por `logging`.

Essa abordagem é útil para middlewares que devem valer para todas as rotas da aplicação.

Também seria possível aplicar o logging rota por rota, como vimos anteriormente. A escolha depende de como a aplicação está organizada, mas para logs de requisição costuma ser mais simples aplicá-lo globalmente.

7.3.6 Logging e erros

O middleware de logging não deve alterar o comportamento da requisição. Sua responsabilidade é observar o que aconteceu e registrar informações úteis.

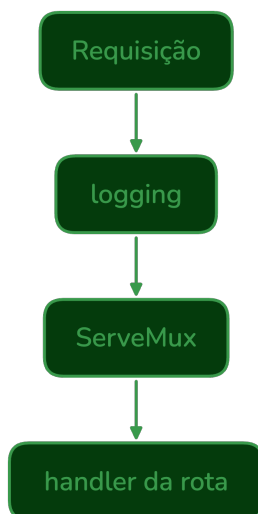


Figura 7.8: Middleware logging para todas as requisições

Por isso, ele normalmente:

- não valida dados;
- não decide se a requisição pode continuar;
- não modifica a resposta;
- não substitui o tratamento de erros da aplicação.

Ele apenas mede, registra e repassa a requisição.

```
next.ServeHTTP(rw, r)
```

Essa separação é importante. Um middleware de autenticação pode interromper a requisição. Um middleware de CORS pode adicionar cabeçalhos. Um middleware de recuperação pode tratar *panics*. Já o middleware de logging deve permanecer focado em registrar o fluxo da requisição.

7.3.7 Código completo

Uma versão completa do middleware de logging pode ser escrita assim:

```

type statusResponseWriter struct {
    http.ResponseWriter
    status      int
    wroteHeader bool
}

func (w *statusResponseWriter) WriteHeader(status int) {
    if w.wroteHeader {
        return
    }

    w.status = status
    w.wroteHeader = true
    w.ResponseWriter.WriteHeader(status)
}

func (w *statusResponseWriter) Write(data []byte) (int, error) {
    if !w.wroteHeader {
        w.WriteHeader(http.StatusOK)
    }

    return w.ResponseWriter.Write(data)
}

func logging(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()

        rw := &statusResponseWriter{
            ResponseWriter: w,
            status:         http.StatusOK,
        }

        next.ServeHTTP(rw, r)

        log.Printf(
            "%s %s %d %s %s",
            r.Method,
            r.URL.Path,
            rw.status,
            time.Since(start),
            r.RemoteAddr,
        )
    })
}

```

Na GoList, esse middleware será uma das camadas comuns da aplicação. Ele nos permitirá

acompanhar as requisições recebidas, observar códigos de resposta e identificar rotas que estejam demorando mais do que o esperado.

Mais adiante no livro, veremos como evoluir essa ideia usando logs estruturados, níveis de log e informações adicionais como request ID e user ID.

7.4 Recover

Em Go, erros esperados normalmente são representados por valores do tipo `error`. Uma entrada inválida, um usuário não encontrado ou uma senha incorreta são situações que fazem parte do funcionamento normal de uma aplicação e devem ser tratadas explicitamente.

Entretanto, algumas falhas podem provocar um `panic`.

Um `panic` interrompe a execução normal da função atual e começa a desfazer a pilha de chamadas. Se nada interceptar esse processo, a falha pode encerrar o fluxo da requisição de forma abrupta.

Em uma API HTTP, isso é perigoso porque uma falha inesperada em uma rota não deve comprometer o funcionamento do servidor. O cliente deve receber uma resposta adequada, normalmente **500 Internal Server Error**, e a aplicação deve registrar o problema para investigação.

É nesse contexto que usamos um middleware de **recover**.

7.4.1 O papel do recover

A função `recover` permite interceptar um `panic`, desde que seja chamada dentro de uma função registrada com `defer`.

De forma simplificada:

```
defer func() {  
    if r := recover(); r != nil {  
        // tratar o panic  
    }  
}()
```

Quando um `panic` acontece, Go executa as funções adiadas com `defer`. Nesse momento, `recover` pode capturar o valor associado ao `panic` e interromper sua propagação.

Em um middleware, essa técnica é especialmente útil porque podemos envolver todo o processamento da requisição.

Se algum middleware interno ou o *handler* final provocar um `panic`, o middleware de `recover` poderá capturá-lo.

7.4.2 Um handler com panic

Considere um *handler* com uma falha de programação.

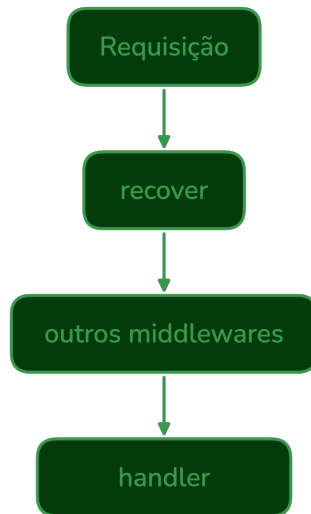


Figura 7.9: Middleware Recover

```
func broken(w http.ResponseWriter, r *http.Request) {  
    items := []string{"leite", "arroz"}  
  
    w.Write([]byte(items[10]))  
}
```

Esse código tenta acessar uma posição inexistente da slice. Ao receber uma requisição, a aplicação produzirá um panic.

```
panic: runtime error: index out of range [10] with length 2
```

Esse tipo de falha não deveria acontecer em uma aplicação correta, mas bugs existem. O middleware de recover atua como uma proteção na fronteira HTTP da aplicação.

7.4.3 Implementando o middleware

Uma primeira versão do middleware pode ser escrita assim:

```
func recoverMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if err := recover(); err != nil {
                log.Printf("panic recuperado: %v", err)

                http.Error(
                    w,
                    "Erro interno do servidor.",
                    http.StatusInternalServerError,
                )
            }
        }()

        next.ServeHTTP(w, r)
    })
}
```

O defer é registrado antes da chamada a `next.ServeHTTP`. Assim, qualquer panic que aconteça durante o processamento da requisição poderá ser interceptado.

```
next.ServeHTTP(w, r)
```

Se nenhum panic ocorrer, o middleware não altera a resposta. A requisição segue normalmente.

Se um panic ocorrer, o bloco adiado será executado, `recover` retornará o valor associado ao panic, o erro será registrado e uma resposta **500 Internal Server Error** será enviada ao cliente.

7.4.4 Registrando mais contexto

Registrar apenas o valor do panic pode não ser suficiente para entender a causa do problema. Em geral, é útil incluir informações da requisição.

```
log.Printf(
    "panic recuperado: method=%s path=%s error=%v",
    r.Method,
    r.URL.Path,
    err,
)
```

Com isso, o log indica qual rota estava sendo processada quando a falha aconteceu.

Uma versão um pouco mais completa seria:

```
func recoverMiddleware(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        defer func() {  
            if err := recover(); err != nil {  
                log.Printf(  
                    "panic recuperado: method=%s path=%s error=%v",  
                    r.Method,  
                    r.URL.Path,  
                    err,  
                )  
  
                http.Error(  
                    w,  
                    "Erro interno do servidor.",  
                    http.StatusInternalServerError,  
                )  
            }  
        }()  
  
        next.ServeHTTP(w, r)  
    })  
}
```

Essa implementação ainda é simples, mas já fornece informações melhores para depuração.

7.4.5 Stack trace

Em situações de erro inesperado, também pode ser útil registrar a pilha de chamadas (*stack trace*). Ela mostra o caminho percorrido pelo programa até o ponto onde o panic ocorreu.

A biblioteca padrão fornece essa informação por meio do pacote `runtime/debug`.

```
debug.Stack()
```

Podemos usar esse valor no log.


```
func recoverMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if err := recover(); err != nil {
                log.Printf(
                    "panic recuperado: method=%s path=%s error=%v\n%s",
                    r.Method,
                    r.URL.Path,
                    err,
                    debug.Stack(),
                )

                http.Error(
                    w,
                    "Erro interno do servidor.",
                    http.StatusInternalServerError,
                )
            }
        }()

        next.ServeHTTP(w, r)
    })
}
```

Essa versão é mais útil durante o desenvolvimento e em ambientes nos quais os logs são coletados para análise posterior.

O *stack trace* deve ser registrado internamente, mas não deve ser enviado ao cliente. A resposta HTTP deve continuar genérica.

Erro interno do servidor.

Expor detalhes internos da aplicação pode revelar caminhos de arquivos, nomes de funções e outras informações sensíveis.

7.4.6 Resposta JSON

Como a GoList utiliza JSON como formato padrão, a resposta de erro também pode seguir esse formato.

Uma estrutura simples seria:

```
type ErrorResponse struct {
    Error string `json:"error"`
}
```

E o middleware poderia escrever a resposta manualmente:

```
w.Header().Set("Content-Type", "application/json")
w.WriteHeader(http.StatusInternalServerError)

json.NewEncoder(w).Encode(ErrorResponse{
    Error: "Erro interno do servidor.",
})
```

O corpo enviado ao cliente seria:

```
{
  "error": "Erro interno do servidor."
}
```

Em uma aplicação maior, é comum centralizar essa lógica em uma função auxiliar, como `writeError`, para manter todas as respostas de erro no mesmo formato.

Essa resposta só será confiável se o *handler* ainda não tiver começado a escrever a resposta. Depois que cabeçalhos ou parte do corpo já foram enviados, o servidor não consegue voltar no tempo e trocar a resposta por um **500** limpo.

7.4.7 Ordem na cadeia

O middleware de `recover` normalmente deve ficar no início da cadeia, envolvendo todos os outros middlewares e o *handler* final.

```
handler := recoverMiddleware(cors(requestID(logging(auth(http.HandlerFunc(createList
```

Ou, usando uma função de encadeamento:

```
handler := chain(
    http.HandlerFunc(createList),
    recoverMiddleware,
    cors,
    requestID,
    logging,
    auth,
)
```

Com essa ordem, `recoverMiddleware` é o middleware mais externo. Isso permite capturar falhas que aconteçam em `cors`, `requestID`, `logging`, `auth` ou `createList`.

```
recover
  cors
    requestID
      logging
        auth
          createList
```

Se o `recover` fosse colocado depois de outros middlewares, ele não conseguiria capturar panics

que acontecessem antes dele.

7.4.8 Limites do recover

O middleware de recover é uma proteção importante, mas não transforma `panic` em mecanismo normal de tratamento de erros.

Erros esperados devem continuar sendo retornados e tratados explicitamente.

Por exemplo:

- JSON inválido deve gerar **400 Bad Request**;
- usuário não autenticado deve gerar **401 Unauthorized**;
- recurso inexistente deve gerar **404 Not Found**;
- falha inesperada deve gerar **500 Internal Server Error**.

Também é importante entender que `recover` captura `panics` que acontecem na mesma goroutine em que ele foi registrado. Se um *handler* iniciar uma nova goroutine e ela provocar um `panic`, esse middleware não conseguirá recuperá-lo diretamente. A nova goroutine precisaria ter sua própria proteção.

Por isso, o `recover` deve ser visto como uma última camada de proteção, não como substituto para código bem estruturado.

7.4.9 Código completo

Uma versão final do middleware pode combinar log, *stack trace* e resposta JSON.

```

type ErrorResponse struct {
    Error string `json:"error"`
}

func recoverMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if err := recover(); err != nil {
                log.Printf(
                    "panic recuperado: method=%s path=%s error=%v\n%s",
                    r.Method,
                    r.URL.Path,
                    err,
                    debug.Stack(),
                )

                w.Header().Set("Content-Type", "application/json")
                w.WriteHeader(http.StatusInternalServerError)

                json.NewEncoder(w).Encode(ErrorResponse{
                    Error: "Erro interno do servidor.",
                })
            }
        }()

        next.ServeHTTP(w, r)
    })
}

```

Na GoList, esse middleware será aplicado globalmente, no início da cadeia. Assim, falhas inesperadas em qualquer rota poderão ser registradas e convertidas em uma resposta HTTP consistente, sem expor detalhes internos ao cliente.

7.5 CORS

APIs HTTP frequentemente são consumidas por aplicações web executadas no navegador. Durante o desenvolvimento da GoList, por exemplo, podemos ter a API rodando em uma porta e o *frontend* em outra.

```

Frontend: http://localhost:5173
API:      http://localhost:8080

```

Embora ambos estejam na mesma máquina, esses endereços possuem **origens** diferentes, pois usam portas diferentes.

Para o navegador, a origem de uma página é formada por:

- esquema, como http ou https;

- domínio, como `localhost` ou `app.example.com`;
- porta, como `5173` ou `8080`.

Quando uma página carregada de uma origem tenta acessar uma API em outra origem, o navegador aplica uma política de segurança chamada **Same-Origin Policy**. Essa política restringe requisições entre origens diferentes para evitar que páginas maliciosas acessem recursos indevidos em nome do usuário.

O **CORS** (*Cross-Origin Resource Sharing*) é o mecanismo que permite ao servidor informar ao navegador quais origens estão autorizadas a ler as respostas da API.

7.5.1 O problema

Considere uma aplicação web carregada em:

```
http://localhost:5173
```

Ela tenta fazer uma requisição para:

```
http://localhost:8080/lists
```

O navegador envia a requisição incluindo um cabeçalho `Origin`.

```
Origin: http://localhost:5173
```

Esse cabeçalho informa ao servidor de qual origem aquela chamada partiu.

Se a resposta da API não incluir os cabeçalhos CORS adequados, o navegador bloqueará o acesso ao resultado, mesmo que o servidor tenha processado a requisição corretamente.

Isso é importante: CORS é uma regra aplicada pelo navegador. Ferramentas como `curl`, clientes HTTP de linha de comando ou outros servidores não seguem essa restrição da mesma forma.

7.5.2 Access-Control-Allow-Origin

O principal cabeçalho de CORS é `Access-Control-Allow-Origin`.

Ele informa quais origens podem ler a resposta no navegador.

```
Access-Control-Allow-Origin: http://localhost:5173
```

Com esse cabeçalho, o servidor permite que uma página carregada de `http://localhost:5173` leia a resposta.

Durante o desenvolvimento, é comum encontrar exemplos usando `*`.

```
Access-Control-Allow-Origin: *
```

Esse valor permite que qualquer origem leia a resposta no navegador. Embora possa ser conveniente em testes locais, deve ser usado com cuidado. Em APIs reais, normalmente é melhor permitir apenas origens conhecidas.

```
1 w.Header().Set("Access-Control-Allow-Origin", "http://localhost:5173")
```

Como esse cabeçalho deve aparecer em várias respostas, CORS é um bom candidato para middleware.

7.5.3 Um middleware simples de CORS

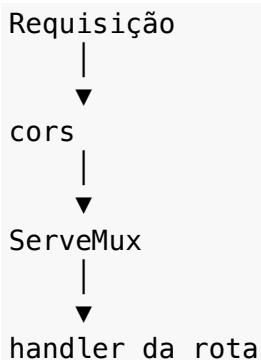
Uma primeira versão pode apenas adicionar o cabeçalho `Access-Control-Allow-Origin` antes de chamar o próximo *handler*.

```
1 func cors(next http.Handler) http.Handler {  
2     return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
3         w.Header().Set("Access-Control-Allow-Origin", "http://localhost:5173")  
4  
5         next.ServeHTTP(w, r)  
6     })  
7 }
```

Aplicando ao ServeMux:

```
1 mux := http.NewServeMux()  
2  
3 mux.HandleFunc("GET /lists", listLists)  
4 mux.HandleFunc("POST /lists", createList)  
5  
6 handler := cors(mux)  
7  
8 http.ListenAndServe(":8080", handler)
```

Agora todas as respostas produzidas pela API incluirão o cabeçalho CORS.



Essa implementação resolve alguns cenários simples, mas ainda não trata todos os casos comuns.

7.5.4 Métodos permitidos

Além da origem, o servidor pode informar quais métodos HTTP são permitidos em requisições de outra origem.

```
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
```

No middleware:

```
1 w.Header().Set(  
2     "Access-Control-Allow-Methods",  
3     "GET, POST, PUT, DELETE, OPTIONS",  
4 )
```

Esse cabeçalho é especialmente importante nas requisições de pré-verificação, conhecidas como *preflight requests*.

7.5.5 Headers permitidos

Também é comum informar quais cabeçalhos o cliente pode enviar.

```
Access-Control-Allow-Headers: Content-Type, Authorization
```

No middleware:

```
1 w.Header().Set(  
2     "Access-Control-Allow-Headers",  
3     "Content-Type, Authorization",  
4 )
```

Isso é necessário, por exemplo, quando o *frontend* envia JSON com `Content-Type: application/json` ou quando envia credenciais no cabeçalho `Authorization`.

7.5.6 Requisições preflight

Antes de algumas requisições, o navegador envia uma requisição `OPTIONS` para verificar se a chamada real é permitida.

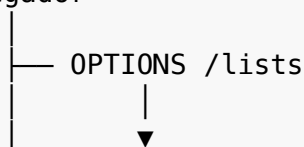
Esse processo é chamado de **preflight**.

Por exemplo, antes de enviar um `POST` com JSON para `/lists`, o navegador pode enviar:

```
OPTIONS /lists  
Origin: http://localhost:5173  
Access-Control-Request-Method: POST  
Access-Control-Request-Headers: Content-Type
```

O servidor deve responder com os cabeçalhos CORS adequados. Se a pré-verificação for aceita, o navegador envia a requisição real.

Navegador





No middleware, podemos tratar requisições OPTIONS diretamente.

```
1 func cors(next http.Handler) http.Handler {
2     return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
3         w.Header().Set("Access-Control-Allow-Origin", "http://localhost:5173")
4         w.Header().Set("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPT
5         w.Header().Set("Access-Control-Allow-Headers", "Content-Type, Authorization
6
7         if r.Method == http.MethodOptions {
8             w.WriteHeader(http.StatusNoContent)
9             return
10        }
11
12        next.ServeHTTP(w, r)
13    })
14 }
```

Quando a requisição é OPTIONS, o middleware responde com **204 No Content** e interrompe o processamento. O *handler* da rota não precisa ser executado.

Para os demais métodos, a requisição segue normalmente.

7.5.7 Permitindo origens específicas

Em vez de liberar uma única origem fixa, podemos permitir uma lista de origens conhecidas.

```
1 func isAllowedOrigin(origin string) bool {
2     allowed := map[string]bool{
3         "http://localhost:5173": true,
4         "https://app.golist.example": true,
5     }
6
7     return allowed[origin]
8 }
```

O middleware pode consultar o cabeçalho `Origin` recebido na requisição.


```

1  origin := r.Header.Get("Origin")
2
3  if isAllowedOrigin(origin) {
4      w.Header().Set("Access-Control-Allow-Origin", origin)
5      w.Header().Add("Vary", "Origin")
6  }

```

Observe que, nesse caso, o servidor devolve a própria origem recebida, desde que ela esteja na lista de origens permitidas.

O cabeçalho Vary: Origin informa a caches intermediários que a resposta pode variar de acordo com o valor do cabeçalho Origin da requisição.

Uma versão completa ficaria assim:

```

1  func cors(next http.Handler) http.Handler {
2      return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
3          origin := r.Header.Get("Origin")
4
5          if isAllowedOrigin(origin) {
6              w.Header().Set("Access-Control-Allow-Origin", origin)
7              w.Header().Add("Vary", "Origin")
8          }
9
10         w.Header().Set("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS")
11         w.Header().Set("Access-Control-Allow-Headers", "Content-Type, Authorization")
12
13         if r.Method == http.MethodOptions {
14             w.WriteHeader(http.StatusNoContent)
15             return
16         }
17
18         next.ServeHTTP(w, r)
19     })
20 }

```

Essa abordagem evita liberar a API para qualquer origem e mantém a configuração explícita.

7.5.8 Credenciais

Algumas aplicações enviam credenciais em requisições entre origens, como cookies ou cabeçalhos de autorização.

Quando isso é necessário, o servidor pode incluir:

```
Access-Control-Allow-Credentials: true
```

No middleware:

```
1 w.Header().Set("Access-Control-Allow-Credentials", "true")
```

Nesse caso, não se deve usar `Access-Control-Allow-Origin: *`. Quando credenciais estão envolvidas, a origem permitida deve ser informada explicitamente.

```
Access-Control-Allow-Origin: http://localhost:5173
```

```
Access-Control-Allow-Credentials: true
```

Essa restrição evita que respostas autenticadas sejam liberadas indiscriminadamente para qualquer origem.

7.5.9 CORS não é autenticação

É importante não confundir CORS com autenticação ou autorização.

CORS controla se o navegador permitirá que uma página de outra origem leia a resposta da API. Ele não substitui validação de usuário, token, sessão ou permissões.

Mesmo que uma origem não esteja autorizada pelo CORS, outros tipos de cliente ainda podem chamar a API diretamente. Por isso, rotas protegidas continuam precisando de autenticação.

CORS

└─ controla acesso do navegador à resposta

Autenticação

└─ identifica quem está fazendo a requisição

Autorização

└─ decide o que esse usuário pode acessar

Na GoList, CORS será usado para permitir que o *frontend* leia respostas da API a partir de origens conhecidas. A proteção das rotas será responsabilidade de middlewares de autenticação e das regras de autorização da aplicação.

7.5.10 Ordem na cadeia

O middleware de CORS geralmente deve aparecer cedo na cadeia, antes dos middlewares que podem interromper a requisição.

```
1 handler := chain(  
2     mux,  
3     recoverMiddleware,  
4     cors,  
5     requestID,  
6     logging,  
7 )
```

Essa posição garante que até respostas de erro ou requisições `OPTIONS` recebam os cabeçalhos necessários.

Quando a rota exigir autenticação, a cadeia pode incluir o middleware de autenticação depois de CORS.

```
1 handler := chain(  
2     http.HandlerFunc(createList),  
3     recoverMiddleware,  
4     cors,  
5     requestID,  
6     logging,  
7     auth,  
8 )
```

Assim, uma requisição OPTIONS de preflight pode ser respondida pelo middleware de CORS antes de chegar à autenticação. Isso é importante porque o preflight normalmente não representa a operação real da aplicação; ele apenas pergunta ao servidor se a chamada poderá ser feita pelo navegador.

7.5.11 Código completo

Uma implementação simples para a GoList pode ser:

```

1 func isAllowedOrigin(origin string) bool {
2     allowed := map[string]bool{
3         "http://localhost:5173": true,
4     }
5
6     return allowed[origin]
7 }
8
9 func cors(next http.Handler) http.Handler {
10    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
11        origin := r.Header.Get("Origin")
12
13        if isAllowedOrigin(origin) {
14            w.Header().Set("Access-Control-Allow-Origin", origin)
15            w.Header().Add("Vary", "Origin")
16        }
17
18        w.Header().Set("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS")
19        w.Header().Set("Access-Control-Allow-Headers", "Content-Type, Authorization")
20
21        if r.Method == http.MethodOptions {
22            w.WriteHeader(http.StatusNoContent)
23            return
24        }
25
26        next.ServeHTTP(w, r)
27    })
28 }

```

Esse middleware adiciona os cabeçalhos necessários, responde requisições de preflight e mantém a lista de origens permitidas explícita. Conforme a aplicação evoluir, a lista de origens poderá vir de uma variável de ambiente ou de uma configuração própria da aplicação.

7.6 Request ID

Em uma API HTTP, várias requisições podem ser processadas ao mesmo tempo. Em produção, isso significa que os logs de diferentes clientes, rotas e operações podem aparecer intercalados.

Considere uma sequência de logs como esta:

```

GET /lists 200 4ms
POST /lists 500 12ms
GET /users/10 200 3ms
POST /lists 500 8ms

```

Se um erro acontece em `POST /lists`, pode ser difícil identificar quais mensagens pertencem à mesma requisição. Conforme a aplicação cresce e passa a registrar logs em várias camadas, essa dificuldade aumenta.

Um **request ID** resolve esse problema associando um identificador único a cada requisição.

```
request_id=8f2c1a GET /lists 200 4ms
request_id=91b7de POST /lists 500 12ms
request_id=cc0a31 GET /users/10 200 3ms
request_id=91b7de erro ao criar lista
```

Agora, mesmo que os logs estejam misturados, podemos filtrar por `request_id=91b7de` e acompanhar tudo que aconteceu durante aquela requisição específica.

7.6.1 O que é um request ID?

Um request ID é um valor gerado, ou reaproveitado, no início do processamento da requisição e propagado ao longo da aplicação.

Ele pode ser usado para:

- correlacionar logs;
- identificar uma requisição em mensagens de erro;
- facilitar investigação de falhas;
- devolver ao cliente um identificador de suporte;
- conectar logs da API com logs de outros serviços.

O request ID normalmente é enviado também na resposta HTTP, por meio de um cabeçalho.

```
X-Request-ID: 8f2c1a9d4b6e7f00
```

Assim, se o cliente receber um erro, ele pode informar esse identificador para facilitar a análise.

7.6.2 Gerando um identificador

Para gerar o identificador, podemos usar a biblioteca padrão. Uma estratégia simples é criar bytes aleatórios com `crypto/rand` e convertê-los para texto hexadecimal com `encoding/hex`.

```
1 func newRequestID() (string, error) {
2     b := make([]byte, 16)
3
4     if _, err := rand.Read(b); err != nil {
5         return "", err
6     }
7
8     return hex.EncodeToString(b), nil
9 }
```

Essa função gera 16 bytes aleatórios e retorna uma string hexadecimal.

Um resultado possível seria:

```
9f0c7a2e4d8b41c3a6d5e7f8190ab234
```

Esse valor não precisa ter significado próprio. Ele serve apenas para identificar aquela requisição durante seu ciclo de vida.

7.6.3 Middleware de request ID

O middleware deve gerar o identificador, adicioná-lo à resposta e disponibilizá-lo para os próximos *handlers*.

Uma primeira versão pode apenas gerar o valor e adicioná-lo como cabeçalho.

```
1 func requestID(next http.Handler) http.Handler {
2     return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
3         id, err := newRequestID()
4         if err != nil {
5             http.Error(w, "Erro interno do servidor.", http.StatusInternalServerError)
6             return
7         }
8
9         w.Header().Set("X-Request-ID", id)
10
11        next.ServeHTTP(w, r)
12    })
13 }
```

Com isso, toda resposta gerada pela API passa a conter o cabeçalho X-Request-ID.

X-Request-ID: 9f0c7a2e4d8b41c3a6d5e7f8190ab234

Essa implementação já ajuda o cliente, mas ainda não permite que outros pontos da aplicação acessem o identificador com facilidade.

Para isso, podemos usar o contexto da requisição.

7.6.4 Contexto da requisição

Cada `http.Request` possui um contexto associado, acessado pelo método `Context()`.

```
1 ctx := r.Context()
```

O contexto pode carregar valores relacionados àquela requisição. Ele deve ser usado para dados que atravessam o fluxo da requisição, como request ID ou usuário autenticado. Para adicionar um valor, usamos `context.WithValue`.

```
1 ctx = context.WithValue(ctx, key, value)
```

Em seguida, criamos uma nova requisição com esse contexto.

```
1 r = r.WithContext(ctx)
```

Essa nova requisição é passada para o próximo *handler*.

```
1 next.ServeHTTP(w, r)
```

Assim, o request ID fica disponível para os demais middlewares e *handlers*.

7.6.5 Chave de contexto

Ao usar `context.WithValue`, é recomendável criar um tipo próprio para a chave. Isso evita colisões com valores adicionados por outros pacotes ou por outros middlewares da própria aplicação.

```
1 type requestIDContextKey struct{}
2
3 var requestIDKey = requestIDContextKey{}
```

Agora podemos armazenar o request ID no contexto.

```
1 ctx := context.WithValue(r.Context(), requestIDKey, id)
2 r = r.WithContext(ctx)
```

O middleware fica assim:

```
1 type requestIDContextKey struct{}
2
3 var requestIDKey = requestIDContextKey{}
4
5 func requestID(next http.Handler) http.Handler {
6     return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
7         id, err := newRequestID()
8         if err != nil {
9             http.Error(w, "Erro interno do servidor.", http.StatusInternalServerError)
10            return
11        }
12
13        w.Header().Set("X-Request-ID", id)
14
15        ctx := context.WithValue(r.Context(), requestIDKey, id)
16        r = r.WithContext(ctx)
17
18        next.ServeHTTP(w, r)
19    })
20 }
```

Agora qualquer código que receba a requisição pode acessar o valor.

```
1 id, _ := r.Context().Value(requestIDKey).(string)
```

7.6.6 Função auxiliar

Para evitar repetir a conversão de tipo em vários lugares, podemos criar uma função auxiliar.

```
1 func getRequestID(r *http.Request) string {
2     id, ok := r.Context().Value(requestIDKey).(string)
3     if !ok {
4         return ""
5     }
6
7     return id
8 }
```

Com isso, um *handler* pode consultar o identificador de forma mais clara.

```
1 func listLists(w http.ResponseWriter, r *http.Request) {
2     requestID := getRequestID(r)
3
4     log.Printf("request_id=%s listando listas", requestID)
5
6     // restante do handler
7 }
```

Essa função também protege o código caso o valor não exista no contexto.

7.6.7 Reaproveitando um ID recebido

Em algumas arquiteturas, um cliente ou outro serviço já envia um identificador no cabeçalho da requisição.

```
X-Request-ID: 4b7d1f9c
```

Nesse caso, a API pode reaproveitar esse valor em vez de gerar um novo.


```
1 func requestID(next http.Handler) http.Handler {
2     return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
3         id := r.Header.Get("X-Request-ID")
4
5         if id == "" {
6             var err error
7
8             id, err = newRequestID()
9             if err != nil {
10                 http.Error(w, "Erro interno do servidor.", http.StatusInternalServerError)
11                 return
12             }
13         }
14
15         w.Header().Set("X-Request-ID", id)
16
17         ctx := context.WithValue(r.Context(), requestIDKey, id)
18         r = r.WithContext(ctx)
19
20         next.ServeHTTP(w, r)
21     })
22 }
```

Essa abordagem ajuda a preservar o mesmo identificador quando uma requisição passa por múltiplos serviços.

Em aplicações públicas, também é comum validar o formato e limitar o tamanho do valor recebido antes de reutilizá-lo. Para os exemplos deste capítulo, manteremos a implementação simples.

7.6.8 Integrando com logging

O request ID se torna especialmente útil quando aparece nos logs.

Podemos atualizar o middleware de logging para consultar o identificador no contexto.

```
1 func logging(next http.Handler) http.Handler {
2     return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
3         start := time.Now()
4
5         next.ServeHTTP(w, r)
6
7         log.Printf(
8             "request_id=%s method=%s path=%s duration=%s",
9             getRequestID(r),
10            r.Method,
11            r.URL.Path,
12            time.Since(start),
13        )
14    })
15 }
```

Para que isso funcione, o middleware de request ID precisa executar antes do middleware de logging.

```
1 handler := chain(
2     mux,
3     recoverMiddleware,
4     cors,
5     requestID,
6     logging,
7 )
```

Com a função `chain` definida anteriormente, essa ordem significa:

```
recover
  cors
    requestID
      logging
        ServeMux
```

Assim, quando `logging` executar, o request ID já estará disponível no contexto da requisição.

7.6.9 Propagando para outros serviços

Quando uma API chama outro serviço HTTP, também pode encaminhar o request ID.

```
1 req, err := http.NewRequestWithContext(  
2     r.Context(),  
3     http.MethodGet,  
4     "http://example.com/resource",  
5     nil,  
6 )  
7 if err != nil {  
8     return err  
9 }  
10  
11 req.Header.Set("X-Request-ID", getRequestID(r))
```

Com isso, diferentes serviços podem registrar o mesmo identificador em seus logs, facilitando a investigação de uma operação que atravessa mais de uma aplicação.

Esse tipo de rastreamento será explorado com mais profundidade em capítulos posteriores sobre observabilidade.

7.6.10 Código completo

Uma implementação completa do middleware pode ser escrita assim:

```

1  type requestIDContextKey struct{}
2
3  var requestIDKey = requestIDContextKey{}
4
5  func newRequestID() (string, error) {
6      b := make([]byte, 16)
7
8      if _, err := rand.Read(b); err != nil {
9          return "", err
10     }
11
12     return hex.EncodeToString(b), nil
13 }
14
15 func getRequestID(r *http.Request) string {
16     id, ok := r.Context().Value(requestIDKey).(string)
17     if !ok {
18         return ""
19     }
20
21     return id
22 }
23
24 func requestID(next http.Handler) http.Handler {
25     return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
26         id := r.Header.Get("X-Request-ID")
27
28         if id == "" {
29             var err error
30
31             id, err = newRequestID()
32             if err != nil {
33                 http.Error(w, "Erro interno do servidor.", http.StatusInternalServerError)
34                 return
35             }
36         }
37
38         w.Header().Set("X-Request-ID", id)
39
40         ctx := context.WithValue(r.Context(), requestIDKey, id)
41         r = r.WithContext(ctx)
42
43         next.ServeHTTP(w, r)
44     })
45 }

```

Na GoList, esse middleware será aplicado globalmente. Ele permitirá correlacionar logs, devolver

um identificador ao cliente e preparar a aplicação para práticas mais avançadas de observabilidade.

7.7 Middleware de autenticação

Algumas rotas de uma API podem ser acessadas por qualquer cliente. Outras exigem que o usuário esteja autenticado.

Na GoList, por exemplo, criar um usuário ou realizar login são operações públicas. Já criar listas de compras, listar itens ou compartilhar uma lista são operações associadas a um usuário específico.

Rotas públicas

```
POST /users
POST /login
```

Rotas protegidas

```
GET /lists
POST /lists
GET /lists/{id}
```

Uma forma ruim de implementar essa proteção seria repetir a mesma verificação dentro de todos os *handlers* protegidos.

```
1 func createList(w http.ResponseWriter, r *http.Request) {
2     token := r.Header.Get("Authorization")
3     if token == "" {
4         http.Error(w, "Não autorizado.", http.StatusUnauthorized)
5         return
6     }
7
8     // restante do handler
9 }
```

Essa abordagem espalha a autenticação pela aplicação e mistura duas responsabilidades diferentes: verificar a identidade do usuário e executar a regra da rota.

Um middleware de autenticação resolve esse problema.

7.7.1 Responsabilidade do middleware

O middleware de autenticação fica antes do *handler* protegido.

Requisição

↓
auth

— inválida: 401 Unauthorized

▼ handler protegido

Sua responsabilidade é:

- ler as credenciais da requisição;
- verificar se elas são válidas;
- identificar o usuário;
- disponibilizar esse usuário para os próximos *handlers*;
- interromper a requisição quando a autenticação falhar.

O *handler* final, por sua vez, pode assumir que a requisição já foi autenticada.

7.7.2 O header Authorization

Credenciais de autenticação são frequentemente enviadas no header Authorization.

Um formato comum é:

```
Authorization: Bearer <token>
```

Por exemplo:

```
Authorization: Bearer abc123
```

O prefixo Bearer indica que o cliente está apresentando um token de acesso. O valor após o espaço é o token propriamente dito.

No *handler* ou middleware, podemos ler esse header com `r.Header.Get`.

```
1 authorization := r.Header.Get("Authorization")
```

Se o header estiver ausente, o usuário não está autenticado.

7.7.3 Extraindo o token

Antes de validar o token, precisamos verificar se o header está no formato esperado.

```
1 func bearerToken(r *http.Request) (string, bool) {
2     authorization := r.Header.Get("Authorization")
3     if authorization == "" {
4         return "", false
5     }
6
7     const prefix = "Bearer "
8
9     if !strings.HasPrefix(authorization, prefix) {
10        return "", false
11    }
12
13    token := strings.TrimSpace(strings.TrimPrefix(authorization, prefix))
14    if token == "" {
15        return "", false
16    }
17
18    return token, true
19 }
```

Essa função retorna o token e um valor booleano indicando se ele foi encontrado corretamente.

```
1 token, ok := bearerToken(r)
2 if !ok {
3     http.Error(w, "Não autorizado.", http.StatusUnauthorized)
4     return
5 }
```

Neste capítulo, ainda não entraremos nos detalhes de criação, assinatura e expiração de tokens. Esses assuntos serão tratados no capítulo de autenticação e autorização. Por enquanto, o objetivo é entender onde a verificação acontece dentro da cadeia de middlewares.

7.7.4 Representando o usuário autenticado

Depois de validar o token, a aplicação precisa saber qual usuário está associado àquela requisição.

Podemos representar isso com uma estrutura simples.

```
1 type AuthenticatedUser struct {
2     ID      int
3     Email   string
4 }
```

Em uma aplicação real, a validação do token poderia consultar uma sessão no banco de dados, verificar um JWT, consultar um cache ou chamar outro serviço.

Para manter o exemplo focado em middleware, vamos representar essa etapa com uma função.

```
1 func findUserByToken(token string) (*AuthenticatedUser, error) {
2     if token != "abc123" {
3         return nil, errors.New("token inválido")
4     }
5
6     return &AuthenticatedUser{
7         ID: 1,
8         Email: "maria@example.com",
9     }, nil
10 }
```

Essa função é apenas didática. Ela simula a validação de um token e retorna o usuário autenticado.

7.7.5 Middleware básico

Com essas peças, podemos montar uma primeira versão do middleware.

```
1 func auth(next http.Handler) http.Handler {
2     return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
3         token, ok := bearerToken(r)
4         if !ok {
5             http.Error(w, "Não autorizado.", http.StatusUnauthorized)
6             return
7         }
8
9         if _, err := findUserByToken(token); err != nil {
10             http.Error(w, "Não autorizado.", http.StatusUnauthorized)
11             return
12         }
13
14         next.ServeHTTP(w, r)
15     })
16 }
```

Esse middleware já protege a rota. Se o token estiver ausente ou inválido, a resposta será **401 Unauthorized** e o *handler* final não será executado.

Se o token for válido, a requisição segue.

Ainda falta, porém, disponibilizar o usuário autenticado para o restante da aplicação.

7.7.6 Usuário no contexto

Assim como fizemos com request ID, podemos armazenar o usuário autenticado no contexto da requisição.

Primeiro, definimos uma chave própria.


```

1  type authContextKey struct{}
2
3  var authenticatedUserKey = authContextKey{}

```

Usar um tipo específico evita colisões com valores adicionados por outros pacotes ou por outros middlewares, como o middleware de request ID.

Depois, adicionamos o usuário ao contexto.

```

1  ctx := context.WithValue(r.Context(), authenticatedUserKey, user)
2  r = r.WithContext(ctx)

```

O middleware passa a ser:

```

1  func auth(next http.Handler) http.Handler {
2      return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
3          token, ok := bearerToken(r)
4          if !ok {
5              http.Error(w, "Não autorizado.", http.StatusUnauthorized)
6              return
7          }
8
9          user, err := findUserByToken(token)
10         if err != nil {
11             http.Error(w, "Não autorizado.", http.StatusUnauthorized)
12             return
13         }
14
15         ctx := context.WithValue(r.Context(), authenticatedUserKey, user)
16         r = r.WithContext(ctx)
17
18         next.ServeHTTP(w, r)
19     })
20 }

```

Agora os *handlers* protegidos podem acessar o usuário autenticado.

7.7.7 Função auxiliar

Para evitar repetir a conversão de tipo, criamos uma função auxiliar.

```

1  func currentUser(r *http.Request) (*AuthenticatedUser, bool) {
2      user, ok := r.Context().Value(authenticatedUserKey).(*AuthenticatedUser)
3      return user, ok
4  }

```

Um *handler* protegido pode usar essa função.

```

1 func createList(w http.ResponseWriter, r *http.Request) {
2     user, ok := currentUser(r)
3     if !ok {
4         http.Error(w, "Não autorizado.", http.StatusUnauthorized)
5         return
6     }
7
8     fmt.Fprintf(w, "criando lista para o usuário %d", user.ID)
9 }

```

Como esse *handler* deve ser registrado atrás do middleware de autenticação, a ausência do usuário no contexto indicaria uma configuração incorreta da rota. Ainda assim, verificar o retorno de `currentUser` torna o código mais seguro.

7.7.8 Aplicando apenas em rotas protegidas

O middleware de autenticação não deve ser aplicado a todas as rotas.

Rotas públicas, como cadastro e login, precisam continuar acessíveis sem token.

```

1 mux := http.NewServeMux()
2
3 mux.HandleFunc("POST /users", createUser)
4 mux.HandleFunc("POST /login", login)

```

Já rotas protegidas devem ser envolvidas com `auth`.

```

1 mux.Handle("GET /lists", auth(http.HandlerFunc(listLists)))
2 mux.Handle("POST /lists", auth(http.HandlerFunc(createList)))

```

Também podemos combinar `auth` com os demais middlewares usando a função `chain`.

```

1 mux.Handle("POST /lists", chain(
2     http.HandlerFunc(createList),
3     recoverMiddleware,
4     cors,
5     requestID,
6     logging,
7     auth,
8 ))

```

Com essa ordem, a requisição passa pelos middlewares comuns e, antes de chegar ao *handler* `createList`, passa pela autenticação.

```

recover
  cors
    requestID

```

```
    logging
    auth
    createList
```

Se a autenticação falhar, `createList` não será executado.

7.7.9 401 e 403

Ao trabalhar com autenticação, é importante distinguir dois códigos de status.

O código **401 Unauthorized** indica que a requisição não possui credenciais válidas. Isso acontece quando o token está ausente, malformado, expirado ou inválido.

401 Unauthorized

O código **403 Forbidden** indica que o usuário foi autenticado, mas não possui permissão para executar aquela ação.

403 Forbidden

Por exemplo:

- sem token: **401 Unauthorized**;
- token inválido: **401 Unauthorized**;
- usuário autenticado tentando acessar lista de outro usuário sem permissão: **403 Forbidden**.

O middleware desta seção trata autenticação, portanto responde **401** quando não consegue identificar o usuário.

Regras de autorização serão tratadas mais adiante, quando a GoList precisar decidir quais usuários podem visualizar, editar ou compartilhar uma lista.

7.7.10 Resposta JSON

Como a GoList usa JSON nas respostas, podemos substituir `http.Error` por uma função auxiliar de erro.

```
1  type ErrorResponse struct {
2      Error string `json:"error"`
3  }
4
5  func writeError(w http.ResponseWriter, status int, message string) {
6      w.Header().Set("Content-Type", "application/json")
7      w.WriteHeader(status)
8
9      json.NewEncoder(w).Encode(ErrorResponse{
10         Error: message,
11     })
12 }
```

O middleware ficaria assim:

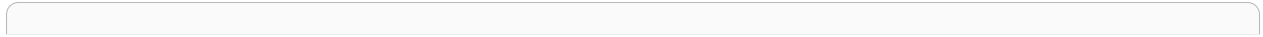
```
1 func auth(next http.Handler) http.Handler {
2     return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
3         token, ok := bearerToken(r)
4         if !ok {
5             writeError(w, http.StatusUnauthorized, "Não autorizado.")
6             return
7         }
8
9         user, err := findUserByToken(token)
10        if err != nil {
11            writeError(w, http.StatusUnauthorized, "Não autorizado.")
12            return
13        }
14
15        ctx := context.WithValue(r.Context(), authenticatedUserKey, user)
16        r = r.WithContext(ctx)
17
18        next.ServeHTTP(w, r)
19    })
20 }
```

O corpo da resposta será:

```
{
  "error": "Não autorizado."
}
```

7.7.11 Código completo

Uma versão completa, ainda didática, pode ser escrita assim:



```

1  type authContextKey struct{}
2
3  var authenticatedUserKey = authContextKey{}
4
5  type AuthenticatedUser struct {
6      ID      int
7      Email   string
8  }
9
10 type ErrorResponse struct {
11     Error string `json:"error"`
12 }
13
14 func writeError(w http.ResponseWriter, status int, message string) {
15     w.Header().Set("Content-Type", "application/json")
16     w.WriteHeader(status)
17
18     json.NewEncoder(w).Encode(ErrorResponse{
19         Error: message,
20     })
21 }
22
23 func bearerToken(r *http.Request) (string, bool) {
24     authorization := r.Header.Get("Authorization")
25     if authorization == "" {
26         return "", false
27     }
28
29     const prefix = "Bearer "
30
31     if !strings.HasPrefix(authorization, prefix) {
32         return "", false
33     }
34
35     token := strings.TrimSpace(strings.TrimPrefix(authorization, prefix))
36     if token == "" {
37         return "", false
38     }
39
40     return token, true
41 }
42
43 func findUserByToken(token string) (*AuthenticatedUser, error) {
44     if token != "abc123" {
45         return nil, errors.New("token inválido")
46     }
47
48     return &AuthenticatedUser{
49         ID:      1,
50         Email:   "maria@example.com",
51     }, nil
52 }
53

```

```
type Middleware func(http.Handler) http.Handler

func chain(handler http.Handler, middlewares ...Middleware) http.Handler {
    for i := len(middlewares) - 1; i >= 0; i-- {
        handler = middlewares[i](handler)
    }

    return handler
}
```

•
•
•
•

```
curl -i http://localhost:8080/lists
curl -i -X POST http://localhost:8080/lists
curl -i http://localhost:8080/rota-inexistente
```

- 1.
- 2.
- 3.
- 4.

```
curl -i http://localhost:8080/lists
```

```
curl -i \  
-H 'X-Request-ID: teste-golist-001' \  
http://localhost:8080/lists
```

•
•
•
•

```
func simulatePanic(w http.ResponseWriter, r *http.Request) {  
    panic("falha simulada")  
}
```

```
curl -i http://localhost:8080/debug/panic
```


-
-
-
-
-

```
curl -i -X OPTIONS \
-H 'Origin: http://localhost:5173' \
-H 'Access-Control-Request-Method: POST' \
-H 'Access-Control-Request-Headers: Content-Type, Authorization' \
http://localhost:8080/lists
```

POST /users

```
POST    /lists
GET     /lists
GET     /lists/{id}
PUT     /lists/{id}
DELETE  /lists/{id}
POST    /lists/{id}/items
GET     /items/{id}
PATCH  /items/{id}
DELETE  /items/{id}
```

```
# Sem credencial: deve retornar 401.
curl -i -X POST http://localhost:8080/lists

# Credencial inválida: deve retornar 401.
curl -i -X POST \
  -H 'Authorization: Bearer invalido' \
  http://localhost:8080/lists

# Credencial válida: deve chegar ao handler.
curl -i -X POST \
  -H 'Authorization: Bearer abc123' \
  http://localhost:8080/lists
```

recover → CORS → request ID → logging → autenticação → handler

```
common := []Middleware{
    recoverMiddleware,
    cors,
    requestID,
    logging,
}

protected := append([]Middleware{}, common...)
protected = append(protected, auth)
```

-
-
-
-
-
-

```
{  
  "name": ""  
}
```

-
-
-
-
-

```
{  
  "error": "erro"  
}
```

```
{  
  "error": "O campo name é obrigatório."  
}
```

Erro da aplicação



Status HTTP adequado



Resposta padronizada



Cliente

```
1 func findListByID(id int) (*List, error) {  
2     // ...  
3 }
```

```
1 list, err := findListByID(id)  
2 if err != nil {  
3     return err  
4 }
```

-
-
-
-
-

```
1 func findListByID(id int) (*List, error) {  
2     list, ok := lists[id]  
3     if !ok {  
4         return nil, ErrListNotFound  
5     }  
6  
7     return &list, nil  
8 }
```

•
•
•
•
•

```
{  
    "error": "Erro interno do servidor."  
}
```

```
1 var ErrListNotFound = errors.New("lista não encontrada")
```

```
1 func findListByID(id int) (*List, error) {
2     list, ok := lists[id]
3     if !ok {
4         return nil, ErrListNotFound
5     }
6
7     return &list, nil
8 }
```

```
1 list, err := findListByID(id)
2 if err != nil {
3     if errors.Is(err, ErrListNotFound) {
4         // responder 404
5         return
6     }
7
8     // responder 500
9     return
10 }
```

```
1 func getList(id int) (*List, error) {
2     list, err := findListByID(id)
3     if err != nil {
4         return nil, fmt.Errorf("buscar lista %d: %w", id, err)
5     }
6
7     return list, nil
8 }
```

```
1 list, err := getList(id)
2 if err != nil {
3     if errors.Is(err, ErrListNotFound) {
4         // ainda funciona
5     }
6 }
```

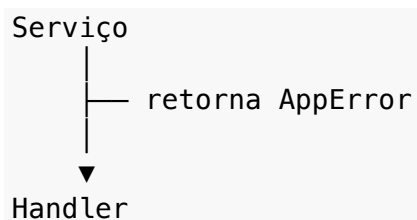
```
1 type AppError struct {
2     Code    string
3     Message string
4     Err     error
5 }
6
7 func (e *AppError) Error() string {
8     return e.Message
9 }
10
11 func (e *AppError) Unwrap() error {
12     return e.Err
13 }
```



```
1 func NewNotFound(message string, err error) *AppError {
2     return &AppError{
3         Code:    "not_found",
4         Message: message,
5         Err:      err,
6     }
7 }
```

```
1 func getList(id int) (*List, error) {
2     list, err := findListByID(id)
3     if err != nil {
4         if errors.Is(err, ErrListNotFound) {
5             return nil, NewNotFound("Lista não encontrada.", err)
6         }
7
8         return nil, fmt.Errorf("buscar lista %d: %w", id, err)
9     }
10
11     return list, nil
12 }
```

```
1 var appErr *AppError
2
3 if errors.As(err, &appErr) {
4     // appErr.Code
5     // appErr.Message
6 }
```

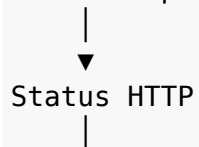




```
buscar lista 42: sql: no rows in result set
```

```
{  
  "error": "Lista não encontrada."  
}
```

Erro da aplicação





Resposta JSON

•
•
•
•
•

```
{  
  "name": ""  
}
```

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
```

```
{
  "error": "0 campo name é obrigatório."
}
```

-
-
-
-

```
GET /lists HTTP/1.1
Host: localhost:8080
```

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json
```

```
{
  "error": "Não autorizado."
}
```

-
-

```
GET /lists/42 HTTP/1.1
Authorization: Bearer token-da-maria
```

```
HTTP/1.1 403 Forbidden
Content-Type: application/json
```

```
{
  "error": "Você não tem permissão para acessar esta lista."
}
```

```
GET /lists/999 HTTP/1.1
```

```
HTTP/1.1 404 Not Found
Content-Type: application/json
```

```
{
  "error": "Lista não encontrada."
}
```

```
{
  "name": "Compras do mês"
}
```

```
HTTP/1.1 409 Conflict
Content-Type: application/json
```

```
{
  "error": "Já existe uma lista com esse nome."
}
```

-
-
-
-

```
HTTP/1.1 500 Internal Server Error  
Content-Type: application/json
```

```
{  
  "error": "Erro interno do servidor."  
}
```

```
1 func statusFromError(err error) int {
2     var appErr *AppError
3     if errors.As(err, &appErr) {
4         switch appErr.Code {
5             case "bad_request":
6                 return http.StatusBadRequest
7             case "unauthorized":
8                 return http.StatusUnauthorized
9             case "forbidden":
10                return http.StatusForbidden
11            case "not_found":
12                return http.StatusNotFound
13            case "conflict":
14                return http.StatusConflict
15        }
16    }
17
18    return http.StatusInternalServerError
19 }
```

```
{
  "error": "Lista não encontrada."
}
```

```
{
  "message": "Não autorizado."
}
```

JSON inválido.

```
1 type ErrorResponse struct {  
2     Error string `json:"error"`  
3 }
```

```
{  
  "error": "Lista não encontrada."  
}
```

```
1 type ErrorResponse struct {  
2     Code    string `json:"code"`  
3     Message string `json:"message"`  
4 }
```

```
{  
  "code": "not_found",  
  "message": "Lista não encontrada."  
}
```

-
-
-
-
-
-


```
1 type FieldError struct {  
2     Field    string `json:"field"`  
3     Message string `json:"message"`  
4 }
```

```
1 type ErrorResponse struct {  
2     Code    string    `json:"code"`  
3     Message string    `json:"message"`  
4     Details []FieldError `json:"details,omitempty"`  
5 }
```

```
{  
  "code": "validation_error",  
  "message": "A requisição possui campos inválidos.",  
  "details": [  
    {  
      "field": "name",  
      "message": "O campo name é obrigatório."  
    },  
    {  
      "field": "description",  
      "message": "O campo description deve possuir no máximo 500 caracteres."  
    }  
  ]  
}
```

```
{  
  "code": "not_found",  
  "message": "Lista não encontrada."  
}
```

```
1 type ErrorResponse struct {  
2     Code      string    `json:"code"`  
3     Message   string    `json:"message"`  
4     RequestID string    `json:"request_id,omitempty"`  
5     Details   []FieldError `json:"details,omitempty"`  
6 }
```

```
{  
  "code": "internal_error",  
  "message": "Erro interno do servidor.",  
  "request_id": "9f0c7a2e4d8b41c3a6d5e7f8190ab234"  
}
```

```

1  type FieldError struct {
2      Field    string `json:"field"`
3      Message  string `json:"message"`
4  }
5
6  type ErrorResponse struct {
7      Code      string `json:"code"`
8      Message   string `json:"message"`
9      RequestID string `json:"request_id,omitempty"`
10     Details   []FieldError `json:"details,omitempty"`
11 }
12
13 func writeError(
14     w http.ResponseWriter,
15     r *http.Request,
16     status int,
17     code string,
18     message string,
19     details []FieldError,
20 ) {
21     w.Header().Set("Content-Type", "application/json")
22     w.WriteHeader(status)
23
24     response := ErrorResponse{
25         Code:      code,
26         Message:   message,
27         RequestID: getRequestID(r),
28         Details:   details,
29     }
30
31     json.NewEncoder(w).Encode(response)
32 }

```

```

1  writeError(
2      w,
3      r,
4      http.StatusNotFound,
5      "not_found",
6      "Lista não encontrada.",
7      nil,
8  )

```

```
1  writeError(  
2      w,  
3      r,  
4      http.StatusBadRequest,  
5      "validation_error",  
6      "A requisição possui campos inválidos.",  
7      []FieldError{  
8          {  
9              Field:  "name",  
10             Message: "O campo name é obrigatório.",  
11         }},  
12     },  
13 )
```

```
1  writeError(  
2      w,  
3      r,  
4      http.StatusInternalServerError,  
5      "internal_error",  
6      "Erro interno do servidor.",  
7      nil,  
8  )
```

log: buscar lista 42: sql: connection refused

```
{  
  "code": "internal_error",  
  "message": "Erro interno do servidor.",  
  "request_id": "9f0c7a2e4d8b41c3a6d5e7f8190ab234"  
}
```

```
{  
  "code": "not_found",  
  "message": "Lista não encontrada.",  
}
```

```
"request_id": "9f0c7a2e4d8b41c3a6d5e7f8190ab234"
}

{
  "code": "validation_error",
  "message": "A requisição possui campos inválidos.",
  "request_id": "9f0c7a2e4d8b41c3a6d5e7f8190ab234",
  "details": [
    {
      "field": "name",
      "message": "O campo name é obrigatório."
    }
  ]
}
```

-
-
-

```
1 func getList(w http.ResponseWriter, r *http.Request) {
2     id, err := strconv.Atoi(r.PathValue("id"))
3     if err != nil {
4         writeError(
5             w,
6             r,
7             http.StatusBadRequest,
8             "bad_request",
9             "ID da lista inválido.",
10            nil,
11        )
12        return
13    }
14
15    list, err := service.GetList(id)
16    if err != nil {
17        writeError(
18            w,
19            r,
20            statusFromError(err),
21            codeFromError(err),
22            messageFromError(err),
23            nil,
24        )
25        return
26    }
27
28    json.NewEncoder(w).Encode(list)
29 }
```

```
1 func(w http.ResponseWriter, r *http.Request)
```

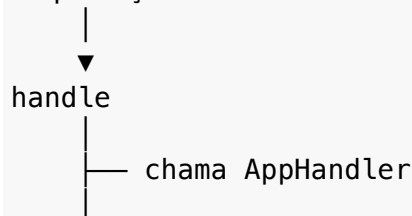
```
1 type AppHandler func(http.ResponseWriter, *http.Request) error
```

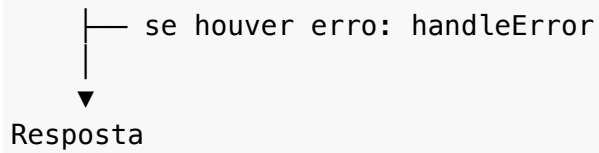
```
1 func getList(w http.ResponseWriter, r *http.Request) error {
2     id, err := strconv.Atoi(r.PathValue("id"))
3     if err != nil {
4         return NewBadRequest("ID da lista inválido.", err)
5     }
6
7     list, err := service.GetList(id)
8     if err != nil {
9         return err
10    }
11
12    w.Header().Set("Content-Type", "application/json")
13    return json.NewEncoder(w).Encode(list)
14 }
```

```
1 func handle(fn AppHandler) http.Handler {
2     return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
3         if err := fn(w, r); err != nil {
4             handleError(w, r, err)
5         }
6     })
7 }
8 }
```

```
1 mux.Handle("GET /lists/{id}", handle(getList))
```

Requisição





```
1 func handleError(w http.ResponseWriter, r *http.Request, err error) {
2     var appErr *AppError
3
4     if errors.As(err, &appErr) {
5         writeError(
6             w,
7             r,
8             statusFromCode(appErr.Code),
9             appErr.Code,
10            appErr.Message,
11            nil,
12        )
13        return
14    }
15
16    log.Printf(
17        "erro inesperado: request_id=%s error=%v",
18        getRequestID(r),
19        err,
20    )
21
22    writeError(
23        w,
24        r,
25        http.StatusInternalServerError,
26        "internal_error",
27        "Erro interno do servidor.",
28        nil,
29    )
30 }
```



```
1 func statusFromCode(code string) int {  
2     switch code {  
3     case "bad_request", "validation_error":  
4         return http.StatusBadRequest  
5     case "unauthorized":  
6         return http.StatusUnauthorized  
7     case "forbidden":  
8         return http.StatusForbidden  
9     case "not_found":  
10        return http.StatusNotFound  
11    case "conflict":  
12        return http.StatusConflict  
13    default:  
14        return http.StatusInternalServerError  
15    }  
16 }
```

```
1 func NewBadRequest(message string, err error) *AppError {
2     return &AppError{
3         Code:    "bad_request",
4         Message: message,
5         Err:      err,
6     }
7 }
8
9 func NewNotFound(message string, err error) *AppError {
10    return &AppError{
11        Code:    "not_found",
12        Message: message,
13        Err:      err,
14    }
15 }
16
17 func NewConflict(message string, err error) *AppError {
18    return &AppError{
19        Code:    "conflict",
20        Message: message,
21        Err:      err,
22    }
23 }
```

```
1 if errors.Is(err, ErrListNotFound) {
2     return NewNotFound("Lista não encontrada.", err)
3 }
```

```
1 type AppError struct {
2     Code    string
3     Message string
4     Details []FieldError
5     Err     error
6 }
```

```
1  if errors.As(err, &appErr) {  
2      writeError(  
3          w,  
4          r,  
5          statusFromCode(appErr.Code),  
6          appErr.Code,  
7          appErr.Message,  
8          appErr.Details,  
9      )  
10     return  
11 }
```

```
1  return &AppError{  
2      Code:    "validation_error",  
3      Message: "A requisição possui campos inválidos.",  
4      Details: []FieldError{  
5          {  
6              Field:    "name",  
7              Message: "O campo name é obrigatório.",  
8          },  
9      },  
10 }
```

•
•
•
•
•
•

```
1 func getList(w http.ResponseWriter, r *http.Request) error {
2     id, err := strconv.Atoi(r.PathValue("id"))
3     if err != nil {
4         return NewBadRequest("ID da lista inválido.", err)
5     }
6
7     list, err := service.GetList(id)
8     if err != nil {
9         return err
10    }
11
12    w.Header().Set("Content-Type", "application/json")
13    return json.NewEncoder(w).Encode(list)
14 }
```

```
1 func handleError(w http.ResponseWriter, r *http.Request, err error) {
2     // identifica o erro
3     // escolhe status
4     // registra logs
5     // escreve resposta padronizada
6 }
```

```
1 mux.Handle("GET /lists/{id}", chain(  
2     handle(getList),  
3     recoverMiddleware,  
4     cors,  
5     requestID,  
6     logging,  
7     auth,  
8 ))
```

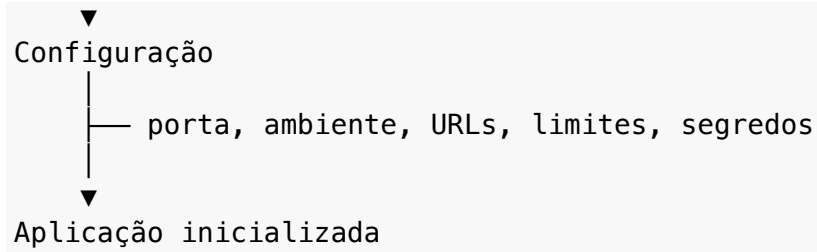

:8080

PORT=3000

-
-
-
-
-
-
-

Código da aplicação

├── regras, handlers, middlewares



-
-
-
-

-
-
-
-
-
-

```
1 http.ListenAndServe(":8080", handler)
```

```
1 allowedOrigin := "http://localhost:5173"
```



```
https://app.golist.example
```

-
-
-
-
-

-
-
-
-
-

```
1 port := os.Getenv("PORT")
2 databaseURL := os.Getenv("DATABASE_URL")
3 environment := os.Getenv("APP_ENV")
```

```
1 type Config struct {  
2     Port      string  
3     Environment string  
4     DatabaseURL string  
5     AllowedOrigin string  
6 }
```

```
1 cfg := LoadConfig()  
2  
3 server := &http.Server{  
4     Addr:    cfg.Port,  
5     Handler: handler,  
6 }
```

```
PORT=8080  
APP_ENV=development  
DATABASE_URL=postgres://user:pass@localhost:5432/golist
```

```
1 port := os.Getenv("PORT")
```

```
1 func main() {  
2     port := os.Getenv("PORT")  
3  
4     fmt.Println("PORT:", port)  
5 }
```

PORT:

PORT=8080 go run .

PORT: 8080

```
1 func envOrDefault(name string, fallback string) string {  
2     value := os.Getenv(name)  
3     if value == "" {  
4         return fallback  
5     }  
6  
7     return value  
8 }
```

```
1 port := envOrDefault("PORT", "8080")  
2 addr := ":" + port
```

```
1 http.ListenAndServe(addr, handler)
```

```
1 databaseURL := os.Getenv("DATABASE_URL")
2 if databaseURL == "" {
3     log.Fatal("DATABASE_URL não foi configurada")
4 }
```

```
1 func requiredEnv(name string) string {
2     value := os.Getenv(name)
3     if value == "" {
4         log.Fatalf("%s não foi configurada", name)
5     }
6
7     return value
8 }
```

```
1 databaseURL := requiredEnv("DATABASE_URL")
```

```
1 value, ok := os.LookupEnv("APP_ENV")
2 if !ok {
3     fmt.Println("APP_ENV não foi definida")
4 }
```

```
1 func envOrDefault(name string, fallback string) string {
2     value, ok := os.LookupEnv(name)
3     if !ok || value == "" {
4         return fallback
5     }
6
7     return value
8 }
```

```
READ_TIMEOUT_SECONDS=5
```

```
1 func envIntOrDefault(name string, fallback int) int {
2     value := os.Getenv(name)
3     if value == "" {
4         return fallback
5     }
6
7     n, err := strconv.Atoi(value)
8     if err != nil {
9         log.Fatalf("%s deve ser um número inteiro", name)
10    }
11
12    return n
13 }
```

```
1 seconds := envIntOrDefault("READ_TIMEOUT_SECONDS", 5)
2 readTimeout := time.Duration(seconds) * time.Second
```

```
ENABLE_DEBUG=true
```

```
1 func envBoolOrDefault(name string, fallback bool) bool {
2     value := os.Getenv(name)
3     if value == "" {
4         return fallback
5     }
6
7     enabled, err := strconv.ParseBool(value)
8     if err != nil {
9         log.Fatalf("%s deve ser true ou false", name)
10    }
11
12    return enabled
13 }
```

```
1 type Config struct {
2     Port          string
3     Environment    string
4     AllowedOrigin string
5 }
6
7 func LoadConfig() Config {
8     return Config{
9         Port:          envOrDefault("PORT", "8080"),
10        Environment:   envOrDefault("APP_ENV", "development"),
11        AllowedOrigin: envOrDefault("ALLOWED_ORIGIN", "http://localhost:5173"),
12    }
13 }
```

```
1  cfg := LoadConfig()
2
3  addr := ":" + cfg.Port
4
5  server := &http.Server{
6      Addr:    addr,
7      Handler: handler,
8  }
```

```
1  func cors(allowedOrigin string) Middleware {
2      return func(next http.Handler) http.Handler {
3          return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
4              w.Header().Set("Access-Control-Allow-Origin", allowedOrigin)
5
6              next.ServeHTTP(w, r)
7          })
8      }
9  }
```

•
•
•
•
•
•

```
go run . -port=8080 -env=development
```

```
1 func main() {  
2     port := flag.String("port", "8080", "porta do servidor HTTP")  
3  
4     flag.Parse()  
5  
6     fmt.Println("porta:", *port)  
7 }
```

•
•
•

```
go run .
```

```
porta: 8080
```

```
go run . -port=3000
```

```
porta: 3000
```



```
1 flag.Parse()
```

```
1 port := flag.String("port", "8080", "porta do servidor HTTP")
2 env := flag.String("env", "development", "ambiente da aplicação")
3
4 flag.Parse()
5
6 fmt.Println(*port, *env)
```

```
1 port := flag.String("port", "8080", "porta do servidor HTTP")
2 debug := flag.Bool("debug", false, "habilita modo debug")
3 timeout := flag.Int("timeout", 5, "timeout em segundos")
```

```
1 flag.Parse()
2
3 fmt.Println("port:", *port)
4 fmt.Println("debug:", *debug)
5 fmt.Println("timeout:", *timeout)
```

```
go run . -port=3000 -debug=true -timeout=10
```

```
port: 3000
debug: true
timeout: 10
```

```
go run . -debug
```

```
go run . -h
```

Usage of ./golist:

```
-debug          habilita modo debug
-env string     ambiente da aplicação (default "development")
-port string    porta do servidor HTTP (default "8080")
```

- 1.
- 2.
- 3.

```
valor padrão < variável de ambiente < flag
```

```
1 func main() {
2     defaultPort := envOrDefault("PORT", "8080")
3
4     port := flag.String("port", defaultPort, "porta do servidor HTTP")
5
6     flag.Parse()
7
8     addr := ":" + *port
9
10    http.ListenAndServe(addr, handler)
11 }
```

```
go run .
```

```
PORT=3000 go run .
```

```
PORT=3000 go run . -port=9090
```

```
1 func LoadConfig(args []string) Config {  
2     flags := flag.NewFlagSet("golist", flag.ExitOnError)  
3  
4     port := flags.String("port", envOrDefault("PORT", "8080"), "porta do servidor H  
5     env := flags.String("env", envOrDefault("APP_ENV", "development"), "ambiente da  
6  
7     flags.Parse(args)  
8  
9     return Config{  
10         Port:      *port,  
11         Environment: *env,  
12     }  
13 }
```

```
1 cfg := LoadConfig(os.Args[1:])
```

```
1  type Config struct {
2      Port      string
3      Environment string
4      AllowedOrigin string
5  }
6
7  func LoadConfig(args []string) Config {
8      flags := flag.NewFlagSet("golist", flag.ExitOnError)
9
10     port := flags.String(
11         "port",
12         envOrDefault("PORT", "8080"),
13         "porta do servidor HTTP",
14     )
15
16     environment := flags.String(
17         "env",
18         envOrDefault("APP_ENV", "development"),
19         "ambiente da aplicação",
20     )
21
22     allowedOrigin := flags.String(
23         "allowed-origin",
24         envOrDefault("ALLOWED_ORIGIN", "http://localhost:5173"),
25         "origem permitida para CORS",
26     )
27
28     flags.Parse(args)
29
30     return Config{
31         Port:      *port,
32         Environment: *environment,
33         AllowedOrigin: *allowedOrigin,
34     }
35 }
```

```
go run .
```

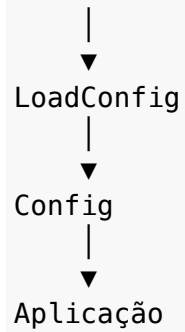
```
PORT=3000 APP_ENV=production go run .
```

```
go run . -port=3000 -env=production -allowed-origin=https://app.golist.example
```

-
-
-
-
-

```
1  type Config struct {  
2      Port      string  
3      Environment string  
4      AllowedOrigin string  
5  }
```

Ambiente e flags



```
1 port := os.Getenv("PORT")
2 origin := os.Getenv("ALLOWED_ORIGIN")
3 env := os.Getenv("APP_ENV")
```

•
•
•
•
•

```
1 cfg := LoadConfig(os.Args[1:])
2
3 server := NewServer(cfg)
```

```
1 type Config struct {
2     Port      string
3     Environment string
4     AllowedOrigin string
5     ReadTimeout time.Duration
6 }
```

•
•
•
•

```
1  type Config struct {  
2      Port      string  
3      Environment string  
4      AllowedOrigin string  
5      ReadTimeout time.Duration  
6      DatabaseURL string  
7  }
```

```
1 func LoadConfig(args []string) Config {
2     flags := flag.NewFlagSet("golist", flag.ExitOnError)
3
4     port := flags.String(
5         "port",
6         envOrDefault("PORT", "8080"),
7         "porta do servidor HTTP",
8     )
9
10    environment := flags.String(
11        "env",
12        envOrDefault("APP_ENV", "development"),
13        "ambiente da aplicação",
14    )
15
16    allowedOrigin := flags.String(
17        "allowed-origin",
18        envOrDefault("ALLOWED_ORIGIN", "http://localhost:5173"),
19        "origem permitida para CORS",
20    )
21
22    readTimeoutSeconds := flags.Int(
23        "read-timeout",
24        envIntOrDefault("READ_TIMEOUT_SECONDS", 5),
25        "timeout de leitura em segundos",
26    )
27
28    flags.Parse(args)
29
30    return Config{
31        Port:          *port,
32        Environment:    *environment,
33        AllowedOrigin:  *allowedOrigin,
34        ReadTimeout:    time.Duration(*readTimeoutSeconds) * time.Second,
35    }
36 }
```

•
•
•


```
1 func (c Config) Validate() error {
2     if c.Port == "" {
3         return errors.New("port não pode ser vazia")
4     }
5
6     if c.Environment == "" {
7         return errors.New("environment não pode ser vazio")
8     }
9
10    if c.ReadTimeout <= 0 {
11        return errors.New("read timeout deve ser maior que zero")
12    }
13
14    return nil
15 }
```

```
1 cfg := LoadConfig(os.Args[1:])
2
3 if err := cfg.Validate(); err != nil {
4     log.Fatalf("configuração inválida: %v", err)
5 }
```

```
1 func (c Config) Addr() string {
2     return ":" + c.Port
3 }
```

```
1 server := &http.Server{
2     Addr:      cfg.Addr(),
3     Handler:    handler,
4     ReadTimeout: cfg.ReadTimeout,
5 }
```

```
1 handler := chain(
2     mux,
3     recoverMiddleware,
4     cors(cfg.AllowedOrigin),
5     requestID,
6     logging,
7 )
```

```
1 server := &http.Server{
2     Addr:      cfg.Addr(),
3     Handler:    handler,
4     ReadTimeout: cfg.ReadTimeout,
5 }
```

```
1 var cfg Config
```

```
1 fmt.Println(cfg.Environment)
```

```
1 func main() {
2     cfg := LoadConfig(os.Args[1:])
3
4     if err := cfg.Validate(); err != nil {
5         log.Fatalf("configuração inválida: %v", err)
6     }
7
8     handler := NewHandler(cfg)
9
10    server := &http.Server{
11        Addr:      cfg.Addr(),
12        Handler:   handler,
13        ReadTimeout: cfg.ReadTimeout,
14    }
15
16    log.Fatal(server.ListenAndServe())
17 }
```

```
1 func TestLoadConfigWithFlags(t *testing.T) {
2     cfg := LoadConfig([]string{
3         "-port=3000",
4         "-env=test",
5         "-allowed-origin=http://localhost:5173",
6     })
7
8     if cfg.Port != "3000" {
9         t.Fatalf("port = %q", cfg.Port)
10    }
11
12    if cfg.Environment != "test" {
13        t.Fatalf("environment = %q", cfg.Environment)
14    }
15 }
```

```
1 func TestLoadConfigWithEnv(t *testing.T) {  
2     t.Setenv("PORT", "3000")  
3  
4     cfg := LoadConfig([]string{})  
5  
6     if cfg.Port != "3000" {  
7         t.Fatalf("port = %q", cfg.Port)  
8     }  
9 }
```

•
•
•

```
1 type Config struct {  
2     Port      string  
3     Environment string  
4     AllowedOrigin string  
5     ReadTimeout time.Duration  
6 }
```

```
APP_ENV=development
```

```
•  
•  
•  
•  
•
```

```
PORT=8080
```

```
APP_ENV=development
```

```
ALLOWED_ORIGIN=http://localhost:5173
```

```
APP_ENV=test
```

```
•  
•  
•  
•  
•
```

```
APP_ENV=production
```

```
•  
•  
•
```

-
-
-

```
1  const (  
2      EnvDevelopment = "development"  
3      EnvTest        = "test"  
4      EnvProduction  = "production"  
5  )
```

```
1  if cfg.Environment == "production" {  
2      // ...  
3  }
```

```
1  if cfg.Environment == EnvProduction {  
2      // ...  
3  }
```

```
1 func (c Config) Validate() error {
2     if c.Port == "" {
3         return errors.New("port não pode ser vazia")
4     }
5
6     switch c.Environment {
7     case EnvDevelopment, EnvTest, EnvProduction:
8     default:
9         return fmt.Errorf("environment inválido: %s", c.Environment)
10    }
11
12    if c.ReadTimeout <= 0 {
13        return errors.New("read timeout deve ser maior que zero")
14    }
15
16    return nil
17 }
```

APP_ENV=production

```
1 func (c Config) IsDevelopment() bool {
2     return c.Environment == EnvDevelopment
3 }
4
5 func (c Config) IsTest() bool {
6     return c.Environment == EnvTest
7 }
8
9 func (c Config) IsProduction() bool {
10    return c.Environment == EnvProduction
11 }
```

```
1 if cfg.IsProduction() {
2     log.Println("iniciando em produção")
3 }
```

```
1 func defaultAllowedOrigin(env string) string {
2     if env == EnvProduction {
3         return ""
4     }
5
6     return "http://localhost:5173"
7 }
```

```
1 allowedOrigin := envOrDefault(
2     "ALLOWED_ORIGIN",
3     defaultAllowedOrigin(environment),
4 )
```

```
1 if cfg.IsProduction() {
2     // regra especial aqui
3 } else {
4     // outra regra ali
5 }
```



```
1 func (c Config) Validate() error {
2     if c.Port == "" {
3         return errors.New("port não pode ser vazia")
4     }
5
6     switch c.Environment {
7     case EnvDevelopment, EnvTest, EnvProduction:
8     default:
9         return fmt.Errorf("environment inválido: %s", c.Environment)
10    }
11
12    if c.IsProduction() && c.AllowedOrigin == "" {
13        return errors.New("allowed origin deve ser configurada em produção")
14    }
15
16    if c.ReadTimeout <= 0 {
17        return errors.New("read timeout deve ser maior que zero")
18    }
19
20    return nil
21 }
```

```
1 func LoadConfig(args []string) Config {
2     flags := flag.NewFlagSet("golist", flag.ExitOnError)
3
4     environment := flags.String(
5         "env",
6         envOrDefault("APP_ENV", EnvDevelopment),
7         "ambiente da aplicação",
8     )
9
10    allowedOriginDefault := defaultAllowedOrigin(*environment)
11
12    allowedOrigin := flags.String(
13        "allowed-origin",
14        envOrDefault("ALLOWED_ORIGIN", allowedOriginDefault),
15        "origem permitida para CORS",
16    )
17
18    port := flags.String(
19        "port",
20        envOrDefault("PORT", "8080"),
21        "porta do servidor HTTP",
22    )
23
24    flags.Parse(args)
25
26    return Config{
27        Port:      *port,
28        Environment: *environment,
29        AllowedOrigin: *allowedOrigin,
30    }
31 }
```

```
1  if cfg.IsProduction() {  
2      timeout = 10 * time.Second  
3  } else {  
4      timeout = 1 * time.Second  
5  }
```

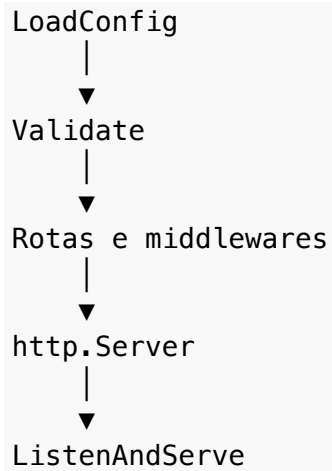
```
READ_TIMEOUT_SECONDS=10
```

•
•
•
•

•
•
•
•
•
•
•
•

main





```
1 func main() {  
2     cfg := LoadConfig(os.Args[1:])  
3  
4     if err := cfg.Validate(); err != nil {  
5         log.Fatalf("configuração inválida: %v", err)  
6     }  
7  
8     // restante da inicialização  
9 }
```

```
1 func newMux() *http.ServeMux {  
2     mux := http.NewServeMux()  
3  
4     mux.HandleFunc("GET /health", health)  
5     mux.HandleFunc("GET /lists", listLists)  
6     mux.HandleFunc("POST /lists", createList)  
7     mux.HandleFunc("GET /lists/{id}", getList)  
8  
9     return mux  
10 }
```

```
1 mux := newMux()
```

```
1 handler := chain(  
2     mux,  
3     recoverMiddleware,  
4     cors(cfg.AllowedOrigin),  
5     requestID,  
6     logging,  
7 )
```

```
1 http.ListenAndServe(":8080", handler)
```

```
1 server := &http.Server{
2     Addr:      cfg.Addr(),
3     Handler:    handler,
4     ReadTimeout: cfg.ReadTimeout,
5 }
```

```
1 type Config struct {
2     Port      string
3     Environment string
4     ReadTimeout time.Duration
5     WriteTimeout time.Duration
6     IdleTimeout time.Duration
7 }
```

```
1 server := &http.Server{
2     Addr:      cfg.Addr(),
3     Handler:    handler,
4     ReadTimeout: cfg.ReadTimeout,
5     WriteTimeout: cfg.WriteTimeout,
6     IdleTimeout:  cfg.IdleTimeout,
7 }
```

```
1 func NewServer(cfg Config, handler http.Handler) *http.Server {
2     return &http.Server{
3         Addr:      cfg.Addr(),
4         Handler:    handler,
5         ReadTimeout: cfg.ReadTimeout,
6     }
7 }
```

```
1 server := NewServer(cfg, handler)
```

```
1 func main() {
2     cfg := LoadConfig(os.Args[1:])
3
4     if err := cfg.Validate(); err != nil {
5         log.Fatalf("configuração inválida: %v", err)
6     }
7
8     mux := newMux()
9
10    handler := chain(
11        mux,
12        recoverMiddleware,
13        cors(cfg.AllowedOrigin),
14        requestID,
15        logging,
16    )
17
18    server := NewServer(cfg, handler)
19
20    log.Printf(
21        "iniciando servidor: addr=%s env=%s",
22        server.Addr,
23        cfg.Environment,
24    )
25
26    if err := server.ListenAndServe(); err != nil {
27        log.Fatalf("servidor encerrado com erro: %v", err)
28    }
29 }
```

•
•
•
•
•
•

-
-
-
-
-

```
1 log.Printf("ambiente: %s", cfg.Environment)
2 log.Printf("endereço HTTP: %s", cfg.Addr())
```

```
1 log.Printf("database url: %s", cfg.DatabaseURL)
```

LoadConfig	-> testável sem servidor
newMux	-> testável com httptest
NewServer	-> testável sem ListenAndServe
main	-> apenas coordena

•
•
•
•
•
•
•
•
•
•

-
-
-

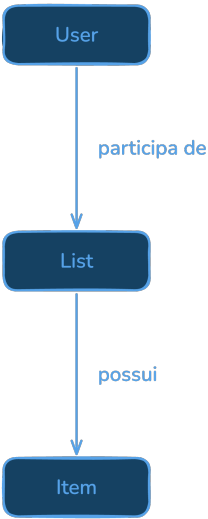


Figura A.1: Entidades da GoList

User

|— cria listas

└─ participa de listas compartilhadas

ShoppingList

└─ possui itens
└─ possui membros
└─ gera eventos

ShoppingItem

└─ pode ser marcado como comprado
└─ pode possuir anexos

Event

└─ representa alterações relevantes na aplicação

•
•
•
•
•
•
•

Usuários

POST /users
GET /users
GET /users/{id}
PUT /users/{id}
DELETE /users/{id}

Listas

```
POST    /lists
GET      /lists
GET      /lists/{id}
PUT      /lists/{id}
DELETE   /lists/{id}
```

Itens

```
POST    /lists/{id}/items
GET      /items/{id}
PATCH   /items/{id}
DELETE   /items/{id}
```

```
{
  "data": {
    "id": "list_123",
    "name": "Compras da semana"
  }
}
```

```
{
  "data": [
    {
      "id": "list_123",
      "name": "Compras da semana"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10
  }
}
```

```
{
  "error": {
```

```
    "code": "VALIDATION_ERROR",  
    "message": "Nome da lista é obrigatório."  
  }  
}
```

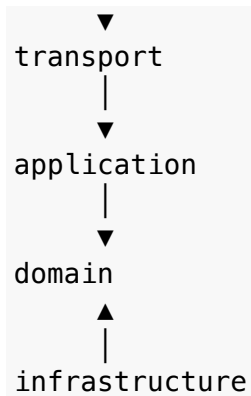
```
golist/  
├── go.mod  
└── main.go
```

```
golist/  
├── go.mod  
├── main.go  
├── routes.go  
├── users.go  
├── lists.go  
└── items.go
```

transport → application → domain ← infrastructure

Cliente HTTP

|



```
golist/
├── cmd/
│   └── go/
│       └── main.go
├── internal/
│   ├── config/
│   ├── middleware/
│   ├── platform/
│   │   ├── database/
│   │   ├── logger/
│   │   └── server/
│   └── modules/
│       ├── auth/
│       │   ├── transport/
│       │   ├── application/
│       │   ├── domain/
│       │   └── infrastructure/
│       └── users/
│           ├── transport/
│           ├── application/
│           ├── domain/
│           └── infrastructure/
```

```
— lists/
  — transport/
  — application/
  — domain/
  — infrastructure/

— items/
  — transport/
  — application/
  — domain/
  — infrastructure/

— sharing/
  — transport/
  — application/
  — domain/
  — infrastructure/

— attachments/
  — transport/
  — application/
  — domain/
  — infrastructure/

— events/
  — transport/
  — application/
  — domain/
  — infrastructure/

— realtime/
  — transport/
  — application/
  — domain/
  — infrastructure/

— migrations/
— tests/
— Dockerfile
— docker-compose.yml
— go.mod
— go.sum
```

-
-
-
-

-
-

-

-

-

-

-

-

-
-
-
-

-
-
-
-
-
-
-

-
-
-